

claude-windows / 6cee04e1-f4f1-4593-86c9-2e3ca3473efd

Metadata

- Source: `claude-windows`
- Kind: `claude`
- Source file: `C:\Users\avidu\.claude\projects\C--Users-avidu-Projects-badminton-highlight-indexer\6cee04e1-f4f1-4593-86c9-2e3ca3473efd\subagents\workflows\wf\_49c1018a-e67\agent-a76622fef6ffe10f2.jsonl`
- SHA-256: `cb61e8576632783d0378d0c718fe29d2072da7975cb1ddee2ae21b7059602930`
- Source modified: `2026-06-18T01:36:50+00:00`
- Imported at: `2026-07-05T16:48:26+00:00`
- project: `wf\_49c1018a-e67`
- session_id: `6cee04e1-f4f1-4593-86c9-2e3ca3473efd`

Transcript

1. user (2026-06-18T01:33:12.221Z)

Adversarial bug-hunter, mature Python repo (badminton-highlight-indexer). Read READ-ONLY in this worktree (origin/master), DO NOT modify:

```
C:/Users/avidu/Projects/badminton-highlight-indexer/.claude/worktrees/hunt-base
```

A prior precision pass found ZERO bugs; you cross-cut by BUG CLASS across ALL files to catch what an area-by-area pass missed. Report a finding ONLY if you READ the code and a CONCRETE trigger reaches it and a maintainer would call it a real defect. Include exact file+line, the trigger, why it's wrong, severity, minimal fix, confidence. PRECISION over recall ? empty list is a valid, respected answer; do NOT manufacture findings. EXCLUDE backend/eval/ablation.py, backend/eval/fusion_compare.py, backend/eval/fusion_audio.py, scripts/nightly_regression.py.

CLASS: subprocess / file / DB resource & error handling. ffmpeg/ffprobe/NVENC calls, open() without context manager, sqlite connections/cursors, temp files. Look for unchecked returncode, leaked handles, swallowed stderr, paths with spaces unquoted, decode errors.

2. assistant (2026-06-18T01:33:14.492Z)

I'll start by exploring the repository structure to understand the codebase, then hunt for subprocess/file/DB resource and error handling bugs.

3. user (2026-06-18T01:33:15.981Z)

```
4bd9d2d fix(eval): ablation litmus tolerates a mixed-schema golden corpus (#209)
HEAD
```

4. assistant (2026-06-18T01:33:18.854Z)

Let me explore the codebase structure to find subprocess, file, and DB operations.

5. user (2026-06-18T01:33:20.880Z)

Found 17 files

```
.claude\worktrees\hunt-base\training\gen0\harness.py
.claude\worktrees\hunt-base\tools\cleanup_caches.py
.claude\worktrees\hunt-base\tests\test_video.py
.claude\worktrees\hunt-base\tests\test_import_smoke.py
.claude\worktrees\hunt-base\scripts\nightly_regression.py
.claude\worktrees\hunt-base\scripts\doctor.py
.claude\worktrees\hunt-base\run_all_tests.py
.claude\worktrees\hunt-base\backend\utils\video.py
.claude\worktrees\hunt-base\backend\utils\run_telemetry.py
.claude\worktrees\hunt-base\backend\tools\rally_slicer.py
.claude\worktrees\hunt-base\backend\pipeline\validators\pts_continuity.py
```

```
.claude\worktrees\hunt-base\backend\pipeline\validators\format.py
.claude\worktrees\hunt-base\backend\pipeline\stitcher\compiler.py
.claude\worktrees\hunt-base\backend\pipeline\segmenters\gemini.py
.claude\worktrees\hunt-base\backend\pipeline\detectors\base.py
.claude\worktrees\hunt-base\backend\pipeline\audio\hit_detector.py
.claude\worktrees\hunt-base\backend\main.py
```

6. user (2026-06-18T01:33:22.232Z)

Found 5 files

```
.claude\worktrees\hunt-base\tests\test_reliability_hardening.py
.claude\worktrees\hunt-base\tests\test_job_worker_loop.py
.claude\worktrees\hunt-base\tests\test_database.py
.claude\worktrees\hunt-base\tests\test_async_jobs.py
.claude\worktrees\hunt-base\backend\infrastructure\database.py
```

7. user (2026-06-18T01:33:25.646Z)

Found 28 files

```
.claude\worktrees\hunt-base\backend\utils\run_telemetry.py
.claude\worktrees\hunt-base\backend\utils\hashing.py
.claude\worktrees\hunt-base\backend\tools\rally_slicer.py
.claude\worktrees\hunt-base\backend\pipeline\stitcher\compiler.py
.claude\worktrees\hunt-base\backend\pipeline\detectors\wasb_infer.py
.claude\worktrees\hunt-base\backend\pipeline\detectors\tracknet_runner.py
.claude\worktrees\hunt-base\backend\pipeline\audio\hit_detector.py
.claude\worktrees\hunt-base\backend\main.py
.claude\worktrees\hunt-base\backend\eval\splits.py
.claude\worktrees\hunt-base\backend\eval\serve_contrast.py
.claude\worktrees\hunt-base\backend\eval\served_gate_a.py
.claude\worktrees\hunt-base\backend\eval\rally_seq_proto.py
.claude\worktrees\hunt-base\backend\eval\regression.py
.claude\worktrees\hunt-base\backend\eval\rally_seg_eval.py
.claude\worktrees\hunt-base\backend\eval\per_user_regression.py
.claude\worktrees\hunt-base\backend\eval\gt_loader.py
.claude\worktrees\hunt-base\backend\eval\golden_fixtures.py
.claude\worktrees\hunt-base\backend\eval\fusion_golden.py
.claude\worktrees\hunt-base\backend\eval\fusion_audio.py
.claude\worktrees\hunt-base\backend\eval\fusion_compare.py
.claude\worktrees\hunt-base\backend\eval\experiment.py
.claude\worktrees\hunt-base\backend\eval\export_wasb_segments.py
.claude\worktrees\hunt-base\backend\eval\classifier.py
.claude\worktrees\hunt-base\backend\eval\distill_local.py
.claude\worktrees\hunt-base\backend\eval\calibrate_local.py
.claude\worktrees\hunt-base\backend\eval\annotations.py
.claude\worktrees\hunt-base\backend\eval\ablation.py
.claude\worktrees\hunt-base\backend\config\loader.py
```

8. user (2026-06-18T01:33:26.972Z)

Found 16 files

```
.claude\worktrees\hunt-base\backend\utils\video.py
.claude\worktrees\hunt-base\backend\utils\run_telemetry.py
.claude\worktrees\hunt-base\backend\tools\rally_slicer.py
.claude\worktrees\hunt-base\backend\reporting\harvest.py
.claude\worktrees\hunt-base\backend\pipeline\validators\resolution_fps.py
.claude\worktrees\hunt-base\backend\pipeline\validators\pts_continuity.py
.claude\worktrees\hunt-base\backend\pipeline\validators\format.py
.claude\worktrees\hunt-base\backend\pipeline\validators\base.py
.claude\worktrees\hunt-base\backend\pipeline\stitcher\compiler.py
.claude\worktrees\hunt-base\backend\pipeline\segmenters\yolo_hybrid.py
.claude\worktrees\hunt-base\backend\pipeline\segmenters\gemini.py
.claude\worktrees\hunt-base\backend\pipeline\detectors\wasb_infer.py
.claude\worktrees\hunt-base\backend\pipeline\audio\__init__.py
```

```
.claude\worktrees\hunt-base\backend\pipeline\audio\hit_detector.py
.claude\worktrees\hunt-base\backend\main.py
.claude\worktrees\hunt-base\backend\eval\fusion_audio.py
```

9. assistant (2026-06-18T01:33:29.284Z)

Let me start with the core subprocess-using files. The ffmpeg/ffprobe wrappers in `video.py` and the stitcher `compiler.py` are prime candidates.

10. user (2026-06-18T01:33:30.160Z)

```
1     import subprocess
2     import logging
3     from typing import Optional
4
5     logger = logging.getLogger(__name__)
6
7     def get_video_duration(video_path: str) -> float:
8         """Returns the exact duration of a video file in seconds using ffprobe.
9         Returns 0.0 on failure."""
10        try:
11            duration_output = subprocess.check_output([
12                "ffprobe", "-v", "error", "-show_entries", "format=duration",
13                "-of", "default=noprint_wrappers=1:nokey=1", video_path
14            ]).decode().strip()
15            return float(duration_output)
16        except Exception as e:
17            logger.error(f"Could not detect video duration for {video_path}: {e}")
18            return 0.0
19
20    def extract_video_chunk(video_path: str, start_time: float, duration: float,
21                           output_path: str, reencode: bool = False,
22                           scale_width: Optional[int] = None) -> bool:
23        """Extracts a chunk from a video file.
24
25        Args:
26            reencode: If True, re-encodes with libx264 (slower but frame-accurate).
27                      If False, uses stream-copy (fast but cuts at nearest keyframe).
28                      Use True for short clips where keyframe drift matters.
29            scale_width: If set (requires reencode), downscale to this width (height auto,
30                        even). Useful for high-res sources headed to an API that analyses at
31                        low res anyway ? shrinks uploads with no analysis loss.
32
33        """
34        cmd = [
35            "ffmpeg", "-y",
36            "-ss", str(start_time),
37            "-t", str(duration),
38            "-i", video_path,
39        ]
40        if reencode:
41            if scale_width:
42                cmd += ["-vf", f"scale={int(scale_width)}:-2"]
43            cmd += ["-c:v", "libx264", "-preset", "ultrafast", "-crf", "23", "-c:a", "aac"]
44        else:
45            cmd += ["-c", "copy"]
46        cmd.append(output_path)
47        try:
48            subprocess.run(cmd, check=True, stdout=subprocess.DEVNULL,
49                           stderr=subprocess.DEVNULL)
50            return True
51        except Exception as e:
52            logger.error(f"Failed to extract chunk {start_time}s - {start_time+duration}s: {e}")
53            return False
54
```

11. assistant (2026-06-18T01:33:32.571Z)

Now let me look at the stitcher compiler and rally_slicer which do more complex ffmpeg work.

12. user (2026-06-18T01:33:33.291Z)

```
1     import os
2     import subprocess
3     import tempfile
4     import logging
5     from pathlib import Path
6     from typing import Any, Dict, List, Optional, Tuple, Union
7
8     logger = logging.getLogger(__name__)
9
10    # Bundled, commercially-clean default watermark font (Apache-2.0). Shipped so the
overlay
11    # renders out-of-the-box without a system fontconfig setup (a common gap on Windows,
where
12    # `font='Sans` fails). Overridable via stitching.watermark.font_path. This module lives
at
13    # backend/pipeline/stitcher/, so parents[2] == backend/.
14    _BUNDLED_FONT = Path(__file__).resolve().parents[2] / "assets" / "fonts" / "RobotoSlab-
Regular.ttf"
15
16    class VideoCompiler:
17        def __init__(self, output_dir: str, end_padding: float = 1.0, begin_padding: float =
0.5,
18                    watermark: Optional[Any] = None):
19            self.output_dir = output_dir
20            self.end_padding = end_padding
21            self.begin_padding = begin_padding
22            # Optional branding overlay. Accepts a WatermarkConfig model or a plain dict;
23            # normalized to a dict (or None) so this module stays config-package-agnostic.
24            self.watermark = self._normalize_watermark(watermark)
25            os.makedirs(output_dir, exist_ok=True)
26
27        @staticmethod
28        def _normalize_watermark(watermark: Optional[Any]) -> Optional[Dict[str, Any]]:
29            if watermark is None:
30                return None
31            if hasattr(watermark, "model_dump"):
32                return watermark.model_dump()
33            if isinstance(watermark, dict):
34                return dict(watermark)
35            return None
36
37        @staticmethod
38        def _ff_quote(path: str) -> str:
39            """Make a filesystem path safe to embed inside single quotes in an ffmpeg
40            filtergraph operand (drawtext fontfile/textfile, movie= source): forward
41            slashes (Windows-friendly), escaped single quotes, AND escaped colons.
42
43            The colon MUST be escaped even inside single quotes: ffmpeg's filtergraph
44            option parser treats ':' as the option separator, so a Windows drive-letter
45            path like ``C:/dir/font.ttf`` otherwise truncates the operand at ``C`` and
46            the encode fails with "Invalid argument". This silently disabled the
47            on-by-default bundled-font watermark (and absolute logo paths) on Windows."""
48            return path.replace("\\", "/").replace("'", r"\'").replace(":", r"\:")
49
50        @staticmethod
51        def _parse_time(val: Union[int, float, str]) -> float:
52            """Normalizes a timestamp value to float seconds.
53            Handles: float/int (pass-through), plain numeric strings ("12.4"),
54            and colon-separated strings ("MM:SS", "HH:MM:SS", "H:MM:SS.ms").
```

```

55 This mirrors the parse_time logic used in gemini.py to ensure
56 the stitcher can always consume whatever format the indexer produced."""
57 if isinstance(val, (int, float)):
58     return float(val)
59 if isinstance(val, str):
60     val = val.strip()
61     if ":" in val:
62         # Handle MM:SS, HH:MM:SS, etc.
63         parts = val.split(":")
64         parts.reverse()
65         total = 0.0
66         for i, p in enumerate(parts):
67             try:
68                 total += float(p) * (60 ** i)
69             except ValueError:
70                 pass
71         return total
72     try:
73         return float(val)
74     except ValueError:
75         logger.warning(f"Could not parse timestamp value '{val}' as a number.
Defaulting to 0.0.")
76         return 0.0
77         logger.warning(f"Unexpected timestamp type {type(val).__name__} for value
'{val}'. Defaulting to 0.0.")
78         return 0.0
79
80 @staticmethod
81 def _merge_overlapping(segments: List[Tuple[float, float]]) -> List[Tuple[float,
float]]:
82     """Normalize, sort, and merge overlapping/duplicate (start, end) ranges.
83
84     A highlight reel must never replay the same footage. The same rally can land in
85     the segment list more than once ? e.g. two detector runs accumulated for one
86     video (add_video is an UPSERT, so re-running without a prune leaves both sets),
87     or
88     chunk-overlap duplicates near boundaries. Stitching those verbatim plays the
89     rally
90     twice. Merging any range that overlaps or abuts the previous one collapses true
91     duplicates/overlaps while preserving genuinely distinct (gapped) rallies.
92     Degenerate rows (end <= start) are dropped. Timestamps may be float/int/str.
93     """
94     parsed = []
95     for raw_s, raw_e in segments:
96         s = VideoCompiler._parse_time(raw_s)
97         e = VideoCompiler._parse_time(raw_e)
98         if e > s:
99             parsed.append((s, e))
100     parsed.sort()
101     merged: List[Tuple[float, float]] = []
102     for s, e in parsed:
103         if merged and s <= merged[-1][1]: # overlaps/abuts previous -> extend it
104             merged[-1] = (merged[-1][0], max(merged[-1][1], e))
105         else:
106             merged.append((s, e))
107     return merged
108
109 def _get_video_duration(self, video_path: str) -> float:
110     """Probes the video file to get its total duration in seconds.
111     Returns 0.0 if it cannot be determined."""
112     try:
113         result = subprocess.run(
114             [
115                 "ffprobe", "-v", "error",
116                 "-show_entries", "format=duration",

```

```

115         "-of", "default=noprint_wrappers=1:nokey=1",
116         video_path
117     ],
118     capture_output=True, text=True, check=True
119 )
120     return float(result.stdout.strip())
121 except Exception as e:
122     logger.warning(f"Could not determine video duration via ffprobe: {e}")
123     return 0.0
124
125 def _watermark_filter(self, tmp_dir: str) -> Optional[str]:
126     """Build the ffmpeg -vf filtergraph that burns the configured watermark (text
127     and/or logo PNG) into a clip during the per-clip re-encode. Returns None when
128     watermarking is disabled or has nothing to draw. The concat stage stays -c copy
129     because the mark is already baked into every clip identically."""
130     wm = self.watermark
131     if not wm or not wm.get("enabled"):
132         return None
133
134     text = (wm.get("text") or "").strip()
135     logo_path = (wm.get("logo_path") or "").strip()
136     if logo_path and not os.path.exists(logo_path):
137         logger.warning(f"[watermark] logo not found: {logo_path} ? skipping logo
138         overlay.")
139         logo_path = ""
140     if not text and not logo_path:
141         return None
142
143     margin = int(wm.get("margin", 24))
144     opacity = float(wm.get("opacity", 0.85))
145     position = str(wm.get("position", "bottom-right"))
146     # drawtext uses lowercase w/h (main) + tw/th (text); overlay uses W/H (main) +
147     w/h (logo).
148     dt_xy = {
149         "bottom-right": f"x=w-tw-{margin}:y=h-th-{margin}",
150         "bottom-left": f"x={margin}:y=h-th-{margin}",
151         "top-right": f"x=w-tw-{margin}:y={margin}",
152         "top-left": f"x={margin}:y={margin}",
153     }.get(position, f"x=w-tw-{margin}:y=h-th-{margin}")
154     ov_xy = {
155         "bottom-right": f"W-w-{margin}:H-h-{margin}",
156         "bottom-left": f"{margin}:H-h-{margin}",
157         "top-right": f"W-w-{margin}:{margin}",
158         "top-left": f"{margin}:{margin}",
159     }.get(position, f"W-w-{margin}:H-h-{margin}")
160
161     drawtext = ""
162     if text:
163         ratio = float(wm.get("font_size_ratio") or 0.035)
164         divisor = max(1, round(1.0 / ratio))
165         # textfile= sidesteps the notorious drawtext text= escaping for arbitrary
166         strings.
167         tf_path = os.path.join(tmp_dir, "watermark.txt")
168         with open(tf_path, "w", encoding="utf-8") as fh:
169             fh.write(text)
170         # Font precedence: explicit font_path ? bundled Apache-2.0 default (renders
171         # without fontconfig) ? fontconfig family (last-resort, environment-
172         dependent).
173         font_path = (wm.get("font_path") or "").strip()
174         if not font_path and _BUNDLED_FONT.exists():
175             font_path = str(_BUNDLED_FONT)
176         if font_path:
177             font_opt = f"fontfile='{self._ff_quote(font_path)}'"
178         else:
179             font_opt = f"font='{wm.get('font', 'Sans')}'"

```

```

176         box = ""
177         if wm.get("box", True):
178             box = f":box=1:boxcolor=black@{min(opacity, 0.5):.2f}:boxborderw=10"
179         drawtext = (
180             f"drawtext=textfile='{self._ff_quote(tf_path)}':{font_opt}"
181             f":fontcolor=white@{opacity:.2f}:fontsize=h/{divisor}:{dt_xy}{box}"
182         )
183
184         if logo_path and drawtext:
185             return
186         f"movie='{self._ff_quote(logo_path)}'[wm];[in][wm]overlay={ov_xy},{drawtext}"
187         if logo_path:
188             return f"movie='{self._ff_quote(logo_path)}'[wm];[in][wm]overlay={ov_xy}"
189         return drawtext
190
191     def compile_highlights(self, raw_video_path: str, segments: List[Tuple[float,
192 float]], output_filename: str) -> str:
193         """
194         Extracts selected segments from a raw video and stitches them together
195         into a seamless highlights file, applying end_padding to prevent abrupt endings.
196         """
197         if not segments:
198             raise ValueError("No highlight segments provided to compile.")
199
200         # Never stitch the same footage twice: collapse overlapping/duplicate ranges
201         # (e.g. accumulated re-run segments or chunk-overlap dupes) before trimming.
202         original_count = len(segments)
203         segments = self._merge_overlapping(segments)
204         if len(segments) < original_count:
205             logger.info("Merged %d overlapping/duplicate segment(s) before stitching: %d
206 -> %d.",
207                 original_count - len(segments), original_count, len(segments))
208         if not segments:
209             raise ValueError("No valid (start < end) highlight segments after merge.")
210
211         if not os.path.exists(raw_video_path):
212             raise FileNotFoundError(f"[COMPILER ERROR] Source video file not found:
213 {raw_video_path}")
214
215         # Defense-in-depth: never let the output name escape output_dir (the API also
216         # validates this at the request boundary).
217         output_path = os.path.join(self.output_dir, os.path.basename(output_filename))
218
219         # Probe video duration so we can detect out-of-range segments
220         video_duration = self._get_video_duration(raw_video_path)
221         if video_duration > 0:
222             logger.info(f"Source video duration: {video_duration:.2f}s")
223         else:
224             logger.warning("Video duration unknown. Cannot validate segment boundaries
225 against video length.")
226
227         # We will create temporary clips and stitch them using FFmpeg concat demuxer
228         temp_clips = []
229         skipped_segments = []
230
231         # Create a temp directory within output_dir
232         with tempfile.TemporaryDirectory(dir=self.output_dir) as tmp_dir:
233             logger.info(f"Trimming {len(segments)} segments
234 (begin_padding={self.begin_padding}s, end_padding={self.end_padding}s)...")
235             # Build the (optional) watermark filtergraph once; it is applied during
236             every
237             # per-clip re-encode so the mark is baked into each clip identically.
238             watermark_filter = self._watermark_filter(tmp_dir)
239             if watermark_filter:
240                 logger.info("[watermark] applying branding overlay to each clip.")

```

```

234         for idx, (raw_start, raw_end) in enumerate(segments):
235             # Normalize timestamps to float seconds ? handles float, int,
236             # plain numeric strings, and colon-separated formats like "MM:SS"
237             start = self._parse_time(raw_start)
238             end = self._parse_time(raw_end)
239             segment_label = f"Segment #{idx+1}/{len(segments)} [{start:.2f}s -
{end:.2f}s]"
240
241             # Validate the segment times
242             if start < 0:
243                 logger.warning(f"{segment_label}: start_time is negative
({start:.2f}s). Clamping to 0.")
244                 start = 0.0
245
246             if start >= end:
247                 logger.error(f"{segment_label}: start_time ({start:.2f}s) >=
end_time ({end:.2f}s). Skipping invalid segment.")
248                 skipped_segments.append((idx, start, end, "start >= end"))
249                 continue
250
251             # Check if segment extends beyond video duration
252             if video_duration > 0:
253                 if start >= video_duration:
254                     logger.error(f"{segment_label}: start_time ({start:.2f}s) is
beyond the video duration ({video_duration:.2f}s). "
255                                 f"This segment cannot be extracted ? the video ran
out. Skipping.")
256                     skipped_segments.append((idx, start, end, "start beyond video
end"))
257                     continue
258
259                 if end > video_duration:
260                     original_end = end
261                     end = video_duration
262                     logger.warning(f"{segment_label}: end_time ({original_end:.2f}s)
exceeds video duration ({video_duration:.2f}s). "
263                                 f"Clamping end_time to {video_duration:.2f}s. The
segment may be truncated.")
264
265                 clip_path = os.path.join(tmp_dir, f"clip_{idx}.mp4")
266
267             # Precise sub-second trim using fast re-encoding to preserve stream
headers
268             padded_start = max(0.0, start - self.begin_padding)
269             padded_end = end + self.end_padding
270
271             # Clamp padded_end to video duration if known
272             if video_duration > 0 and padded_end > video_duration:
273                 padded_end = video_duration
274                 logger.info(f"{segment_label}: Padded end ({end +
self.end_padding:.2f}s) exceeds video duration. "
275                             f"Clamping to {video_duration:.2f}s (end padding
partially applied).")
276
277             trim_duration = padded_end - padded_start
278             logger.info(f"Trimming {segment_label} -> [{padded_start:.3f}s -
{padded_end:.3f}s] (duration: {trim_duration:.2f}s)")
279
280             base_cmd = [
281                 "ffmpeg", "-y",
282                 "-ss", str(round(padded_start, 3)),
283                 "-to", str(round(padded_end, 3)),
284                 "-i", raw_video_path,
285                 "-c:v", "libx264",
286                 "-preset", "superfast",

```



```

287         "-crf", "22",
288         "-c:a", "aac",
289         "-b:a", "128k",
290     ]
291     vf_args = ["-vf", watermark_filter] if watermark_filter else []
292
293     # Try with the watermark first; if that encode fails (e.g. a bad
font/logo
294     # path), retry once WITHOUT it so a branding misconfig can never nuke
the
295     # whole reel. Clips stay codec-identical either way, so concat -c copy
holds.
296     attempts = [vf_args, []] if vf_args else [[]]
297     produced = False
298     last_exc: Optional[subprocess.CalledProcessError] = None
299     cmd = base_cmd + [clip_path]
300     for attempt_vf in attempts:
301         cmd = base_cmd + attempt_vf + [clip_path]
302         try:
303             subprocess.run(cmd, capture_output=True, check=True)
304         except subprocess.CalledProcessError as e:
305             last_exc = e
306             if attempt_vf and vf_args:
307                 _err = (e.stderr.decode(errors="replace").strip()[-300:]
308                     if getattr(e, "stderr", None) else "")
309                 logger.warning(
310                     f"{segment_label}: watermark encode failed; retrying
without it. "
311                     f"ffmpeg: {_err or '(no stderr captured)}'"
312                 )
313                 continue
314             break
315         if os.path.exists(clip_path) and os.path.getsize(clip_path) > 0:
316             produced = True
317             if not attempt_vf and vf_args:
318                 logger.warning(f"{segment_label}: encoded WITHOUT the
watermark (the watermark filter failed).")
319             break
320         last_exc = None # ffmpeg succeeded but produced nothing
321         break
322
323     if produced:
324         temp_clips.append(clip_path)
325     elif last_exc is not None:
326         stderr_msg = last_exc.stderr.decode(errors='replace').strip()
327         logger.error(f"{segment_label}: FFmpeg trim failed.")
328         print(f"  Command: {' '.join(cmd)}")
329         print(f"  FFmpeg stderr: {stderr_msg[-500:]}") # Last 500 chars to
avoid flooding
330         skipped_segments.append((idx, start, end, f"ffmpeg error:
{stderr_msg[-200:]}"))
331         continue
332     else:
333         logger.error(f"{segment_label}: FFmpeg exited successfully but
output clip is missing or empty. Skipping.")
334         skipped_segments.append((idx, start, end, "empty output clip"))
335         continue
336
337     # Report summary of skipped segments
338     if skipped_segments:
339         logger.info(f"\n[SUMMARY] {len(skipped_segments)} out of {len(segments)}
segments were SKIPPED:")
340         for seg_idx, seg_start, seg_end, reason in skipped_segments:
341             logger.info(f"  - Segment #{seg_idx+1} [{seg_start:.2f}s -
{seg_end:.2f}s]: {reason}")

```

```

342         logger.info("")
343
344     if not temp_clips:
345         raise RuntimeError(
346             f"[COMPILER ERROR] Failed to generate ANY valid highlight clips. "
347             f"All {len(segments)} segments were skipped. "
348             f"Check the errors above for details. "
349             f"Common causes: video file is corrupted, segments are out of range,
or ffmpeg is not installed."
350         )
351
352     logger.info(f"Successfully trimmed {len(temp_clips)}/{len(segments)} clips.
Preparing to stitch...")
353
354     # Create a list file for FFmpeg concat demuxer
355     list_file_path = os.path.join(tmp_dir, "clips.txt")
356     with open(list_file_path, "w") as f:
357         for clip in temp_clips:
358             # Escape single quotes in path
359             safe_path = clip.replace("'", "'\\'")
360             f.write(f"file '{safe_path}'\n")
361
362     logger.info("Stitching clips together...")
363     # Stitch them together without re-encoding (since they were encoded with
identical settings)
364     concat_cmd = [
365         "ffmpeg", "-y",
366         "-f", "concat",
367         "-safe", "0",
368         "-i", list_file_path,
369         "-c", "copy",
370         output_path
371     ]
372
373     try:
374         subprocess.run(concat_cmd, capture_output=True, check=True)
375
376         # Final verification
377         if not os.path.exists(output_path) or os.path.getsize(output_path) == 0:
378             raise RuntimeError(
379                 f"[COMPILER ERROR] FFmpeg concatenation appeared to succeed, "
380                 f"but output file is missing or empty: {output_path}"
381             )
382
383         output_size_mb = os.path.getsize(output_path) / (1024 * 1024)
384         logger.info(f"Dynamic highlight reel saved: {output_path}
({output_size_mb:.1f} MB)")
385         return output_path
386     except subprocess.CalledProcessError as e:
387         stderr_msg = e.stderr.decode(errors='replace').strip()
388         raise RuntimeError(
389             f"[COMPILER ERROR] FFmpeg concatenation failed.\n"
390             f"Command: {' '.join(concat_cmd)}\n"
391             f"FFmpeg stderr: {stderr_msg[-500:]}"
392         ) from e
393

```

13. assistant (2026-06-18T01:33:36.418Z)

This file is well-hardened. Let me look at the database infrastructure, which is the prime target for connection/cursor resource leaks.

14. user (2026-06-18T01:33:37.783Z)

```

1     import sqlite3

```

```

2     import json
3     import logging
4     from contextlib import contextmanager
5     from typing import List, Dict, Any, Optional, Tuple
6
7     logger = logging.getLogger(__name__)
8
9     class Database:
10         def __init__(self, db_path: str):
11             self.db_path = db_path
12             self._init_db()
13
14         @contextmanager
15         def _connect(self):
16             """Transaction scope that also CLOSES the connection.
17
18             ``with sqlite3.connect(...)`` alone is only a commit/rollback scope ? it
19             never closes, so every call leaked its connection to GC (lingering file
20             handles on the WAL sidecars under Windows). Internal methods use this.
21             """
22             conn = self._get_connection()
23             try:
24                 with conn:
25                     yield conn
26             finally:
27                 conn.close()
28
29         def _get_connection(self):
30             # busy_timeout: wait (not fail) on a locked DB ? needed once the async job model
31             # adds concurrent writers (a swallowed 'database is locked' silently drops
32             segments).
33             conn = sqlite3.connect(self.db_path, timeout=30.0)
34             conn.row_factory = sqlite3.Row
35             # Enforce ON DELETE CASCADE / SET NULL ? SQLite leaves FKs off per-connection.
36             conn.execute("PRAGMA foreign_keys = ON")
37             # WAL allows concurrent readers during a write; busy_timeout bounds lock waits.
38             conn.execute("PRAGMA busy_timeout = 30000")
39             try:
40                 conn.execute("PRAGMA journal_mode = WAL")
41             except sqlite3.OperationalError:
42                 pass # e.g. a read-only/networked FS that can't do WAL ? degrade gracefully
43             return conn
44
45         def _init_db(self):
46             """Initializes tables for videos, play_segments, and events."""
47             with self._connect() as conn:
48                 cursor = conn.cursor()
49
50                 # Videos Table
51                 cursor.execute("""
52                     CREATE TABLE IF NOT EXISTS videos (
53                         id TEXT PRIMARY KEY,
54                         filepath TEXT NOT NULL,
55                         processed_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
56                         status TEXT NOT NULL,
57                         validation_results TEXT
58                     )
59                 """)
60
61                 # Play Segments Table (Extensible metadata JSON)
62                 cursor.execute("""
63                     CREATE TABLE IF NOT EXISTS play_segments (
64                         id INTEGER PRIMARY KEY AUTOINCREMENT,
65                         video_id TEXT NOT NULL,
66                         start_time REAL NOT NULL,

```

```

66         end_time REAL NOT NULL,
67         duration REAL NOT NULL,
68         server TEXT,
69         receiver TEXT,
70         metadata TEXT,
71         FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE
72     )
73 """)
74
75 # Events Table
76 cursor.execute("""
77     CREATE TABLE IF NOT EXISTS events (
78         id INTEGER PRIMARY KEY AUTOINCREMENT,
79         segment_id INTEGER NOT NULL,
80         timestamp REAL NOT NULL,
81         type TEXT NOT NULL,
82         player TEXT,
83         details TEXT,
84         FOREIGN KEY (segment_id) REFERENCES play_segments(id) ON DELETE
CASCADE
85     )
86 """)
87
88 # Versioned migrations live here. PRAGMA user_version is the schema
89 # contract ? bump it for every change and add an ordered `if version < N`
90 # block below. Tests assert the expected version, so accidental drift fails
CI.
91 version = cursor.execute("PRAGMA user_version").fetchone()[0]
92
93 if version < 1:
94     # v1: raw candidate detections (pre-confirmation), detector-agnostic.
95     # Common fields are columns; detector-specific metrics go in `stats`
JSON,
96     # so a new segmenter reuses this table by setting its own
`detector`/`stats`.
97     cursor.execute("""
98         CREATE TABLE IF NOT EXISTS candidate_segments (
99             id INTEGER PRIMARY KEY AUTOINCREMENT,
100             video_id TEXT NOT NULL,
101             detector TEXT NOT NULL,
102             chunk_index INTEGER,
103             start_time REAL NOT NULL,
104             end_time REAL NOT NULL,
105             duration REAL NOT NULL,
106             confidence REAL,
107             stats TEXT,
108             detection_params TEXT,
109             confirmed_segment_id INTEGER,
110             human_label TEXT,
111             notes TEXT,
112             created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
113             FOREIGN KEY (video_id) REFERENCES videos(id) ON DELETE CASCADE,
114             FOREIGN KEY (confirmed_segment_id) REFERENCES play_segments(id)
ON DELETE SET NULL
115         )
116     """)
117     cursor.execute("""
118         CREATE INDEX IF NOT EXISTS idx_candidates_video_detector
119         ON candidate_segments(video_id, detector)
120     """)
121     cursor.execute("PRAGMA user_version = 1")
122
123 if version < 2:
124     # v2: async job model (DEFERRED_HARDENING_PLAN #16). One row per
submitted

```

```

125         # /api/process request. `status` is the EXECUTION state of the job
126         # (queued|running|done|failed); the pipeline's own verdict (e.g. a video
127         # that failed validation) lives inside `result` JSON as
result["status"].
128         # `result` is the full sync-era response dict (incl. report paths), so
the
129         # observability report is the job's artifact, per the plan.
130         cursor.execute("""
131             CREATE TABLE IF NOT EXISTS jobs (
132                 id TEXT PRIMARY KEY,
133                 video_path TEXT NOT NULL,
134                 video_id TEXT,
135                 segmenter TEXT,
136                 player_pool TEXT,
137                 max_frames INTEGER,
138                 status TEXT NOT NULL DEFAULT 'queued',
139                 progress TEXT,
140                 error TEXT,
141                 result TEXT,
142                 created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
143                 started_at TIMESTAMP,
144                 finished_at TIMESTAMP
145             )
146         """)
147         cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_status ON
jobs(status)")
148         cursor.execute("PRAGMA user_version = 2")
149
150     if version < 3:
151         # v3: self-scheduling execution policy (EXECUTION_POLICY.md P2).
`policy` = the
152         # JSON ExecutionPolicy a job carries; `next_poll_at` = when a 'waiting'
job is due
153         # to resume. A 'waiting' status + due-time lets a job SELF-SCHEDULE ?
any worker
154         # re-claims it via `claim_due_job` when due ? with NO master node: the
table IS
155         # the coordinator. ALTER (not recreate) so existing v2 job rows survive.
156         cursor.execute("ALTER TABLE jobs ADD COLUMN policy TEXT")
157         cursor.execute("ALTER TABLE jobs ADD COLUMN next_poll_at TIMESTAMP")
158         cursor.execute("CREATE INDEX IF NOT EXISTS idx_jobs_due ON jobs(status,
next_poll_at)")
159         cursor.execute("PRAGMA user_version = 3")
160
161     if version < 4:
162         # v4: personalization groundwork (PERSONALIZATION_PLAN.md §2). Add a
NULLABLE
163         # `user_id` to videos + candidate_segments so a future per-user / multi-
tenant
164         # path can scope rows to an owner. **NULL = local install = byte-
identical to
165         # today** ? every existing row stays NULL and every existing query
(which never
166         # filters on user_id) is unaffected. ALTER (not recreate) so v1-v3 rows
survive.
167         # Additive ONLY: no served-behavior change rides on this slice.
168         cursor.execute("ALTER TABLE videos ADD COLUMN user_id TEXT")
169         cursor.execute("ALTER TABLE candidate_segments ADD COLUMN user_id TEXT")
170         # Partial indexes (WHERE user_id IS NOT NULL) keep the NULL=local fast
path free
171         # of index churn while making the eventual per-user lookups cheap.
172         cursor.execute(
173             "CREATE INDEX IF NOT EXISTS idx_videos_user ON videos(user_id) "
174             "WHERE user_id IS NOT NULL")
175         cursor.execute(

```

```

176         "CREATE INDEX IF NOT EXISTS idx_candidates_user "
177         "ON candidate_segments(user_id) WHERE user_id IS NOT NULL")
178     cursor.execute("PRAGMA user_version = 4")
179
180 if version < 5:
181     # v5: per-user model store (PERSONALIZATION_PLAN.md §2). One row per
182     # (user_id,
183     # version): the resolved per-user `fusion_config` overlay + an INLINE
184     # `classifier_json` (tenant-safe ? no file artifact), plus the promotion
185     # and provenance that earned it. `is_active` pins the one row the
186     # resolves (the FUTURE owner-gated wiring into `_select_for_handoff`).
187     # only persists/loads it ? NOTHING reads it on the served path yet.
188     #
189     # user_id          ? owner of the model (NULL allowed = the local
190     # install)
191     # version          ? monotonic per-user model version
192     # baseline_version ? the shared baseline this was fit against
193     # feature_schema_hash ? fingerprint of the feature set; MINOR-vs-
194     # MAJOR re-fit gate
195     # fusion_config    ? JSON per-user FusionConfig overlay (cue_flags
196     # / thresholds)
197     # classifier_json  ? JSON LogisticRegression.to_dict() (None =
198     # config-only model)
199     # promotion_evidence ? JSON per_user_gate PromotionDecision (why it
200     # was promoted)
201     # n_videos_at_fit  ? held-out-user corpus size when fit (min_n
202     # trust floor input)
203     # is_active        ? the one resolved row for this user (partial-
204     # unique below)
205     cursor.execute("""
206         CREATE TABLE IF NOT EXISTS user_models (
207             id INTEGER PRIMARY KEY AUTOINCREMENT,
208             user_id TEXT,
209             version INTEGER NOT NULL DEFAULT 1,
210             baseline_version TEXT,
211             feature_schema_hash TEXT,
212             fusion_config TEXT,
213             classifier_json TEXT,
214             promotion_evidence TEXT,
215             n_videos_at_fit INTEGER NOT NULL DEFAULT 0,
216             is_active INTEGER NOT NULL DEFAULT 0,
217             created_at TIMESTAMP DEFAULT CURRENT_TIMESTAMP
218         )
219     """)
220     # One model version per user (NULLs compare distinct in SQLite, so the
221     # local
222     # install can still carry multiple versioned rows; the active-row guard
223     # is what enforces a single served model).
224     cursor.execute("""
225         CREATE UNIQUE INDEX IF NOT EXISTS idx_user_models_user_version
226         ON user_models(user_id, version)
227     """)
228     # At most ONE active model per user: a partial-unique index over the
229     # active rows.
230     # (SQLite treats NULL user_id rows as distinct, so the local install's
231     # active row is still guarded ? there is one local user.)
232     cursor.execute("""
233         CREATE UNIQUE INDEX IF NOT EXISTS idx_user_models_one_active
234         ON user_models(user_id) WHERE is_active = 1

```

```

225         """)
226         cursor.execute("PRAGMA user_version = 5")
227         # future: if version < 6: cursor.execute("ALTER TABLE ..."); PRAGMA
user_version = 6
228
229         conn.commit()
230
231     def add_video(self, video_id: str, filepath: str, status: str, validation_results:
List[Dict[str, Any]]) -> bool:
232         """Register or update a video row WITHOUT touching its child segments.
233
234         Uses UPSERT, not INSERT OR REPLACE: REPLACE deletes the conflicting row, which ?
235         with foreign_keys ON ? cascade-deletes every
play_segment/event/candidate_segment.
236         That made a status update (e.g. marking 'failed' on re-validation, or
'processed'
237         before the segmenter runs) silently destroy prior good results. UPSERT mutates
the
238         row in place; clearing segments is now an explicit, deliberate operation
239         (clear_video_data / prune_segments).
240         """
241         try:
242             with self._connect() as conn:
243                 conn.cursor().execute(
244                     "INSERT INTO videos (id, filepath, status, validation_results)
VALUES (?, ?, ?, ?) "
245                     "ON CONFLICT(id) DO UPDATE SET "
246                     "filepath=excluded.filepath, status=excluded.status, "
247                     "validation_results=excluded.validation_results",
248                     (video_id, filepath, status, json.dumps(validation_results))
249                 )
250                 conn.commit()
251             return True
252         except Exception as e:
253             logger.error(f"Failed to add video: {e}")
254             return False
255
256     def segment_watermarks(self, video_id: str) -> Tuple[int, int]:
257         """Return (max play_segment id, max candidate_segment id) for a video (0 if
258         none).
259
260         Captured before a (re)segmentation so the run's new rows (id > watermark) can be
261         told apart from prior rows (id <= watermark), enabling destroy-AFTER-confirm.
262         """
263         with self._connect() as conn:
264             cur = conn.cursor()
265             p = cur.execute("SELECT COALESCE(MAX(id), 0) FROM play_segments WHERE
video_id = ?",
266                             (video_id,)).fetchone()[0]
267             c = cur.execute("SELECT COALESCE(MAX(id), 0) FROM candidate_segments WHERE
video_id = ?",
268                             (video_id,)).fetchone()[0]
269             return int(p), int(c)
270
271     def prune_segments(self, video_id: str, *, play_upto: Optional[int] = None,
play_after: Optional[int] = None, cand_upto: Optional[int] =
None,
272                       cand_after: Optional[int] = None) -> None:
273         """Delete play/candidate segments by id range (events cascade from
play_segments).
274
275         Used to finish a (re)segmentation: keep the NEW rows and drop the OLD ones on a
276         successful run (`*_upto` = the pre-run watermark), or drop the NEW partial
277         rows
and keep the OLD ones when the run failed/produced nothing (`*_after`).

```

```

278         """
279         with self._connect() as conn:
280             cur = conn.cursor()
281             if play_upto is not None:
282                 cur.execute("DELETE FROM play_segments WHERE video_id = ? AND id <= ?",
(video_id, play_upto))
283             if play_after is not None:
284                 cur.execute("DELETE FROM play_segments WHERE video_id = ? AND id > ?",
(video_id, play_after))
285             if cand_upto is not None:
286                 cur.execute("DELETE FROM candidate_segments WHERE video_id = ? AND id <=
?", (video_id, cand_upto))
287             if cand_after is not None:
288                 cur.execute("DELETE FROM candidate_segments WHERE video_id = ? AND id >
?", (video_id, cand_after))
289             conn.commit()
290
291         def add_play_segment(self, video_id: str, start_time: float, end_time: float,
server: Optional[str] = None, receiver: Optional[str] = None, metadata: Optional[Dict[str, Any]]
= None) -> Optional[int]:
292             try:
293                 with self._connect() as conn:
294                     cursor = conn.cursor()
295                     cursor.execute(
296                         "INSERT INTO play_segments (video_id, start_time, end_time,
duration, server, receiver, metadata) VALUES (?, ?, ?, ?, ?, ?, ?)",
297                         (video_id, start_time, end_time, end_time - start_time, server,
receiver, json.dumps(metadata or {})))
298                     )
299                     conn.commit()
300                     return cursor.lastrowid
301             except Exception as e:
302                 logger.error(f"Failed to add play segment: {e}")
303                 return None
304
305         def add_event(self, segment_id: int, timestamp: float, event_type: str, player:
Optional[str] = None, details: Optional[Dict[str, Any]] = None) -> bool:
306             try:
307                 with self._connect() as conn:
308                     conn.cursor().execute(
309                         "INSERT INTO events (segment_id, timestamp, type, player, details)
VALUES (?, ?, ?, ?, ?)",
310                         (segment_id, timestamp, event_type, player, json.dumps(details or
{})))
311                     )
312                     conn.commit()
313                     return True
314             except Exception as e:
315                 logger.error(f"Failed to add event: {e}")
316                 return False
317
318         def add_candidate_segment(self, video_id: str, detector: str, start_time: float,
end_time: float,
319                                     chunk_index: Optional[int] = None, confidence:
Optional[float] = None,
320                                     stats: Optional[Dict[str, Any]] = None,
321                                     detection_params: Optional[Dict[str, Any]] = None) ->
Optional[int]:
322             """Persist one raw (pre-confirmation) detection. `stats`/`detection_params` are
323             detector-specific dicts stored as JSON. Returns the new candidate id, or
None."""
324             try:
325                 with self._connect() as conn:
326                     cursor = conn.cursor()
327                     cursor.execute(

```



```

328         """INSERT INTO candidate_segments
329             (video_id, detector, chunk_index, start_time, end_time, duration,
330              confidence, stats, detection_params)
331             VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)"""
332         (video_id, detector, chunk_index, start_time, end_time, end_time -
start_time,
333          confidence, json.dumps(stats or {}), json.dumps(detection_params or
{}))
334     )
335     conn.commit()
336     return cursor.lastrowid
337 except Exception as e:
338     logger.error(f"Failed to add candidate segment: {e}")
339     return None
340
341 def link_candidate_to_segment(self, candidate_id: int, segment_id: int) -> bool:
342     """Record that a candidate detection was confirmed as a given play_segment."""
343     try:
344         with self._connect() as conn:
345             conn.cursor().execute(
346                 "UPDATE candidate_segments SET confirmed_segment_id = ? WHERE id =
?",
347                 (segment_id, candidate_id)
348             )
349             conn.commit()
350             return True
351     except Exception as e:
352         logger.error(f"Failed to link candidate {candidate_id} to segment
{segment_id}: {e}")
353         return False
354
355 def query_candidate_segments(self, video_id: str, detector: Optional[str] = None,
356                             only_unlabeled: bool = False) -> List[Dict[str, Any]]:
357     """Fetch candidate detections for the annotation / fine-tuning workflow.
358     `stats` and `detection_params` are deserialized from JSON."""
359     query = "SELECT * FROM candidate_segments WHERE video_id = ?"
360     params: List[Any] = [video_id]
361
362     if detector:
363         query += " AND detector = ?"
364         params.append(detector)
365     if only_unlabeled:
366         query += " AND human_label IS NULL"
367
368     query += " ORDER BY start_time"
369
370     candidates = []
371     with self._connect() as conn:
372         rows = conn.cursor().execute(query, params).fetchall()
373         for row in rows:
374             d = dict(row)
375             d["stats"] = json.loads(d["stats"] or "{}")
376             d["detection_params"] = json.loads(d["detection_params"] or "{}")
377             candidates.append(d)
378     return candidates
379
380 def get_video(self, video_id: str) -> Optional[Dict[str, Any]]:
381     with self._connect() as conn:
382         row = conn.cursor().execute("SELECT * FROM videos WHERE id = ?",
(video_id,)).fetchone()
383         if row:
384             d = dict(row)
385             d["validation_results"] = json.loads(d["validation_results"] or "[]")
386             return d
387     return None

```

```

388
389     def query_play_segments(self, video_id: str, filters: Dict[str, Any]) ->
List[Dict[str, Any]]:
390         """
391         Dynamically queries play segments based on filters:
392         - duration_min: float
393         - server: string
394         - receiver: string
395         - event_type: string (e.g. smash, foul)
396         """
397         query = "SELECT r.* FROM play_segments r WHERE r.video_id = ?"
398         params = [video_id]
399
400         if filters.get("duration_min"):
401             query += " AND r.duration >= ?"
402             params.append(filters["duration_min"])
403
404         if filters.get("server"):
405             query += " AND r.server = ?"
406             params.append(filters["server"])
407
408         if filters.get("receiver"):
409             query += " AND r.receiver = ?"
410             params.append(filters["receiver"])
411
412         if filters.get("event_type"):
413             # Check if there is an event of a specific type inside this segment
414             query += " AND EXISTS (SELECT 1 FROM events e WHERE e.segment_id = r.id AND
e.type = ?)"
415             params.append(filters["event_type"])
416
417         segments_list = []
418         with self._connect() as conn:
419             cursor = conn.cursor()
420             rows = cursor.execute(query, params).fetchall()
421
422             for row in rows:
423                 r_dict = dict(row)
424                 r_id = r_dict["id"]
425                 r_dict["metadata"] = json.loads(r_dict["metadata"] or "{}")
426
427                 # Fetch associated events
428                 event_rows = cursor.execute("SELECT * FROM events WHERE segment_id = ?",
(r_id,)).fetchall()
429                 r_dict["events"] = []
430                 for er in event_rows:
431                     e_dict = dict(er)
432                     e_dict["details"] = json.loads(e_dict["details"] or "{}")
433                     r_dict["events"].append(e_dict)
434
435                 segments_list.append(r_dict)
436
437         return segments_list
438
439     def clear_video_data(self, video_id: str):
440         with self._connect() as conn:
441             conn.cursor().execute("DELETE FROM videos WHERE id = ?", (video_id,))
442             conn.commit()
443
444     # --- Async job store (DEFERRED_HARDENING_PLAN #16) -----
445     # Writers follow the repo convention (swallow + logger.error, return bool); the
446     # worker logs loudly on a failed transition so a job can't silently wedge.
447
448     def create_job(self, job_id: str, video_path: str, segmenter: Optional[str] = None,
449                   player_pool: Optional[List[str]] = None,

```

```

450         max_frames: Optional[int] = None,
451         policy: Optional[Dict[str, Any]] = None) -> bool:
452     """Insert a new job in state 'queued'. ``policy`` is an optional JSON
ExecutionPolicy
453     (EXECUTION_POLICY.md) the worker honours; None ? the worker's default policy."""
454     try:
455         with self._connect() as conn:
456             conn.cursor().execute(
457                 "INSERT INTO jobs (id, video_path, segmenter, player_pool,
max_frames, status, policy) "
458                 "VALUES (?, ?, ?, ?, ?, 'queued', ?)",
459                 (job_id, video_path, segmenter,
460                  json.dumps(player_pool) if player_pool is not None else None,
461                  max_frames, json.dumps(policy) if policy is not None else None)
462             )
463             conn.commit()
464             return True
465     except Exception as e:
466         logger.error(f"Failed to create job {job_id}: {e}")
467         return False
468
469     def mark_job_running(self, job_id: str) -> bool:
470         try:
471             with self._connect() as conn:
472                 conn.cursor().execute(
473                     "UPDATE jobs SET status='running', started_at=CURRENT_TIMESTAMP, "
474                     "progress='starting' WHERE id = ?", (job_id,))
475                 conn.commit()
476             return True
477         except Exception as e:
478             logger.error(f"Failed to mark job {job_id} running: {e}")
479             return False
480
481     def set_job_progress(self, job_id: str, progress: str) -> bool:
482         try:
483             with self._connect() as conn:
484                 conn.cursor().execute(
485                     "UPDATE jobs SET progress=? WHERE id = ?", (progress, job_id))
486                 conn.commit()
487             return True
488         except Exception as e:
489             logger.error(f"Failed to set progress for job {job_id}: {e}")
490             return False
491
492     def set_job_video_id(self, job_id: str, video_id: str) -> bool:
493         try:
494             with self._connect() as conn:
495                 conn.cursor().execute(
496                     "UPDATE jobs SET video_id=? WHERE id = ?", (video_id, job_id))
497                 conn.commit()
498             return True
499         except Exception as e:
500             logger.error(f"Failed to set video_id for job {job_id}: {e}")
501             return False
502
503     def mark_job_done(self, job_id: str, result: Dict[str, Any]) -> bool:
504         """Job EXECUTED to completion. The pipeline's own verdict (success/failed
505         validation) is inside `result` ? job status 'done' means 'the worker
finished'."""
506         try:
507             with self._connect() as conn:
508                 conn.cursor().execute(
509                     "UPDATE jobs SET status='done', progress='finished', result=?, "
510                     "finished_at=CURRENT_TIMESTAMP WHERE id = ?",
511                     (json.dumps(result), job_id))

```

```

512         conn.commit()
513     return True
514 except Exception as e:
515     logger.error(f"Failed to mark job {job_id} done: {e}")
516     return False
517
518 def mark_job_failed(self, job_id: str, error: str) -> bool:
519     """Worker crashed / raised ? infrastructure failure, not a pipeline verdict."""
520     try:
521         with self._connect() as conn:
522             conn.cursor().execute(
523                 "UPDATE jobs SET status='failed', error=?, "
524                 "finished_at=CURRENT_TIMESTAMP WHERE id = ?", (error, job_id))
525             conn.commit()
526         return True
527     except Exception as e:
528         logger.error(f"Failed to mark job {job_id} failed: {e}")
529         return False
530
531 def get_job(self, job_id: str) -> Optional[Dict[str, Any]]:
532     """Fetch one job; `result`/`player_pool` JSON deserialized."""
533     with self._connect() as conn:
534         row = conn.cursor().execute("SELECT * FROM jobs WHERE id = ?",
535 (job_id,)).fetchone()
536         if not row:
537             return None
538         d = dict(row)
539         d["result"] = json.loads(d["result"]) if d["result"] else None
540         d["player_pool"] = json.loads(d["player_pool"]) if d["player_pool"] else
541         None
542         d["policy"] = json.loads(d["policy"]) if d.get("policy") else None
543         return d
544
545 # --- Self-scheduling execution policy (EXECUTION_POLICY.md P2)
546 -----
547 def set_job_policy(self, job_id: str, policy: Optional[Dict[str, Any]]) -> bool:
548     """Attach/replace a job's JSON ExecutionPolicy."""
549     try:
550         with self._connect() as conn:
551             conn.cursor().execute(
552                 "UPDATE jobs SET policy=? WHERE id = ?",
553                 (json.dumps(policy) if policy is not None else None, job_id))
554             conn.commit()
555         return True
556     except Exception as e:
557         logger.error(f"Failed to set policy for job {job_id}: {e}")
558         return False
559
560 def set_job_waiting(self, job_id: str, delay_sec: float, progress: Optional[str] =
561 None) -> bool:
562     """Self-schedule: park a job in 'waiting' until ``delay_sec`` from now,
563 releasing the
564 worker. `claim_due_job` re-picks it when due ? cooperative, no master.
565 ``next_poll_at`` is
566 computed in SQLite's own UTC format so it compares directly to
567 CURRENT_TIMESTAMP."""
568     try:
569         with self._connect() as conn:
570             conn.cursor().execute(
571                 "UPDATE jobs SET status='waiting', "
572                 "next_poll_at=datetime('now', ?), "
573                 "progress=COALESCE(?, progress) WHERE id = ?",
574                 (f"+{max(0, int(delay_sec))} seconds", progress, job_id))
575             conn.commit()
576         return True

```

```

570         except Exception as e:
571             logger.error(f"Failed to set job {job_id} waiting: {e}")
572             return False
573
574     def seconds_until_next_due(self) -> Optional[float]:
575         """Seconds until the soonest parked ('waiting') job becomes due, so the worker
can
576         schedule a wake-up (EXECUTION_POLICY.md P3 self-scheduling). Returns 0.0 if one
is
577         already due, or None if nothing is waiting. A NULL ``next_poll_at`` ? due now
(0.0)."""
578         try:
579             with self._connect() as conn:
580                 row = conn.cursor().execute(
581                     "SELECT MIN(CASE WHEN next_poll_at IS NULL THEN 0 ELSE "
582                     "    (julianday(next_poll_at) - julianday('now')) * 86400.0 END) AS
secs "
583                     "FROM jobs WHERE status='waiting']").fetchone()
584                 if row is None or row["secs"] is None:
585                     return None
586                 return max(0.0, float(row["secs"]))
587         except Exception as e:
588             logger.error(f"Failed to compute next-due delay: {e}")
589             return None
590
591     def claim_due_job(self) -> Optional[Dict[str, Any]]:
592         """Atomically claim ONE runnable job ? 'queued', or 'waiting' whose
``next_poll_at`` is due ?
593         flipping it to 'running'. Compare-and-set on (id, status) so concurrent workers
never
594         double-claim (the DB row-claim is the only coordination needed; no master).
Earliest-due
595         first. Returns the claimed job dict, or None if nothing is runnable."""
596         try:
597             with self._connect() as conn:
598                 cur = conn.cursor()
599                 row = cur.execute(
600                     "SELECT id, status FROM jobs WHERE status='queued' "
601                     "OR (status='waiting' AND (next_poll_at IS NULL OR next_poll_at <=
CURRENT_TIMESTAMP)) "
602                     "ORDER BY COALESCE(next_poll_at, created_at) ASC LIMIT
1").fetchone()
603                 if not row:
604                     return None
605                 cur.execute(
606                     "UPDATE jobs SET status='running', started_at=CURRENT_TIMESTAMP "
607                     "WHERE id=? AND status=?", (row["id"], row["status"]))
608                 conn.commit()
609                 if cur.rowcount != 1:
610                     return None # lost the race to a concurrent
worker
611                 return self.get_job(row["id"])
612         except Exception as e:
613             logger.error(f"Failed to claim a due job: {e}")
614             return None
615
616     def fail_stale_jobs(self, reason: str = "interrupted by server restart") -> int:
617         """Crash recovery: any job still queued/running from a previous process is dead
618         (the executor is in-process). Mark failed so clients see a terminal state, not a
619         hang. Called once at startup. Returns the number of jobs swept."""
620         try:
621             with self._connect() as conn:
622                 cur = conn.cursor()
623                 cur.execute(
624                     "UPDATE jobs SET status='failed', error=?,

```

```

finished_at=CURRENT_TIMESTAMP "
625         "WHERE status IN ('queued', 'running')", (reason,))
626         conn.commit()
627         return cur.rowcount
628     except Exception as e:
629         logger.error(f"Failed to sweep stale jobs: {e}")
630         return 0
631
632     # --- Per-user model store (PERSONALIZATION_PLAN.md §2, v5) -----
633     # Groundwork ONLY: these persist/load the per-user model row. NOTHING on the served
634     # pipeline reads them yet ? the serving bridge (backend/pipeline/personalization/
635     # serving.py) resolves them, and wiring that bridge into the production decision
636     # seam
637     # (`fusion_hybrid._select_for_handoff`) is the NEXT, owner-gated PR. NULL user_id =
638     # the local install. `fusion_config` / `classifier_json` / `promotion_evidence` are
639     # JSON-serialised dicts; the rest are scalar columns.
640
641     def upsert_user_model(self, user_id: Optional[str], *, version: int = 1,
642                           baseline_version: Optional[str] = None,
643                           feature_schema_hash: Optional[str] = None,
644                           fusion_config: Optional[Dict[str, Any]] = None,
645                           classifier_json: Optional[Dict[str, Any]] = None,
646                           promotion_evidence: Optional[Dict[str, Any]] = None,
647                           n_videos_at_fit: int = 0,
648                           is_active: bool = False) -> Optional[int]:
649         """Insert or replace ONE per-user model row keyed by `(user_id, version)`.
650
651         Idempotent on that key (UPSERT, not INSERT-OR-REPLACE: REPLACE would re-allocate
652         the
653         rowid). When `is_active` is True this row becomes the user's single active
654         model ?
655         any previously-active row for the SAME user is first cleared, so the
656         ``idx_user_models_one_active`` partial-unique invariant always holds (one active
657         model
658         per user). Returns the row id, or None on failure.
659         """
660         try:
661             with self._connect() as conn:
662                 cur = conn.cursor()
663                 if is_active:
664                     # Demote the current active row first (NULL user_id = local: IS NULL
665                     match).
666                     if user_id is None:
667                         cur.execute(
668                             "UPDATE user_models SET is_active = 0 "
669                             "WHERE user_id IS NULL AND is_active = 1 AND version != ?",
670                             (version,))
671                     else:
672                         cur.execute(
673                             "UPDATE user_models SET is_active = 0 "
674                             "WHERE user_id = ? AND is_active = 1 AND version != ?",
675                             (user_id, version))
676                 cur.execute(
677                     """INSERT INTO user_models
678                     (user_id, version, baseline_version, feature_schema_hash,
679                     fusion_config, classifier_json, promotion_evidence,
680                     n_videos_at_fit, is_active)
681                     VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
682                     ON CONFLICT(user_id, version) DO UPDATE SET
683                     baseline_version=excluded.baseline_version,
684                     feature_schema_hash=excluded.feature_schema_hash,
685                     fusion_config=excluded.fusion_config,
686                     classifier_json=excluded.classifier_json,
687                     promotion_evidence=excluded.promotion_evidence,
688                     n_videos_at_fit=excluded.n_videos_at_fit,

```

```

684         is_active=excluded.is_active"",
685         (user_id, version, baseline_version, feature_schema_hash,
686         json.dumps(fusion_config) if fusion_config is not None else None,
687         json.dumps(classifier_json) if classifier_json is not None else
None,
688         json.dumps(promotion_evidence) if promotion_evidence is not None
else None,
689         n_videos_at_fit, 1 if is_active else 0))
690     conn.commit()
691     rid = cur.execute(
692         "SELECT id FROM user_models WHERE "
693         + ("user_id IS NULL" if user_id is None else "user_id = ?")
694         + " AND version = ?",
695         ((version,) if user_id is None else (user_id, version))).fetchone()
696     return int(rid[0]) if rid else None
697 except Exception as e:
698     logger.error(f"Failed to upsert user_model (user={user_id}, v={version}):
{e}")
699     return None
700
701 def get_active_user_model(self, user_id: Optional[str]) -> Optional[Dict[str, Any]]:
702     """Return the single active per-user model row for ``user_id`` (NULL = local
install),
703     with ``fusion_config`` / ``classifier_json`` / ``promotion_evidence`` JSON
deserialised,
704     or None if the user has no active model. Does not touch the served pipeline."""
705     with self._connect() as conn:
706         if user_id is None:
707             row = conn.cursor().execute(
708                 "SELECT * FROM user_models WHERE user_id IS NULL AND is_active =
1").fetchone()
709         else:
710             row = conn.cursor().execute(
711                 "SELECT * FROM user_models WHERE user_id = ? AND is_active = 1",
712                 (user_id,)).fetchone()
713         if not row:
714             return None
715         d = dict(row)
716         for col in ("fusion_config", "classifier_json", "promotion_evidence"):
717             d[col] = json.loads(d[col]) if d[col] else None
718         return d
719

```

15. assistant (2026-06-18T01:33:40.778Z)

This DB module is well-hardened with the `\_connect` context manager. Now let me look at the rally_slicer, hashing, and audio hit_detector which use ffmpeg.

16. user (2026-06-18T01:33:41.454Z)

```

1     """Slice individual rally clips out of a (long) annotated video.
2
3     Use cases:
4     - **Offline verification** ? pull rally N out of a 30-min match to eyeball its
5       boundaries against the labels.
6     - **Per-video calibration/training** ? grab a handful of labeled rallies as the
7       supervision the segmenter is tuned/adapted on for that specific video.
8
9     Given a labels CSV (the golden `rally_number,start_time,end_time[,start_mmss,
10    end_mmss]` shape) and a set of rally numbers, it writes one padded clip per rally:
11    <out_dir>/<video_stem>__rally_<n>.mp4
12
13    Padding matches the highlight compiler (begin 0.5 s / end 1.0 s) so a sliced clip
14    shows the same lead-in/out the model is trained and evaluated against. The window
15    math (`compute_clip_windows`) is pure and unit-tested; ffmpeg is only touched in

```



```

80         video_duration: Optional[float] = None) -> List[ClipWindow]:
81     """Apply padding and clamp to the video, for the requested rallies.
82
83     rally_numbers may be ["all"] to take every labeled rally (in time order).
84     Unknown rally ids are skipped (caller can diff to warn).
85     """
86     if len(rally_numbers) == 1 and rally_numbers[0].lower() == "all":
87         wanted = sorted(labels, key=lambda r: labels[r][0])
88     else:
89         wanted = [r for r in rally_numbers if r in labels]
90
91     windows: List[ClipWindow] = []
92     for rally in wanted:
93         start, end = labels[rally]
94         ps = max(0.0, start - begin_pad)
95         pe = end + end_pad
96         if video_duration is not None and video_duration > 0:
97             if ps >= video_duration:
98                 continue # entirely past the end
99             pe = min(pe, video_duration)
100         if pe <= ps:
101             continue
102         windows.append(ClipWindow(rally=rally, start=round(ps, 3), end=round(pe, 3)))
103     return windows
104
105
106 def _probe_duration(video_path: str) -> Optional[float]:
107     try:
108         out = subprocess.run(
109             ["ffprobe", "-v", "error", "-show_entries", "format=duration",
110              "-of", "default=noprint_wrappers=1:nokey=1", video_path],
111             capture_output=True, text=True, check=True,
112             ).stdout.strip()
113         return float(out)
114     except (subprocess.SubprocessError, OSError, ValueError):
115         return None
116
117
118 def slice_rallies(video_path: str, labels_csv: str, rally_numbers: List[str],
119                  out_dir: str, begin_pad: float = BEGIN_PADDING,
120                  end_pad: float = END_PADDING) -> List[str]:
121     """Write one padded clip per requested rally; returns the clip paths written."""
122     if not os.path.exists(video_path):
123         raise FileNotFoundError(f"video not found: {video_path}")
124     labels = load_rally_labels(labels_csv)
125     if not labels:
126         raise ValueError(f"no usable rally rows in {labels_csv}")
127     os.makedirs(out_dir, exist_ok=True)
128
129     duration = _probe_duration(video_path)
130     windows = compute_clip_windows(labels, rally_numbers, begin_pad, end_pad, duration)
131     if not windows:
132         raise ValueError("no rallies matched the request (check --rallies vs the CSV)")
133
134     stem = os.path.splitext(os.path.basename(video_path))[0]
135     written: List[str] = []
136     for w in windows:
137         safe_rally = w.rally.replace(".", "_")
138         out_path = os.path.join(out_dir, f"{stem}__rally_{safe_rally}.mp4")
139         cmd = [
140             "ffmpeg", "-y",
141             "-ss", str(w.start), "-to", str(w.end), "-i", video_path,
142             "-c:v", "libx264", "-preset", "superfast", "-crf", "22",
143             "-c:a", "aac", "-b:a", "128k",
144             out_path,

```

```

145         ]
146         try:
147             subprocess.run(cmd, capture_output=True, check=True)
148         except subprocess.CalledProcessError as e:
149             err = e.stderr.decode(errors="replace").strip()[-300:]
150             print(f"[slicer] rally {w.rally}: ffmpeg failed: {err}")
151             continue
152         if os.path.exists(out_path) and os.path.getsize(out_path) > 0:
153             written.append(out_path)
154             print(f"[slicer] rally {w.rally} -> {out_path}
[{{w.start:.3f}}-{{w.end:.3f}}s]")
155         return written
156
157
158     def main() -> None:
159         p = argparse.ArgumentParser(description="Slice rally clips from an annotated
video.")
160         p.add_argument("--video", required=True, help="Source video file")
161         p.add_argument("--labels", required=True, help="Labels CSV (golden rally format)")
162         p.add_argument("--rallies", required=True,
163                         help="Comma-separated rally numbers (e.g. 7,7.5,16) or 'all'")
164         p.add_argument("--out-dir", default="output/rally_clips", help="Output directory")
165         p.add_argument("--begin-pad", type=float, default=BEGIN_PADDING)
166         p.add_argument("--end-pad", type=float, default=END_PADDING)
167         args = p.parse_args()
168
169         rally_numbers = [r.strip() for r in args.rallies.split(",") if r.strip()]
170         written = slice_rallies(args.video, args.labels, rally_numbers,
171                                args.out_dir, args.begin_pad, args.end_pad)
172         print(f"\nwrote {len(written)} clip(s) to {args.out_dir}")
173
174
175     if __name__ == "__main__":
176         main()
177

```

17. assistant (2026-06-18T01:33:44.290Z)

Let me look at the hashing, audio hit_detector, and the WASB/tracknet detectors.

18. user (2026-06-18T01:33:45.033Z)

```

1     """Canonical file-identity hashing for the indexer.
2
3     `video_id` is the MD5 of the *whole* video file. Historically each endpoint
4     (`backend/main.py`, `run_eval.py`, `scripts/run_phase3_eval.py`,
5     `scripts/run_process_and_stitch.py`)
6     defined its own `get_file_md5` with a different read-chunk size (64KB / 8KB / 1MB).
7     The hash values agreed (same bytes hashed), but the duplicated code was a footgun:
8     any future change to one copy would silently diverge the DB key.
9
10    This module is the single source of truth. Import `compute_video_id` everywhere a
11    video needs to be keyed.
12    """
13
14    from __future__ import annotations
15
16    import hashlib
17
18    # 64 KB matches the value used by the live pipeline (backend/main.py) before this
19    # was centralized. The chunk size does not affect the resulting digest ? it only
20    # bounds memory use while streaming large files.
21    _CHUNK_SIZE = 65536
22

```

```

23 def compute_video_id(filepath: str) -> str:
24     """Return the MD5 hex digest of the whole file ? the canonical video_id.
25
26     Streams the file in fixed-size chunks so multi-gigabyte recordings never load
27     into memory at once.
28     """
29     hasher = hashlib.md5()
30     with open(filepath, "rb") as f:
31         for chunk in iter(lambda: f.read(_CHUNK_SIZE), b""):
32             hasher.update(chunk)
33     return hasher.hexdigest()
34

```

19. user (2026-06-18T01:33:45.595Z)

```

1     """Audio shuttle-hit ("thwack") detection ? local, dependency-light (numpy + ffmpeg).
2
3     Stages 0?1 of docs/AUDIO_RALLY_DETECTION_PLAN.md: extract the audio track, build a
4     spectral-flux onset envelope (with pre-emphasis to boost the broadband HF transient
5     a racket?shuttle contact makes), threshold it adaptively, and return (timestamp,
6     strength) hit candidates. Pure numpy + stdlib `wave` (NO scipy); ffmpeg (subprocess)
7     only for extraction. Consumed by the E1 go/no-go probe; later feeds rally fusion
8     (ADR-007) if E1 passes. WASB is never modified ? this is a separate signal.
9     """
10    from __future__ import annotations
11
12    import os
13    import wave
14    import tempfile
15    import subprocess
16    from dataclasses import dataclass
17    from typing import List, Optional, Tuple
18
19    import numpy as np
20
21
22    @dataclass
23    class HitEvent:
24        t: float          # seconds
25        strength: float    # onset-envelope peak value (arbitrary units)
26
27
28    def extract_audio(video_path: str, out_wav: Optional[str] = None, sr: int = 16000) ->
str:
29        """ffmpeg ? mono 16-bit PCM WAV at `sr`. Returns the wav path (temp if not given).
30        Audio-only decode, so it's fast even for multi-GB video."""
31        if out_wav is None:
32            fd, out_wav = tempfile.mkstemp(suffix=".wav")
33            os.close(fd)
34            cmd = ["ffmpeg", "-y", "-i", video_path, "-vn", "-ac", "1", "-ar", str(sr),
35                  "-c:a", "pcm_s16le", out_wav]
36            subprocess.run(cmd, check=True, stdout=subprocess.DEVNULL,
stderr=subprocess.DEVNULL)
37            return out_wav
38
39
40    def load_wav(path: str) -> Tuple[np.ndarray, int]:
41        """Read a PCM WAV ? (float32 samples in [-1,1], sample_rate). Mono-downmixes."""
42        with wave.open(path, "rb") as w:
43            sr, n, sw, ch = w.getframerate(), w.getnframes(), w.getsampwidth(),
w.getnchannels()
44            raw = w.readframes(n)
45            if sw == 2:
46                a = np.frombuffer(raw, dtype=np.int16).astype(np.float32) / 32768.0
47            elif sw == 4:

```

```

48         a = np.frombuffer(raw, dtype=np.int32).astype(np.float32) / 2147483648.0
49     elif sw == 1:
50         a = np.frombuffer(raw, dtype=np.uint8).astype(np.float32) / 128.0 - 1.0
51     else:
52         raise ValueError(f"unsupported sample width {sw}")
53     if ch > 1:
54         a = a.reshape(-1, ch).mean(axis=1)
55     return a, sr
56
57
58     def _preemphasis(x: np.ndarray, coef: float = 0.97) -> np.ndarray:
59         if len(x) < 2:
60             return x.copy()
61         return np.append(x[0], x[1:] - coef * x[:-1])
62
63
64     def onset_envelope(samples: np.ndarray, sr: int, frame: int = 1024,
65                        hop: int = 256, band: Optional[Tuple[float, float]] = None) ->
66     Tuple[np.ndarray, float]:
67         """Spectral-flux onset strength per hop (pure-numpy STFT). Returns (env,
68         hop_seconds).
69
70         `band=(lo_hz, hi_hz)` restricts the flux to a frequency band ? a shuttle "thwack"
71         is HF-rich while voices/footsteps live lower, so band-limiting (e.g. 3-9 kHz)
72         suppresses non-shuttle transients. Needs sr high enough for hi_hz (Nyquist =
73         sr/2)."""
74         x = _preemphasis(np.asarray(samples, dtype=np.float32))
75         hop_sec = hop / float(sr)
76         if len(x) < frame:
77             return np.zeros(0, dtype=np.float32), hop_sec
78         win = np.hanning(frame).astype(np.float32)
79         n_frames = 1 + (len(x) - frame) // hop
80         idx = np.arange(frame)[None, :] + hop * np.arange(n_frames)[:, None]
81         frames = x[idx] * win
82         mag = np.abs(np.fft.rfft(frames, axis=1))
83         if band is not None:
84             freqs = np.fft.rfftfreq(frame, d=1.0 / sr)
85             keep = (freqs >= band[0]) & (freqs <= band[1])
86             if keep.any():
87                 mag = mag[:, keep] # band-limited flux
88         diff = np.diff(mag, axis=0) # frame-to-frame change
89         flux = np.sqrt((np.maximum(diff, 0.0) ** 2).sum(axis=1)) # positive (onset) flux
90         env = np.concatenate([[0.0], flux]).astype(np.float32) # align to n_frames
91         return env, hop_sec
92
93
94     def _moving_stats(env: np.ndarray, win: int) -> Tuple[np.ndarray, np.ndarray]:
95         """Sliding-window mean and std (reflect-padded, via cumsum). win forced odd."""
96         if win % 2 == 0:
97             win += 1
98         pad = win // 2
99         p = np.pad(env.astype(np.float64), pad, mode="reflect")
100         c1 = np.cumsum(np.insert(p, 0, 0.0))
101         c2 = np.cumsum(np.insert(p * p, 0, 0.0))
102         s1 = c1[win:] - c1[:-win]
103         s2 = c2[win:] - c2[:-win]
104         mean = s1 / win
105         var = np.maximum(s2 / win - mean * mean, 0.0)
106         return mean[:len(env)], np.sqrt(var)[:len(env)]
107
108
109     def detect_hits(samples: np.ndarray, sr: int, k: float = 2.5, win_frames: int = 41,
110                    min_gap_sec: float = 0.08, band: Optional[Tuple[float, float]] = None)
111     -> List[HitEvent]:
112         """Detect impact transients: local maxima of the onset envelope that exceed an

```

```

109         adaptive threshold (moving mean + k*std), keeping the strongest within
`min_gap_sec`.
110         `k` is the sensitivity knob (lower = more hits); `band` band-limits the onset flux
111         to the shuttle-click frequencies (e.g. (3000, 9000))."""
112         env, hop_sec = onset_envelope(samples, sr, band=band)
113         if len(env) < 3:
114             return []
115         mean, std = _moving_stats(env, win_frames)
116         thr = mean + k * std
117         is_peak = (env[1:-1] >= env[:-2]) & (env[1:-1] > env[2:]) & (env[1:-1] > thr[1:-1])
118         peak_idx = np.nonzero(is_peak)[0] + 1
119         if peak_idx.size == 0:
120             return []
121         # enforce min inter-peak gap: greedily keep strongest first
122         min_gap = max(1, int(round(min_gap_sec / hop_sec)))
123         order = peak_idx[np.argsort(-env[peak_idx])]
124         taken = np.zeros(len(env), dtype=bool)
125         kept = []
126         for i in order:
127             lo, hi = max(0, i - min_gap), min(len(env), i + min_gap + 1)
128             if taken[lo:hi].any():
129                 continue
130             taken[i] = True
131             kept.append(i)
132         kept.sort()
133         return [HitEvent(t=float(i * hop_sec), strength=float(env[i])) for i in kept]
134
135
136     def detect_hits_in_video(video_path: str, sr: int = 16000, **kw) -> List[HitEvent]:
137         """Convenience: extract audio from a video and detect hits. Cleans up the temp
wav."""
138         wav = extract_audio(video_path, sr=sr)
139         try:
140             samples, real_sr = load_wav(wav)
141             return detect_hits(samples, real_sr, **kw)
142         finally:
143             try:
144                 os.remove(wav)
145             except OSError:
146                 pass
147

```

20. assistant (2026-06-18T01:33:49.233Z)

These are clean. Let me look at the WASB/tracknet detectors and the yolo_hybrid segmenter which do heavier ffmpeg/subprocess work.

21. user (2026-06-18T01:33:50.673Z)

```

1     """TrackNet shuttle-trajectory runner (WSL2 subprocess adapter).
2
3     This bridges the Windows-side indexer to a TensorFlow TrackNetV4 install living
4     inside WSL2 (ADR-001 in docs/DECISIONS.md). The flow is:
5
6         1. Translate the Windows video path to its WSL-visible form (and optionally
7            stage it into the WSL filesystem to avoid slow /mnt/c per-frame reads).
8         2. Invoke ``src/predict.py`` inside the TrackNetV4 conda env via ``wsl``.
9         3. Read back the per-frame ``Frame,Visibility,X,Y`` CSV it writes.
10        4. Convert that trajectory into high-motion "action windows" ? the same
11           candidate-rally shape HybridYoloSegmenter produces ? so the rest of the
12           pipeline (AI handoff, DB population) is unchanged.
13
14     Nothing here imports TensorFlow; all model code stays in WSL.
15     """
16

```

```

17     import os
18     import csv
19     import shlex
20     import logging
21     import subprocess
22     from dataclasses import dataclass
23     from typing import List, Dict, Any, Optional, Tuple
24
25     from backend.pipeline.detectors.base import DetectorRunner, WslCommandMixin,
wsl_tilde_quote
26
27     logger = logging.getLogger(__name__)
28
29
30     @dataclass
31     class TrackNetConfig:
32         """Resolved settings for a TrackNet WSL run (built from config.json ->
indexing.tracknet)."""
33         distro: str = "Ubuntu"
34         repo_dir: str = "~/models/TrackNetV4"          # WSL path to the cloned repo
35         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
36         conda_env: str = "TrackNetV4"
37         weights_path: str = ""                        # WSL path to the .h5/.keras weights
38         (REQUIRED)
39         queue_length: int = 5
40         stage_in_wsl: bool = True                    # copy video into WSL fs first
41         (perf)
42         wsl_stage_dir: str = "~/clips"                # where staged videos/outputs live
43         in WSL
44         timeout_sec: int = 1800                      # 30 min; kills a hung call (0 = no
timeout)
45         keep_frames: bool = False                    # retain staged video after success
46         (debug only)
47         min_free_gb: float = 0.0                     # warn if WSL free space below this
(0 = off)
48
49
50     @classmethod
51     def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "TrackNetConfig":
52         tn = (idx_cfg or {}).get("tracknet", {}) or {}
53         return cls(
54             distro=tn.get("wsl_distro", "Ubuntu"),
55             repo_dir=tn.get("repo_dir", "~/models/TrackNetV4"),
56             conda_sh=tn.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
57             conda_env=tn.get("conda_env", "TrackNetV4"),
58             weights_path=tn.get("weights_path", ""),
59             queue_length=int(tn.get("queue_length", 5)),
60             stage_in_wsl=bool(tn.get("stage_in_wsl", True)),
61             wsl_stage_dir=tn.get("wsl_stage_dir", "~/clips"),
62             timeout_sec=int(tn.get("timeout_sec", 1800)),
63             keep_frames=bool(tn.get("keep_frames", False)),
64             min_free_gb=float(tn.get("min_free_gb", 0.0)),
65         )
66
67
68     def to_wsl_mnt_path(win_path: str) -> str:
69         """Translate a Windows path (C:\\Users\\x) to its WSL /mnt form (/mnt/c/Users/x).
70
71         Already-POSIX paths (starting with / or ~) are returned unchanged so the
72         same helper is safe to call on either kind of path.
73         """
74         p = str(win_path)
75         if p.startswith("/") or p.startswith("~"):
76             return p
77         p = p.replace("\\", "/")
78         if len(p) >= 2 and p[1] == ":":

```

```

74         drive = p[0].lower()
75         return f"/mnt/{drive}{p[2:]}"
76     return p
77
78
79 @dataclass
80 class TrajectoryPoint:
81     frame: int
82     visible: bool
83     x: float
84     y: float
85
86
87 class TrackNetRunner(WslCommandMixin, DetectorRunner):
88     """Runs TrackNet predict.py in WSL and turns the output CSV into action windows.
89
90     Implements the common DetectorRunner contract so a future Linux-native or
91     alternative trajectory runner can be substituted without touching the hybrids.
92     """
93
94     def __init__(self, cfg: TrackNetConfig):
95         self.cfg = cfg
96
97     def healthcheck(self) -> Tuple[bool, str]:
98         """Verify the WSL env is usable: conda env exists, TF imports, GPU visible.
99
100         Returns (ok, message). Cheap to call before a long run.
101         """
102         cmd = (
103             f"conda activate {self.cfg.conda_env} && "
104             f"python -c \"import tensorflow as tf; "
105             f"print('TF', tf.__version__, 'GPUs', "
106             f"len(tf.config.list_physical_devices('GPU')))\n"
107             )
108         try:
109             res = self._wsl_bash(cmd)
110         except FileNotFoundError:
111             return False, "wsl` not found ? is this running on Windows with WSL2
112             installed?"
113         except subprocess.TimeoutExpired:
114             return False, "WSL healthcheck timed out."
115         out = (res.stdout or "").strip()
116         if res.returncode != 0:
117             return False, f"WSL env check failed: {(res.stderr or out).strip()}"
118         return True, out
119
120 # ----- inference
121
122 def run_predict(self, video_win_path: str, output_win_dir: str,
123                 video_id: Optional[str] = None) -> Optional[str]:
124     """Run predict.py on a video. Returns the Windows path to the produced CSV, or
125     None.
126
127     ``output_win_dir`` is a Windows directory (under the repo's output/) that
128     WSL writes into via its /mnt view, so the Windows process can read the CSV
129     back with no copy. ``video_id`` (file MD5) namespaces the staged video ? and
130     therefore predict.py's ``<stem>_predict.csv`` output ? so two videos that share
131     a filename cannot collide on the output CSV.
132     """
133     if not self.cfg.weights_path:
134         logger.error(
135             "TrackNet weights_path is not set. Set indexing.tracknet.weights_path "
136             "in config.json to the WSL path of your trained TrackNetV4 weights."
137         )
138     return None

```

```

136
137 # Resolve relative dirs BEFORE the /mnt translation ? to_wsl_mnt_path passes
138 # relative paths through unchanged, so WSL would resolve them against the
139 # predict.py cwd instead of the repo (same bug class as wasb_runner).
140 output_win_dir = os.path.abspath(output_win_dir)
141 os.makedirs(output_win_dir, exist_ok=True)
142 # Normalize separators so basename/splitext work when tests (or collector)
143 # pass Windows-style paths on a Linux host.
144 video_win_path = str(video_win_path).replace("\\", "/")
145 base = os.path.basename(video_win_path)
146 stem, ext = os.path.splitext(base)
147
148 # predict.py only accepts .avi/.mp4 and names the CSV "<stem>_predict.csv".
149 if ext.lower() not in (".mp4", ".avi"):
150     logger.error("TrackNet predict.py only supports .mp4/.avi (got %s)", ext)
151     return None
152
153 wsl_out = to_wsl_mnt_path(output_win_dir)
154 # Namespace by video_id so two same-filename videos don't collide on the CSV.
155 # predict.py derives its output name from the (staged) video stem, so staging
156 # under the namespaced name propagates into "<key>_predict.csv".
157 key = f"{stem}__{video_id[:12]}" if video_id else stem
158 expected_csv_win = os.path.join(output_win_dir, f"{key}_predict.csv")
159
160 # Resolve the video path WSL will read.
161 if self.cfg.stage_in_wsl:
162     staged = self._stage_video(video_win_path, f"{key}{ext}")
163     if staged is None:
164         return None
165     wsl_video = staged
166 else:
167     # No staging: predict.py reads /mnt directly and writes
168     # "<stem>_predict.csv".
169     # We cannot rename its output, so fall back to the un-namespaced name.
170     wsl_video = to_wsl_mnt_path(video_win_path)
171     expected_csv_win = os.path.join(output_win_dir, f"{stem}_predict.csv")
172
173 if self.cfg.min_free_gb > 0:
174     free = self._wsl_free_gb(self.cfg.wsl_stage_dir)
175     if free is not None and free < self.cfg.min_free_gb:
176         logger.warning("Low WSL disk: %.1f GB free at %s (< min_free_gb=%.1f); "
177             "consider running tools/cleanup_caches.py.",
178             free, self.cfg.wsl_stage_dir, self.cfg.min_free_gb)
179
180 cmd = (
181     f"conda activate {self.cfg.conda_env} && "
182     f"cd {self.cfg.repo_dir}/src && "
183     f"python predict.py "
184     f"--video_path {wsl_tilde_quote(wsl_video)} "
185     f"--model_weights {wsl_tilde_quote(self.cfg.weights_path)} "
186     f"--output_dir {wsl_tilde_quote(wsl_out)} "
187     f"--queue_length {self.cfg.queue_length}"
188 )
189 logger.info("Running TrackNet inference on %s ...", base)
190 try:
191     res = self._wsl_bash(cmd)
192 except subprocess.TimeoutExpired:
193     logger.error("TrackNet inference timed out after %ss.",
194         self.cfg.timeout_sec)
195     return None
196
197 if res.returncode != 0:
198     logger.error("TrackNet predict.py failed:\n%s", (res.stderr or
199         res.stdout)[-2000:])
200     return None

```



```

198
199         if not os.path.exists(expected_csv_win):
200             logger.error("TrackNet finished but expected CSV not found at %s",
expected_csv_win)
201             return None
202             logger.info("TrackNet trajectory CSV: %s", expected_csv_win)
203
204             # Success ? the CSV is the durable output; drop the staged video copy (a pure,
205             # regenerable intermediate). On earlier failure we return above without
cleaning.
206             if self.cfg.stage_in_wsl and not self.cfg.keep_frames:
207                 logger.info("Cache hygiene: removing staged video for %s "
208                             "(set indexing.tracknet.keep_frames=true to retain).", key)
209                 self._wsl_rm_rf([wsl_video])
210             return expected_csv_win
211
212     def _stage_video(self, video_win_path: str, base: str) -> Optional[str]:
213         """Copy the input video into the WSL filesystem (avoids slow /mnt/c reads).
214
215         Returns the WSL path to the staged copy, or None on failure.
216         """
217         src_mnt = to_wsl_mnt_path(video_win_path)
218         reason = self.wsl_source_unreadable_reason(src_mnt)
219         if reason:
220             logger.error("Cannot stage video into WSL: %s", reason)
221             return None
222         dest = f"{self.cfg.wsl_stage_dir}/{base}"
223         cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)}
{wsl_tilde_quote(dest)} && echo OK"
224         try:
225             res = self._wsl_bash(cmd)
226         except subprocess.TimeoutExpired:
227             logger.error("Staging video into WSL timed out.")
228             return None
229         if res.returncode != 0 or "OK" not in (res.stdout or ""):
230             logger.error("Failed to stage video into WSL: %s", (res.stderr or
res.stdout).strip())
231             return None
232         return dest
233
234     # ----- CSV -> windows
235
236     @staticmethod
237     def parse_trajectory_csv(csv_win_path: str) -> List[TrajectoryPoint]:
238         """Parse a TrackNet predict CSV (columns: Frame,Visibility,X,Y)."""
239         points: List[TrajectoryPoint] = []
240         with open(csv_win_path, newline="") as f:
241             reader = csv.DictReader(f)
242             for row in reader:
243                 try:
244                     points.append(TrajectoryPoint(
245                         frame=int(row["Frame"]),
246                         visible=int(row["Visibility"]) == 1,
247                         x=float(row["X"]),
248                         y=float(row["Y"]),
249                     ))
250                 except (KeyError, ValueError):
251                     continue
252             points.sort(key=lambda p: p.frame)
253         return points
254
255     @staticmethod
256     def trajectory_to_action_windows(
257         points: List[TrajectoryPoint],
258         fps: float,

```

```

259         frame_width: float,
260         chunk_start: float = 0.0,
261         velocity_thresh: float = 0.02,
262         merge_gap: float = 2.0,
263         min_window_duration: float = 2.0,
264         chunk_index: Optional[int] = None,
265         max_window_duration: float = 0.0,
266         min_split_gap: float = 0.5,
267         window_hard_cap: bool = False,
268         window_inactivity_end: float = 0.0,
269         inactivity_min_density: float = 0.34,
270         inactivity_rally_thresh: float = 0.08,
271         window_blob_fallback: bool = False,
272         window_blob_factor: float = 4.0,
273     ) -> List[Dict[str, Any]]:
274         """Convert a shuttle trajectory into candidate rally windows.
275
276         A frame is "active" when the shuttle is visible AND its inter-frame
277         displacement (normalised by frame width, per second) exceeds
278         ``velocity_thresh`` ? i.e. the shuttle is in flight. Active frames within
279         ``merge_gap`` seconds of each other are grouped into one window; short
280         windows are dropped. Mirrors HybridYoloSegmenter's windowing so the dict
281         shape feeding the AI-handoff phase is identical.
282
283         ``max_window_duration`` (0 = OFF, the default ? behaviour unchanged): a guard
284         against the *over-merge* failure where "shuttle moving" is conflated with "rally in
285         progress" and a single window swallows several rallies + the dead time between them. When
286         set, any window longer than this is recursively SPLIT at its largest internal inactivity
287         gap (the longest stretch with no in-flight shuttle ? the most likely rally boundary),
288         provided that gap is at least ``min_split_gap`` seconds. This is the bounded-windowing
289         fix from ``docs/RALLY_QUALITY_RESEARCH.md`` §6 (W1); it never merges, only cuts, so
290         recall cannot drop, and it is config-gated default-OFF until measured on the golden set.
291
292         ``window_hard_cap`` (default OFF) makes ``max_window_duration`` a TRUE cap: when
293         an over-long window has NO internal inactivity gap to split on (a continuously-
294         active trajectory ? the shuttle never stops moving, e.g. the real-footage
295         ``mahadevpura-1`` blob where the gap-splitter alone left a single 174 s window), it is recursively
296         hard-chopped at its temporal midpoint until every piece is ? the cap. Still only
297         cuts (never merges), so recall cannot drop; gated default-OFF until measured.
298
299         ``window_blob_fallback`` (default OFF ? the R5 last-resort blob splitter):
300         unlike ``window_hard_cap`` (which midpoint-chops BEFORE the R4 trim, so it fires on
301         every gap-less over-cap window and over-segments normal clips), this runs AFTER the R4
302         inactivity trim and force-chops ONLY a genuine *blob* ? a group still longer than
303         ``window_blob_factor`` × ``max_window_duration`` once the gap-split and the principled rally-end trim
304         have both had their chance (e.g. mahadevpura-1's ~65 s residual ? 11× a 6 s cap; a 7 s rally
305         is left alone). The blob is midpoint-chopped down to ? the cap, those forced cuts are
306         logged, and

```

```

306         each resulting window is flagged ``forced_cut=True`` ? surfacing the clip as one
that needs
307         rally-STATE cues (net-crossings / two-sided motion), not a bigger cap. Only cuts
(recall
308         can't drop). Requires ``max_window_duration`` > 0; gated default-OFF until
measured.
309         """
310         if fps <= 0:
311             fps = 30.0
312         if frame_width <= 0:
313             frame_width = 1280.0
314
315         # Compute per-frame normalised velocity from the last *visible* point.
316         active = [] # (frame_idx, velocity)
317         prev: Optional[TrajectoryPoint] = None
318         for p in points:
319             if not p.visible:
320                 prev = None # break the trajectory; absent shuttle = not in flight
321                 continue
322             if prev is not None:
323                 dframes = p.frame - prev.frame
324                 if dframes > 0:
325                     dist = ((p.x - prev.x) ** 2 + (p.y - prev.y) ** 2) ** 0.5
326                     velocity = (dist / frame_width) * (fps / dframes)
327                     if velocity > velocity_thresh:
328                         active.append((p.frame, velocity))
329             prev = p
330
331         if not active:
332             return []
333
334         max_gap_frames = int(merge_gap * fps)
335
336         def _finalize(group: List[Tuple[int, float]]) -> Dict[str, Any]:
337             vels = [v for _, v in group]
338             return {
339                 "start": group[0][0] / fps + chunk_start,
340                 "end": group[-1][0] / fps + chunk_start,
341                 "active_frame_count": len(group),
342                 "peak_velocity": max(vels),
343                 "mean_velocity": sum(vels) / len(vels),
344                 "track_id_count": 1, # single shuttle; kept for window-shape parity
345                 "chunk_index": chunk_index,
346             }
347
348         groups: List[List[Tuple[int, float]]] = []
349         group = [active[0]]
350         for af in active[1:]:
351             if af[0] - group[-1][0] <= max_gap_frames:
352                 group.append(af)
353             else:
354                 groups.append(group)
355                 group = [af]
356         groups.append(group)
357
358         if max_window_duration > 0:
359             groups = TrackNetRunner._split_overlong_groups(
360                 groups, int(max_window_duration * fps), int(min_split_gap * fps),
361                 hard_cap=window_hard_cap)
362
363         # R4 ? rally-END inactivity trim (default OFF). Applied AFTER the cap split so
each rally
364         # piece has its own post-rally drift tail removed: the velocity windowing keeps
a window
365         # "active" while the shuttle drifts/gets-picked-up above threshold, over-

```

```

extending the END
366         # (the measured gap vs golden). Trim back to the last frame whose trailing
window is still
367         # dense with motion (rally on); a sustained low-density run = rally over. Only
cuts.
368         if window_inactivity_end > 0:
369             tw = int(window_inactivity_end * fps)
370             groups = [TrackNetRunner._trim_inactive_tail(g, tw, inactivity_min_density,
371                                                         inactivity_rally_thresh) for g
in groups]
372
373         # R5 ? blob-fallback (default OFF). LAST resort: after the gap-split AND the R4
trim, any
374         # group still longer than the cap is a gap-less, rally-end-signal-less blob;
force-chop it to
375         # ? the cap at the midpoint and flag every piece, so the degenerate single-
window outcome is
376         # avoided and the clip is surfaced (logged) as needing rally-STATE cues, not a
bigger cap.
377         forced_flags = [False] * len(groups)
378         if window_blob_fallback and max_window_duration > 0:
379             groups, forced_flags = TrackNetRunner._blob_fallback_chop(
380                 groups, int(max_window_duration * fps), window_blob_factor)
381
382         windows = []
383         for g, forced in zip(groups, forced_flags):
384             w = _finalize(g)
385             if forced:
386                 w["forced_cut"] = True
387             windows.append(w)
388         return [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]
389
390     @staticmethod
391     def _trim_inactive_tail(group: List[Tuple[int, float]], trail_frames: int,
392                             min_density: float, rally_thresh: float) -> List[Tuple[int,
float]]:
393         """R4: trim the post-rally drift tail. After a rally the shuttle keeps moving
*slowly*
394         (picked up / walked / knock-up) above ``velocity_thresh``, so the velocity
windowing
395         over-extends the END. A frame is "fast" (real rally motion) iff its velocity >
``rally_thresh``
396         (higher than the active threshold). The rally is "on" at frame ``i`` while its
trailing
397         ``trail_frames`` window holds >= ``min_density`` fraction of fast frames; trim
back to the
398         LAST such frame ? everything after is sustained slow drift (rally over). Only
cuts (recall
399         can't rise); a window whose tail never goes slow is untouched, and if no fast-
dense point
400         exists at all the group is kept whole (fail-safe, no over-trim). O(n) two-
pointer; the START
401         is left alone (already ~Gemini-accurate per the n=2 boundary-error finding)."""
402         if trail_frames <= 0 or len(group) < 2:
403             return group
404         frames = [f for f, _ in group]
405         fast = [1 if v > rally_thresh else 0 for _, v in group]
406         lo = 0
407         fast_sum = 0
408         last_dense = -1
409         for i in range(len(frames)):
410             fast_sum += fast[i]
411             while frames[lo] <= frames[i] - trail_frames:
412                 fast_sum -= fast[lo]
413                 lo += 1

```

```

414         win_count = i - lo + 1
415         if win_count > 0 and fast_sum / win_count >= min_density:
416             last_dense = i
417         return group[: last_dense + 1] if last_dense >= 0 else group
418
419     @staticmethod
420     def _split_overlong_groups(groups: List[List[Tuple[int, float]]], max_win_frames:
int,
421                               min_split_frames: int,
422                               hard_cap: bool = False) -> List[List[Tuple[int, float]]]:
423         """Recursively split any active-frame group longer than ``max_win_frames`` at
its largest
424         internal gap (? ``min_split_frames``) ? an inactivity-aware cut at the most
likely rally
425         boundary. Splitting only subdivides existing groups (never merges/extends), so
it cannot
426         invent or drop coverage; a group with no gap ? the floor is left intact (a
genuinely long
427         rally) UNLESS ``hard_cap`` is set, in which case such a group is hard-chopped at
its
428         temporal midpoint until every piece is ? ``max_win_frames`` (makes the cap a
true upper
429         bound on continuously-active trajectories). Iterative worklist to bound stack
depth."""
430         if max_win_frames <= 0:
431             return groups
432         out: List[List[Tuple[int, float]]] = []
433         work = list(groups)
434         while work:
435             g = work.pop()
436             if len(g) < 2 or (g[-1][0] - g[0][0]) <= max_win_frames:
437                 out.append(g)
438                 continue
439             best_i, best_gap = -1, -1
440             for i in range(1, len(g)):
441                 gap = g[i][0] - g[i - 1][0]
442                 if gap > best_gap:
443                     best_gap, best_i = gap, i
444             if best_i < 0 or best_gap < min_split_frames:
445                 # Too long but no real dead-time gap. Default: keep as one (a genuine
long rally).
446                 # hard_cap: chop at the temporal midpoint so the cap is actually
enforced.
447                 if hard_cap:
448                     mid = (g[0][0] + g[-1][0]) / 2.0
449                     split_i = min(range(1, len(g)), key=lambda i: abs(g[i][0] - mid))
450                     work.append(g[:split_i])
451                     work.append(g[split_i:])
452                 else:
453                     out.append(g)
454                     continue
455                 work.append(g[:best_i])
456                 work.append(g[best_i:])
457             out.sort(key=lambda grp: grp[0][0])
458         return out
459
460     @staticmethod
461     def _blob_fallback_chop(groups: List[List[Tuple[int, float]]], max_win_frames: int,
462                             blob_factor: float = 4.0
463                             ) -> Tuple[List[List[Tuple[int, float]]], List[bool]]:
464         """R5 blob splitter ? the gap-less, R4-survived residual. After the W1 gap-split
AND the R4
465         inactivity trim have each had their chance, a group still longer than
``blob_factor`` x
466         ``max_win_frames`` is a genuine continuously-active blob (no internal gap, no

```

```

rally-end
467         signal the principled rules could find ? the real-footage ``mahadevpura-1`` ~65
s residual ?
468         11× a 6 s cap). The factor is what keeps this SURGICAL: a normal long rally
(~1?2× the cap)
469         is left ALONE; only the degenerate blob qualifies. As a LAST RESORT the blob is
470         midpoint-chopped to ? the cap and every resulting piece is FLAGGED as a forced
cut, so the
471         degenerate single-window outcome is avoided and the clip is surfaced (logged) as
needing
472         rally-STATE cues ? net-crossings / two-sided motion ? not a bigger cap. Only
cuts (recall
473         can't drop). Returns ``(groups, forced_flags)`` aligned 1:1; ``forced_flags[i]``
is True iff
474         that piece came from a forced chop. Iterative worklist to bound stack depth."""
475         if max_win_frames <= 0:
476             return groups, [False] * len(groups)
477         blob_frames = max_win_frames * max(1.0, blob_factor)
478         out: List[List[Tuple[int, float]]] = []
479         flags: List[bool] = []
480         forced = 0
481         for g in groups:
482             if len(g) < 2 or (g[-1][0] - g[0][0]) <= blob_frames:
483                 out.append(g)
484                 flags.append(False)
485                 continue
486             pieces: List[List[Tuple[int, float]]] = []
487             work = [g]
488             while work:
489                 h = work.pop()
490                 if len(h) < 2 or (h[-1][0] - h[0][0]) <= max_win_frames:
491                     pieces.append(h)
492                     continue
493                 mid = (h[0][0] + h[-1][0]) / 2.0
494                 split_i = min(range(1, len(h)), key=lambda i: abs(h[i][0] - mid)) #
noqa: B023
495                 work.append(h[:split_i])
496                 work.append(h[split_i:])
497             pieces.sort(key=lambda grp: grp[0][0])
498             out.extend(pieces)
499             flags.extend([True] * len(pieces))
500             forced += len(pieces) - 1
501         if forced:
502             logger.warning(
503                 "R5 blob-fallback: forced %d midpoint cut(s) on continuously-active
group(s) with "
504                 "no rally-end signal ? this clip needs rally-STATE cues (net-crossings /
two-sided "
505                 "motion), not a bigger cap", forced)
506         return out, flags
507

```

22. assistant (2026-06-18T01:33:54.121Z)

Let me check the base detector (WslCommandMixin) and the wasb runner/infer which do the heavy subprocess work.

23. user (2026-06-18T01:33:54.916Z)

```

1         """Abstract base for detector runners (TrackNet, WASB, future native, etc.).
2
3         This completes the Detector Runner Abstraction (low-regret item 6 continuation / item 1
4         of the post-observability re-prioritization) to de-risk the WSL/conda dependency for
5         the high-accuracy shuttle trajectory substrate used by tracknet_hybrid / wasb_hybrid.
6

```

```

7 The ABC lets us:
8 - Keep the current WSL adapters working unchanged for Windows users.
9 - Swap in (or A/B test) a Linux-native implementation later with zero changes to
10 the hybrid segmenters or the model-agnostic windowing brain.
11 - Provide a single place to evolve the healthcheck / error contract and thread
12 detector readiness into per-video reports.
13
14 Contract (deliberately matches what the concrete runners + hybrids + tests already did):
15 - run_predict(video_win_path: str, output_win_dir: str) -> Optional[str]
16   Run detector, return the Windows-side path to the produced trajectory CSV
17   (e.g. ".../clip_predict.csv" or ".../clip_wasb.csv"), or None on any failure.
18   The caller (hybrid) is responsible for output dir; impls may stage the video
19   into WSL fs for perf.
20
21 - healthcheck() -> tuple[bool, str]
22   Return (ok, message). Never raises. "ok" decides whether to proceed to run_predict.
23   Message is logged (and can be surfaced to reports or UX in future).
24   Intended to be cheap (env existence, import, GPU visibility, weights present).
25
26 Existing behavior is 100% preserved. The only additions are the inheritance and
27 the explicit common type for future implementers.
28
29 How to add a new runner (for docs / future contributors):
30 1. Subclass DetectorRunner in a new or existing module under pipeline/detectors/.
31 2. Implement run_predict and healthcheck (return (bool, str) tuple).
32 3. If it produces the same CSV schema (Frame,Visibility,X,Y), reuse or re-export
33   TrackNetRunner.parse_trajectory_csv and .trajectory_to_action_windows (they are
34   deliberately static and model-agnostic). If the new detector needs different
35   windowing, it can implement its own or we generalize later.
36 4. Lazy-import heavy native deps inside the methods (so importing the segmenter
37   registry never pulls torch/tf on a machine that doesn't have them).
38 5. Update the two hybrid segmenters (or introduce a detector= choice) to accept
39   DetectorRunner instances (they already do the right thing via duck-typing + the
40   type hints we add here).
41 6. Add unit tests that exercise the contract (isinstance, healthcheck shape, run
42   returns str|None) using mocks for the external bits ? see test_tracknet_runner.py.
43 7. No change to eval/calibration code paths (they consume the CSV + the static
44   windowing helpers directly).
45
46 Optional Linux-native path (the main de-risk goal):
47 A LinuxNativeRunner (or ONNXRunner) would:
48 - Skip all wsl / _stage / conda activate.
49 - Load weights locally (torch or onnx) or call a native binary.
50 - Write the exact same CSV format so trajectory_to_action_windows works verbatim.
51 - healthcheck can just check "weights file exists and torch importable + CUDA?".
52 This removes the WSL requirement for Linux users / future SaaS workers while
53 keeping the exact same candidate quality characteristics (or better, once native
54 weights are available).
55
56 Error surfacing into reports:
57 The healthcheck message is already logged by the hybrids on failure. The reporting
58 harvest sees the outcome as "0 candidates" + detector name. When we wire richer
59 segmenter Results (or pass detector_health into emit_indexer_report context), the
60 (ok, msg) can be recorded explicitly under segmentation.details or a new detector
61 stage. The ABC makes that future change local to one place.
62
63 See also:
64 - backend/pipeline/segmenters/{tracknet_hybrid,wasb_hybrid}.py (the two callers)
65 - tests/test_tracknet_runner.py (contract + pure logic tests; hybrids mock the runner)
66 - docs/TRACKNET_WSL_SETUP.md and CODE_MAP.md (2.2 Detectors section)
67 ""
68
69 import logging
70 import shlex
71 import subprocess

```

```

72 from abc import ABC, abstractmethod
73 from typing import Any, Callable, List, Optional, Tuple
74
75 logger = logging.getLogger(__name__)
76
77
78 def wsl_tilde_quote(path: str) -> str:
79     """shlex.quote that still lets bash expand a leading `~/`.
80
81     ``shlex.quote("~/clips/x.mp4")`` single-quotes the whole path, so bash
82     treats ``~`` literally and ``cp``/``mkdir`` fail with "No such file or
83     directory" ? this broke live WSL staging for the default ``~/clips``
84     stage dir (found by the 2026-06-11 wasb_hybrid smoke run; the BACKLOG
85     "shlex-quote breaks ~ expansion" item). Quoting only the part after
86     ``~/`` keeps expansion AND protects spaces/specials in the remainder.
87     """
88     if path == "~":
89         return path
90     if path.startswith("~/"):
91         return "~/ " + shlex.quote(path[2:])
92     return shlex.quote(path)
93
94
95 class WslCommandMixin:
96     """Shared ``wsl -d <distro> bash -c 'source <conda_sh> && <cmd>'`` invocation.
97
98     Lives beside (not on) DetectorRunner: a future Linux-native runner must not
99     inherit WSL plumbing. Requires ``self.cfg`` with ``distro``, ``conda_sh``
100     and ``timeout_sec`` attributes (TrackNetConfig and WasbConfig both qualify).
101     ``timeout_sec == 0`` maps to None = wait forever (configs default 1800s).
102     """
103
104     cfg: Any
105
106     def _wsl_bash(self, inner_cmd: str) -> subprocess.CompletedProcess:
107         full = f"source {self.cfg.conda_sh} && {inner_cmd}"
108         argv = ["wsl", "-d", self.cfg.distro, "bash", "-c", full]
109         logger.debug("WSL exec: %s", full)
110         return subprocess.run(argv, capture_output=True, text=True,
111                               timeout=(self.cfg.timeout_sec or None))
112
113     # ---- cache hygiene helpers (shared by both WSL runners) -----
114     def _wsl_free_gb(self, wsl_path: str) -> Optional[float]:
115         """Best-effort free space (GB) on the WSL filesystem holding ``wsl_path``.
116
117         Returns None if it can't be determined (never raises). Used as a pre-run
118         disk guard so a long extraction doesn't fill the disk mid-flight.
119         """
120         try:
121             res = self._wsl_bash(f"df -P -B1 {wsl_tilde_quote(wsl_path)} | awk
122             'NR==2{{print $4}}'")
123             except (subprocess.TimeoutExpired, OSError):
124                 return None
125             try:
126                 return int((res.stdout or "").strip()) / (1024 ** 3)
127             except (ValueError, AttributeError):
128                 return None
129
130     def _wsl_rm_rf(self, wsl_paths: List[str]) -> None:
131         """Best-effort recursive delete of WSL paths (post-success cleanup; never
132         raises)."""
133         paths = [p for p in wsl_paths if p]
134         if not paths:
135             return
136         quoted = " ".join(wsl_tilde_quote(p) for p in paths)

```



```

135         try:
136             self._wsl_bash(f"rm -rf {quoted}")
137         except (subprocess.TimeoutExpired, OSError) as e:
138             logger.warning("WSL cache cleanup failed (non-fatal): %s", e)
139
140     def wsl_source_unreadable_reason(self, src_mnt: str) -> Optional[str]:
141         """Actionable message if ``src_mnt`` is NOT readable from inside WSL, else None.
142
143         Google Drive (``G:``/Drive File Stream), UNC and many network/virtual
144         filesystems are invisible to WSL's ``/mnt`` ? staging a video from such a path
145         fails with a cryptic ``cp: cannot stat`` and the whole run silently produces no
146         trajectory (i.e. "0 rallies", a wasted run that looks like a quality result).
147         A cheap ``test -r`` preflight turns that into one clear, fixable error BEFORE
148         any
149         frame extraction. Best-effort: a flaky/timed-out probe returns None so the real
150         ``cp`` still gets its chance (we never block a genuinely-readable source).
151         """
152         try:
153             res = self._wsl_bash(f"test -r {shlex.quote(src_mnt)} && echo READABLE")
154         except (subprocess.TimeoutExpired, OSError):
155             return None
156         if "READABLE" in (res.stdout or ""):
157             return None
158         return (
159             f"video source is not readable from WSL at '{src_mnt}'. Google Drive, UNC, "
160             f"and other network/virtual filesystems are invisible to WSL's /mnt ? copy "
161             f"the video to a local disk (e.g. C:\\ or the WSL filesystem) and retry."
162         )
163
164     class DetectorRunner(ABC):
165         """Common interface for shuttle/ball trajectory detectors (WSL or otherwise)."""
166
167         @abstractmethod
168         def run_predict(self, video_win_path: str, output_win_dir: str,
169                        video_id: Optional[str] = None) -> Optional[str]:
170             """Run inference and return path to trajectory CSV (or None on failure).
171
172             ``video_id`` (the file MD5) namespaces all staged/cached/output artifacts so two
173             videos that happen to share a filename cannot reuse each other's frames or CSV.
174             It is optional for back-compat; callers in the live pipeline always pass it.
175             """
176             pass
177
178         @abstractmethod
179         def healthcheck(self) -> Tuple[bool, str]:
180             """Quick check if the runner (env, weights, etc.) is usable.
181
182             Returns (ok, message). Never raises. Cheap to call before a long run.
183             """
184             pass
185
186         # De facto trajectory contract: the trajectory hybrids call these
187         # unconditionally after run_predict. Concrete runners reuse/re-export the
188         # model-agnostic TrackNetRunner staticmethods (same CSV schema + windowing);
189         # a runner with custom windowing supplies its own with the same shapes.
190         parse_trajectory_csv: Callable[..., Any]
191         trajectory_to_action_windows: Callable[..., Any]
192

```

24. assistant (2026-06-18T01:33:57.553Z)

Let me look at the wasb_infer.py and the validators which use ffmpeg/ffprobe. Also let me check the main.py and run_telemetry.py.

25. user (2026-06-18T01:33:58.990Z)

```
1      """WASB shuttle-trajectory inference on an arbitrary frame sequence (runs in WSL).
2
3      WASB-SBDT only ships *dataset* evaluation; this is a thin wrapper that reuses its
4      own building blocks (ImageDataset transform + detector + online tracker) to run on
5      any folder of extracted frames and emit a trajectory CSV in the exact shape the
6      indexer's `tracknet_runner.parse_trajectory_csv` already consumes:
7
8          Frame,Visibility,X,Y
9
10     It must run INSIDE the WSL `wasb` conda env, from the WASB-SBDT `src/` dir (it
11     imports WASB's `detectors`/`trackers`/`dataloaders`). The Windows-side adapter
12     (`wasb_runner.py`) stages this file + the frames into WSL and invokes it.
13
14     Resumable across reboots
15     -----
16     The GPU detector pass (one forward per sliding window ? ~21k for a 6-min clip) is
17     the slow, expensive part; the CPU tracker pass that turns detections into a
18     trajectory is cheap and *stateful* (it must see every frame in order, so it can't
19     resume mid-stream). We exploit that split:
20
21     * The detector pass writes its per-window raw outputs incrementally to
22       ``<cache>/det_raw.jsonl`` (flushed+fsync'd every ``--flush-every`` windows) and
23       records progress in ``<cache>/manifest.json`` (atomic write). A reboot loses at
24       most ``--flush-every`` windows of GPU work instead of the whole pass.
25     * On restart we replay the cached windows, skip the DataLoader ahead to the first
26       un-cached window, and continue. Once every window is cached, the detector pass
27       is skipped entirely and we just re-run the (cheap, deterministic) tracker over
28       the full ordered cache -> trajectory CSV. So a lost trajectory CSV costs seconds,
29       not the GPU hour.
30
31     The cache is keyed by the output path (``<out>_wasbcache/`` by default), lives next
32     to the output (NOT in %TEMP%), and is invalidated automatically if the frames dir,
33     weights, sport, window size, or frame count change.
34
35     Usage (in WSL, from ~/models/WASB-SBDT/src):
36     python wasb_infer.py --frames_dir /path/frames --weights
37     ../pretrained_weights/wasb_badminton_best.pth.tar \
38     --out /path/traj.csv [--sport badminton] [--limit N] [--cache-dir DIR] [--flush-
39     every 1000] [--fresh]
40     """
41     import os
42     import os.path as osp
43     import csv
44     import glob
45     import json
46     import argparse
47     import logging
48     from collections import defaultdict
49     from contextlib import contextmanager
50
51     logger = logging.getLogger(__name__)
52
53     CACHE_VERSION = 1
54     _DET_FILE = "det_raw.jsonl"
55     _MANIFEST_FILE = "manifest.json"
56
57     def _build_cfg(weights: str, sport: str):
58         """Compose the WASB eval config via Hydra, overriding model/weights/device."""
59         from hydra import compose, initialize
60         with initialize(config_path="configs", version_base=None):
61             cfg = compose(config_name="eval", overrides=[
62                 f"dataset={sport}",
```

```

62         "model=wasb",
63         f"detector.model_path={weights}",
64         "runner.device=cuda",
65         "runner.gpus=[0]",          # this box has a single GPU; default config assumes
more
66     ])
67     return cfg
68
69
70     def _frame_id(path: str) -> int:
71         """Best-effort integer frame id from a frame filename (e.g. frame_000180.png ->
180)."""
72         stem = osp.splitext(osp.basename(path))[0]
73         digits = "".join(ch for ch in stem if ch.isdigit())
74         return int(digits) if digits else -1
75
76
77     # ----- #
78     # Resumable detector cache (pure-Python, no torch/numpy ? unit-testable)
79     # ----- #
80     def _cache_paths(cache_dir: str):
81         return osp.join(cache_dir, _DET_FILE), osp.join(cache_dir, _MANIFEST_FILE)
82
83
84     def default_cache_dir(out_csv: str) -> str:
85         """Stable cache dir next to the trajectory output (survives reboots; not %TEMP%)."""
86         d = osp.dirname(osp.abspath(out_csv))
87         stem = osp.splitext(osp.basename(out_csv))[0]
88         return osp.join(d, stem + "_wasbcache")
89
90
91     def _atomic_write_text(path: str, text: str) -> None:
92         """Write then rename so a crash mid-write can't leave a half-written file."""
93         tmp = path + ".tmp"
94         with open(tmp, "w") as f:
95             f.write(text)
96             f.flush()
97             os.fsync(f.fileno())
98         os.replace(tmp, path)
99
100
101     def _det_plain(det) -> list:
102         """A detector candidate dict {'xy': np.array([x,y]), 'score', 'scale'} -> JSON-able
list."""
103         xy = det["xy"]
104         return [float(xy[0]), float(xy[1]), float(det["score"]), int(det["scale"])]
105
106
107     def _dets_to_tracker(plain_list):
108         """Inverse of _det_plain: rebuild the dicts the tracker expects (xy MUST be a numpy
array,
109         the tracker does vector math / np.linalg.norm on it)."""
110         import numpy as np
111         return [{"xy": np.array([p[0], p[1]], dtype=float), "score": p[2], "scale": p[3]}
112                 for p in plain_list]
113
114
115     def write_manifest(cache_dir: str, manifest: dict) -> None:
116         _, mpath = _cache_paths(cache_dir)
117         _atomic_write_text(mpath, json.dumps(manifest, indent=2))
118
119
120     def read_manifest(cache_dir: str):
121         _, mpath = _cache_paths(cache_dir)
122         if not osp.exists(mpath):

```

```

123         return None
124     try:
125         with open(mpath) as f:
126             return json.load(f)
127     except (json.JSONDecodeError, OSError):
128         return None
129
130
131     def manifest_compatible(manifest: dict, *, frames_dir: str, weights: str, sport: str,
132                             frames_in: int, frames_total: int) -> bool:
133         """A cache is reusable only if the run that produced it matches this run's
134         inputs."""
135         if not manifest or manifest.get("version") != CACHE_VERSION:
136             return False
137         return (
138             manifest.get("frames_dir") == osp.abspath(frames_dir)
139             and manifest.get("weights") == weights
140             and manifest.get("sport") == sport
141             and manifest.get("frames_in") == frames_in
142             and manifest.get("frames_total") == frames_total
143         )
144
145     def reset_cache(cache_dir: str) -> None:
146         """Drop any existing cache so the next run starts clean."""
147         det_jsonl, mpath = _cache_paths(cache_dir)
148         for p in (det_jsonl, mpath):
149             if osp.exists(p):
150                 os.remove(p)
151
152
153     def load_and_clean_cache(cache_dir: str):
154         """Read the longest *contiguous* (w == line index) valid prefix of det_raw.jsonl,
155         rebuild the per-frame detection accumulation, and rewrite the file to exactly that
156         prefix so a trailing half-written line from a crash can't corrupt future appends.
157
158         Returns (windows_done, det_by_fid) where det_by_fid maps frame_id -> list of
159         [x, y, score, scale] candidates (accumulated across every window the frame is in,
160         in window order ? identical to a non-resumed run).
161         """
162         det_jsonl, _ = _cache_paths(cache_dir)
163         det_by_fid = defaultdict(list)
164         valid: list = []
165         if osp.exists(det_jsonl):
166             with open(det_jsonl, "r") as f:
167                 for line in f:
168                     s = line.strip()
169                     if not s:
170                         continue
171                     try:
172                         obj = json.loads(s)
173                     except json.JSONDecodeError:
174                         break # truncated trailing line (crash mid-
flush)
175                     if obj.get("w") != len(valid): # non-contiguous -> stop at the gap
176                         break
177                     valid.append(s)
178                     for fid, plain in obj["f"]:
179                         det_by_fid[int(fid)].extend([list(p) for p in plain])
180                 # Guarantee a clean append boundary by rewriting only the good prefix.
181                 _atomic_write_text(det_jsonl, ("\n".join(valid) + "\n") if valid else "")
182         return len(valid), det_by_fid
183
184
185     def _append_windows(fh, objs) -> None:

```

```

186         """Append window records as JSONL and durably flush (bounded loss on crash)."""
187         for o in objs:
188             fh.write(json.dumps(o, separators=(",", ":")) + "\n")
189         fh.flush()
190         os.fsync(fh.fileno())
191
192
193     def _atomic_write_trajectory(out_csv: str, rows) -> None:
194         os.makedirs(osp.dirname(osp.abspath(out_csv)), exist_ok=True)
195         tmp = out_csv + ".tmp"
196         with open(tmp, "w", newline="") as f:
197             w = csv.writer(f)
198             w.writerow(["Frame", "Visibility", "X", "Y"])
199             w.writerows(rows)
200             f.flush()
201             os.fsync(f.fileno())
202         os.replace(tmp, out_csv)
203
204
205     # ----- #
206     # Frame addressing + windowing (pure; no torch/cv2 ? unit-testable)
207     # ----- #
208     _STREAM_FRAME_FMT = "{:08d}.png"
209
210
211     def synthetic_frame_path(index: int) -> str:
212         """Stable, index-encoding frame name used by the streaming path.
213
214         It mirrors the on-disk names ``extract_frames`` writes (``{i:08d}.png``) so the
215         downstream integer-id parser (``_frame_id``) yields the SAME frame id whether the
216         frames came from disk or from the in-memory stream. The streaming image loader
217         inverts this name back to an index to fetch the decoded frame.
218         """
219         return _STREAM_FRAME_FMT.format(int(index))
220
221
222     def build_windows(frames: list, frames_in: int):
223         """Sliding windows of ``frames_in`` consecutive frame *paths* (the unit of GPU work
224         + checkpoint). Pure list math, identical for the disk and streaming paths.
225
226         Returns a list of windows, each a list of ``frames_in`` consecutive paths
227         (``[frames[i:i+frames_in] for i in range(len(frames) - frames_in + 1)]``). The
228         caller wraps each window into the ``{"frames", "annos"}`` sample shape WASB's
229         ``ImageDataset`` expects; keeping that out of here stays torch-free for unit tests.
230         """
231         return [frames[i:i + frames_in] for i in range(len(frames) - frames_in + 1)]
232
233
234     class SequentialFrameStore:
235         """In-memory sliding buffer over a forward-only frame reader, sized for the
236         detector's sliding-window access pattern (peak 0(window) frames, not 0(video)).
237
238         The detector DataLoader runs with ``shuffle=False`` and (in streaming mode)
239         ``num_workers=0``. ``ImageDataset.__getitem__(i)`` reads the frames of window ``i``
240         in order ? ``i, i+1, ..., i+window-1`` ? and samples are requested ``i = start,
241         start+1, ...``. So the global read sequence dips back by ``window-1`` at each window
242         boundary (``0,1,2, 1,2,3, 2,3,4, ...`` for ``window=3``); the highest index seen so
243         far never decreases. We keep only frames at or above ``max_seen - (window-1)`` (the
244         earliest index any not-yet-finished window can still ask for) and decode each source
245         frame exactly once via the forward-only ``reader(index)``.
246         """
247
248         def __init__(self, reader, total: int, window: int = 1, start_index: int = 0):
249             self._reader = reader
250             self._total = int(total)

```

```

251         self._window = max(1, int(window))
252         self._buf: dict = {}
253         # On a resumed run the detector's first needed frame is `start_index`; begin
254         # pulling there so the reader can skip (seek past) the already-cached prefix
255         # instead of decoding 0..start_index-1 just to throw them away.
256         self._next = int(start_index)          # next index not yet pulled from the reader
257         self._max_seen = int(start_index) - 1
258         self._floor = int(start_index)         # lowest index we still keep (never evict
below)
259
260     def get(self, index: int):
261         index = int(index)
262         if index < self._floor:
263             raise KeyError(
264                 f"frame {index} already evicted (below floor {self._floor}); the access
"
265                 f"pattern must stay within a window of the highest frame seen")
266         if index >= self._total:
267             raise KeyError(f"frame {index} out of range (total {self._total})")
268         # Pull forward from the reader until the requested index is buffered.
269         while self._next <= index:
270             self._buf[self._next] = self._reader(self._next)
271             self._next += 1
272         if index > self._max_seen:
273             self._max_seen = index
274         # Evict frames older than the earliest a still-running window could re-request:
275         # once we've seen index `m`, no future window starts before `m - (window-1)`.
276         new_floor = max(self._floor, self._max_seen - (self._window - 1))
277         for k in range(self._floor, new_floor):
278             self._buf.pop(k, None)
279         self._floor = new_floor
280         return self._buf[index]
281
282
283     def _index_from_synthetic_path(path: str) -> int:
284         """Invert ``synthetic_frame_path``: a frame path -> its integer source index.
285
286         Reuses ``_frame_id`` (digits-only parse) so the index and the cached frame-id stay
287         in lockstep with the on-disk naming scheme.
288         """
289         return _frame_id(path)
290
291
292     def _bgr_to_pil_rgb(frame_bgr):
293         """Convert an in-memory BGR uint8 frame (as ``cv2.VideoCapture`` yields) to the
EXACT
294         PIL RGB image WASB's disk path produces.
295
296         The disk path is ``frame_bgr -> cv2.imwrite(PNG) ->
Image.open(PNG).convert('RGB')``;
297         because PNG is lossless that round-trip equals ``cv2.cvtColor(frame_bgr,
COLOR_BGR2RGB)``
298         bit-for-bit (verified empirically, max|diff|=0). Returning that here makes the
streamed
299         frame indistinguishable from the on-disk one to everything downstream.
300         """
301         import cv2
302         from PIL import Image
303         return Image.fromarray(cv2.cvtColor(frame_bgr, cv2.COLOR_BGR2RGB))
304
305
306     def _make_streamed_read_image(store, real_read_image):
307         """Build the ``read_image`` replacement used during a streaming detector pass.
308
309         A synthetic frame path -> the in-memory decoded frame for its index (as a PIL RGB

```

```

image,
310     matching ``read_image``'s real return type); any path that doesn't parse to an index
falls
311     through to ``real_read_image``. Pure ? imports no WASB module ? so it is unit-
testable
312     without the WSL-only ``dataloaders`` package.
313     """
314     def _read_image(path, *args, **kwargs):
315         idx = _index_from_synthetic_path(path)
316         if idx < 0:
317             return real_read_image(path, *args, **kwargs)
318         return _bgr_to_pil_rgb(store.get(idx))
319     return _read_image
320
321
322     @contextmanager
323     def _streaming_read_image_patch(frame_loader, total: int, window: int = 1, start_index:
int = 0):
324         """Scope a shim over WASB's ``read_image`` so ``ImageDataset`` is fed in-memory
decoded
325         frames during the detector pass, byte-for-byte as it would over real PNGs.
326
327         WASB's ``ImageDataset.__getitem__`` reads each frame via ``read_image(path)`` ? NOT
328         ``cv2.imread``. ``read_image`` (``utils.utils``) does
329         ``Image.open(path).convert('RGB')``,
330         returning a PIL **RGB** image, after an ``osp.exists(path)`` guard. We therefore
feed it a
331         PIL RGB image built from the streamed BGR frame (see ``_bgr_to_pil_rgb``), which is
332         byte-identical to the disk round-trip, and the shim never touches the filesystem so
the
333         ``osp.exists`` guard is sidestepped for synthetic stream paths.
334
335         We patch ``dataloaders.dataset_loader.read_image`` ? the binding the consumer
actually
336         calls (``dataset_loader`` does ``from utils import read_image``, so the name lives
in that
337         module's namespace; patching ``utils.read_image`` would NOT rebind the already-
imported
338         reference).
339
340         ``frame_loader(index) -> BGR uint8 ndarray`` is a forward-only sequential decoder
wrapped
341         in a ``SequentialFrameStore`` (sliding buffer sized to ``window`` = ``frames_in``);
on a
342         resume we start at ``start_index`` so the decoder seeks past already-cached windows.
The
343         original reader is restored on exit.
344         """
345         from dataloaders import dataset_loader as _dl
346         store = SequentialFrameStore(frame_loader, total, window=window,
start_index=start_index)
347         real_read_image = _dl.read_image
348         # Rebind read_image in the consuming module for the duration of the detector pass;
349         # restore on exit. (Plain assignment, not setattr-with-constant, to stay ruff
B010-clean.)
350         _dl.read_image = _make_streamed_read_image(store, real_read_image) # type:
ignore[assignment]
351         try:
352             yield store
353         finally:
354             _dl.read_image = real_read_image # type: ignore[assignment]
355
356     # ----- #
357     # Inference

```

```

358 # ----- #
359 def run(frames, weights: str, out_csv: str, sport: str = "badminton",
360         limit: int = 0, batch_size: int = 8, num_workers: int = 4,
361         log_every_batches: int = 50, cache_dir: "str | None" = None,
362         flush_every: int = 1000, fresh: bool = False,
363         frame_loader=None, source_key: "str | None" = None) -> str:
364     """Run WASB detection+tracking over an ordered frame source -> trajectory CSV.
365
366     Two equivalent ways to supply pixels (output is bit-identical between them):
367
368     * **frames-on-disk** (default): ``frames`` is a directory path string; frames are
369       globbed (``*.png``/``*.jpg``) and ``ImageDataset`` reads each file via
370       ``cv2.imread``.
371     * **streaming** (fast path): ``frames`` is a pre-built ordered list of (synthetic)
372       frame paths and ``frame_loader(index) -> BGR uint8 ndarray`` supplies the decoded
373       pixels in-memory. We scope a shim over WASB's ``read_image`` (the actual per-frame
374       reader, which returns a PIL RGB image) over the DataLoader pass so every other
375       line of
376       ``ImageDataset.__getitem__`` runs UNCHANGED ? the tensor the detector sees is
377       identical
378       to the disk round-trip (``cv2.imwrite`` PNG -> ``Image.open().convert('RGB')`` ==
379       ``cvtColor(bgr, BGR2RGB)``), verified byte-identical). Streaming forces
380       ``num_workers=0``
381       (the in-process shim + buffer can't cross worker processes).
382
383     ``source_key`` overrides the cache-keying identity (defaults to the abspath of the
384     frames dir); the streaming path passes a stable ``video:<abspath>`` key so a resume
385     after reboot still matches.
386     """
387     import time
388     import torch
389     from torch.utils.data import DataLoader
390     from detectors import build_detector
391     from trackers import build_tracker
392     from dataloaders import build_img_transforms, build_seq_transforms
393     from dataloaders.dataset_loader import ImageDataset
394     from utils import Center
395
396     streaming = frame_loader is not None
397
398     cfg = _build_cfg(weights, sport)
399     frames_in = int(cfg["model"]["frames_in"])
400     input_wh = (int(cfg["model"]["inp_width"]), int(cfg["model"]["inp_height"]))
401     output_wh = (int(cfg["model"]["out_width"]), int(cfg["model"]["out_height"]))
402
403     if streaming:
404         # `frames` is already the ordered list of (synthetic) frame paths.
405         if not isinstance(frames, (list, tuple)):
406             raise SystemExit("streaming mode requires `frames` to be a list of frame
407             paths")
408         frames = list(frames)
409         frames_dir = source_key or "stream"
410     else:
411         frames_dir = frames
412         frames = sorted(glob.glob(osp.join(frames_dir, "*.png"))) +
413         glob.glob(osp.join(frames_dir, "*.jpg"))
414         if limit:
415             frames = frames[:limit]
416         if len(frames) < frames_in:
417             where = frames_dir if not streaming else "video stream"
418             raise SystemExit(f"need >= {frames_in} frames, found {len(frames)} in {where}")
419
420     # Sliding windows of `frames_in` consecutive frames (the unit of GPU work +
421     checkpoint).
422     samples = []

```



```

416         for window in build_windows(frames, frames_in):
417             annos = [{"frame_path": p, "center": Center(is_visible=False, x=-1.0, y=-1.0)}
for p in window]
418             samples.append({"frames": window, "annos": annos})
419             windows_total = len(samples)
420
421             # ---- cache setup / resume decision ----- #
422             if cache_dir is None:
423                 cache_dir = default_cache_dir(out_csv)
424                 os.makedirs(cache_dir, exist_ok=True)
425                 det_jsonl, _ = _cache_paths(cache_dir)
426                 cache_key = source_key if source_key is not None else osp.abspath(frames_dir)
427                 manifest = None if fresh else read_manifest(cache_dir)
428                 resume = manifest_compatible(
429                     manifest or {}, frames_dir=cache_key, weights=weights, sport=sport,
430                     frames_in=frames_in, frames_total=len(frames),
431                 )
432                 if resume:
433                     windows_done, det_by_fid = load_and_clean_cache(cache_dir)
434                     logger.info(f"resuming from cache: {windows_done}/{windows_total} windows
already detected")
435                 else:
436                     if manifest is not None:
437                         logger.info("cache incompatible with this run -> starting fresh")
438                         reset_cache(cache_dir)
439                         windows_done, det_by_fid = 0, defaultdict(list)
440
441                 base_manifest = {
442                     "version": CACHE_VERSION,
443                     "frames_dir": cache_key,
444                     "weights": weights,
445                     "sport": sport,
446                     "frames_in": frames_in,
447                     "frames_total": len(frames),
448                     "windows_total": windows_total,
449                     "windows_done": windows_done,
450                 }
451                 write_manifest(cache_dir, base_manifest)
452
453             # ---- detector pass over the un-cached windows ----- #
454             remaining = samples[windows_done:]
455             if remaining:
456                 _, transform_test = build_img_transforms(cfg)
457                 try:
458                     _, seq_transform_test = build_seq_transforms(cfg)
459                 except Exception:
460                     seq_transform_test = None
461                 ds = ImageDataset(cfg, remaining, input_wh, output_wh,
462                                 transform=transform_test, seq_transform=seq_transform_test,
is_train=False)
463                 # Streaming forces single-process loading: the in-memory frame store + the
464                 # cv2.imread shim live in THIS process and can't be pickled to DataLoader
workers.
465                 loader_workers = 0 if streaming else num_workers
466                 loader = DataLoader(ds, batch_size=batch_size, shuffle=False,
num_workers=loader_workers)
467                 detector = build_detector(cfg)
468
469                 from contextlib import ExitStack
470
471                 t0 = time.time()
472                 done = windows_done
473                 win_idx = windows_done
474                 log_every = max(1, log_every_batches)
475                 flush_n = max(1, flush_every)

```

```

476         pending = []
477         logger.info(f"detecting on {len(remaining)} remaining windows "
478                    f"(total {windows_total}, batch={batch_size},
workers={loader_workers}, "
479                    f"source={'video-stream' if streaming else 'frames-dir'})...")
480         det_f = open(det_jsonl, "a")
481         try:
482             with ExitStack() as stack:
483                 stack.enter_context(torch.no_grad())
484                 # In streaming mode, route ImageDataset's per-frame `cv2.imread(path)`
to the
485                 # in-memory decoded frame for that synthetic path; every other line of
486                 # `__getitem__` runs unchanged so the tensor matches the PNG round-trip
exactly.
487                 # The detector's first needed frame is `windows_done` (window i starts
at
488                 # frame i), so prime the store there for a resume.
489                 if streaming:
490                     stack.enter_context(
491                         _streaming_read_image_patch(frame_loader, len(frames),
492                                                    window=frames_in,
start_index=windows_done))
493                 for bi, batch in enumerate(loader):
494                     imgs, _hms, trans = batch[0], batch[1], batch[2]
495                     img_paths = [list(t) for t in batch[-1]] # [frame_in][batch] ->
path
496                     batch_results, _ = detector.run_tensor(imgs, trans)
497                     nb = imgs.shape[0]
498                     for ib in range(nb):
499                         frames_payload = []
500                         for ie in sorted(batch_results[ib].keys()):
501                             p = img_paths[ie][ib]
502                             fid = _frame_id(p)
503                             plain = [_det_plain(d) for d in batch_results[ib][ie]]
504                             frames_payload.append([fid, plain])
505                             det_by_fid[fid].extend(plain)
506                             pending.append({"w": win_idx, "f": frames_payload})
507                             win_idx += 1
508                     done += nb
509                     if len(pending) >= flush_n or done >= windows_total:
510                         _append_windows(det_f, pending)
511                         pending = []
512                         base_manifest["windows_done"] = done
513                         write_manifest(cache_dir, base_manifest)
514                     if (bi + 1) % log_every == 0 or done >= windows_total:
515                         el = time.time() - t0
516                         new_done = done - windows_done
517                         rate = new_done / el if el > 0 else 0.0
518                         eta = (windows_total - done) / rate if rate > 0 else 0.0
519                         logger.info(f"{done}/{windows_total} "
520                                   f"({100 * done // max(1, windows_total)}%) |
{rate:.1f} win/s | "
521                                   f"elapsed {el:.0f}s | ETA {eta:.0f}s")
522                     if pending:
523                         _append_windows(det_f, pending)
524                         base_manifest["windows_done"] = done
525                         write_manifest(cache_dir, base_manifest)
526                 finally:
527                     det_f.close()
528                 logger.info(f"detection done in {time.time() - t0:.0f}s; running tracker...")
529             else:
530                 logger.info("detector cache already complete -> tracker-only re-run")
531
532         # ---- tracker re-run over the full ordered cache (cheap, deterministic) ---- #
533         tracker = build_tracker(cfg)

```

```

534     tracker.refresh()
535     rows = []
536     for fid in sorted(det_by_fid.keys()):
537         r = tracker.update(_dets_to_tracker(det_by_fid[fid]))
538         rows.append((fid, 1 if r.get("visi") else 0, r.get("x", -1), r.get("y", -1)))
539
540     _atomic_write_trajectory(out_csv, rows)
541     visible = sum(1 for _, v, *_ in rows if v)
542     logger.info(f"wrote {len(rows)} rows -> {out_csv}")
543     logger.info(f"visible-shuttle frames: {visible}/{len(rows)}")
544     return out_csv
545
546
547 def extract_frames(video_path: str, frames_dir: str) -> str:
548     """Decode a video to PNG frames named by source frame index (00000000.png ...).
549
550     Idempotent: if the dir already holds the expected number of frames (matching the
551     container's reported frame count) we skip re-decoding ? re-extraction after a
552     reboot is wasteful. NOTE: cv2 PNG extraction is slow; for big clips prefer
553     extracting with ffmpeg on the host and passing --frames_dir.
554     """
555     import cv2
556     os.makedirs(frames_dir, exist_ok=True)
557     cap = cv2.VideoCapture(video_path)
558     if not cap.isOpened():
559         raise SystemExit(f"cannot open video: {video_path}")
560     expected = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
561     existing = len(glob.glob(osp.join(frames_dir, "*.png")))
562     if expected > 0 and existing >= expected:
563         cap.release()
564         logger.info(f"reusing {existing} already-extracted frames -> {frames_dir}")
565         return frames_dir
566     i = 0
567     while True:
568         ok, frame = cap.read()
569         if not ok:
570             break
571         cv2.imwrite(osp.join(frames_dir, f"{i:08d}.png"), frame)
572         i += 1
573     cap.release()
574     logger.info(f"extracted {i} frames -> {frames_dir}")
575     return frames_dir
576
577
578 def _probe_frame_count(video_path: str) -> int:
579     """Container-reported frame count via cv2 (0 if unknown). Used to size the
580     stream."""
581     import cv2
582     cap = cv2.VideoCapture(video_path)
583     if not cap.isOpened():
584         raise SystemExit(f"cannot open video: {video_path}")
585     n = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
586     cap.release()
587     return n
588
589 def make_video_frame_loader(video_path: str):
590     """Return ``(frame_loader, count_hint, release)`` for output-preserving streaming.
591
592     ``frame_loader(index)`` -> BGR uint8 ndarray`` decodes the video forward-only with one
593     long-lived ``cv2.VideoCapture``. It MUST be called with non-decreasing indices (the
594     detector's sliding-window order); it reads sequentially and only uses a frame-seek
595     to
596     skip forward when a *resume* starts past the current position. Each ``cap.read()``
597     returns exactly the BGR uint8 array that ``cv2.imwrite``/``cv2.imread`` of a PNG

```

would

```
597     yield (PNG is lossless), so the detector sees identical pixels to the disk path.
598
599     Returns the count hint from the container so the caller can build the frame list;
600     ``release`` closes the capture.
601     """
602     import cv2
603     cap = cv2.VideoCapture(video_path)
604     if not cap.isOpened():
605         raise SystemExit(f"cannot open video: {video_path}")
606     count_hint = int(cap.get(cv2.CAP_PROP_FRAME_COUNT) or 0)
607     state = {"pos": 0} # next index cap.read() will return
608
609     def frame_loader(index: int):
610         index = int(index)
611         if index < state["pos"]:
612             raise RuntimeError(
613                 f"streaming decode is forward-only; got index {index} after
{state['pos']}")
614         if index > state["pos"]:
615             # Forward skip only happens when resuming past cached windows. Seek once.
616             cap.set(cv2.CAP_PROP_POS_FRAMES, index)
617             state["pos"] = index
618         ok, frame = cap.read()
619         if not ok:
620             raise RuntimeError(f"failed to decode frame {index} from {video_path}")
621         state["pos"] += 1
622         return frame
623
624     def release():
625         cap.release()
626
627     return frame_loader, count_hint, release
628
629
630     def run_video_streaming(video_path: str, weights: str, out_csv: str, sport: str =
"badminton",
631                             limit: int = 0, batch_size: int = 8,
632                             log_every_batches: int = 50, cache_dir: "str | None" = None,
633                             flush_every: int = 1000, fresh: bool = False) -> str:
634         """Fast path: decode ``video_path`` on the fly and feed frames straight to the
635         detector, WITHOUT extracting every frame to a PNG first (the disk round-trip that
636         dominates a long clip). Output is bit-identical to the frames-on-disk path; see
637         ``run``'s docstring for why.
638         """
639         frame_loader, count_hint, release = make_video_frame_loader(video_path)
640         try:
641             if count_hint <= 0:
642                 raise SystemExit(
643                     f"could not determine frame count for {video_path}; cannot stream "
644                     f"(fall back to frame extraction with --frames_dir)")
645             frame_list = [synthetic_frame_path(i) for i in range(count_hint)]
646             source_key = "video:" + osp.abspath(video_path)
647             return run(frame_list, weights, out_csv, sport, limit,
648                       batch_size=batch_size, num_workers=0,
649                       log_every_batches=log_every_batches, cache_dir=cache_dir,
650                       flush_every=flush_every, fresh=fresh,
651                       frame_loader=frame_loader, source_key=source_key)
652         finally:
653             release()
654
655
656     def main():
657         ap = argparse.ArgumentParser()
658         ap.add_argument("--frames_dir", help="folder of pre-extracted frames")
```

```

659         ap.add_argument("--video", help="video file; frames are extracted internally (in
WSL)")
660         ap.add_argument("--frames_out_dir", help="where to extract frames when --video is
used")
661         ap.add_argument("--stream-video", action="store_true",
662                         help="FAST PATH: decode --video on the fly and feed frames straight
to "
663                             "the detector (no PNG extraction). Output-equivalent to the
disk path.")
664         ap.add_argument("--weights", required=True)
665         ap.add_argument("--out", required=True)
666         ap.add_argument("--sport", default="badminton")
667         ap.add_argument("--limit", type=int, default=0, help="cap #frames (for quick
tests)")
668         ap.add_argument("--batch-size", type=int, default=8)
669         ap.add_argument("--num-workers", type=int, default=4)
670         ap.add_argument("--log-every-batches", type=int, default=50, help="progress log
cadence")
671         ap.add_argument("--cache-dir", default=None,
672                         help="resumable detector cache dir (default: <out>_wasbcache next to
--out)")
673         ap.add_argument("--flush-every", type=int, default=1000,
674                         help="fsync the detector cache every N windows (bounds reboot
loss)")
675         ap.add_argument("--fresh", action="store_true", help="ignore any existing cache and
start over")
676         args = ap.parse_args()
677
678         if args.stream_video:
679             if not args.video:
680                 raise SystemExit("--stream-video requires --video")
681             run_video_streaming(args.video, args.weights, args.out, args.sport, args.limit,
682                                batch_size=args.batch_size,
683                                log_every_batches=args.log_every_batches,
cache_dir=args.cache_dir,
684                                flush_every=args.flush_every, fresh=args.fresh)
685             return
686
687         frames_dir = args.frames_dir
688         if args.video:
689             frames_dir = extract_frames(args.video, args.frames_out_dir or (args.video +
"_frames"))
690         if not frames_dir:
691             raise SystemExit("provide --frames_dir or --video")
692         run(frames_dir, args.weights, args.out, args.sport, args.limit,
693            batch_size=args.batch_size, num_workers=args.num_workers,
694            log_every_batches=args.log_every_batches, cache_dir=args.cache_dir,
695            flush_every=args.flush_every, fresh=args.fresh)
696
697
698     if __name__ == "__main__":
699         main()
700

```

26. assistant (2026-06-18T01:34:03.854Z)

I notice something in `wasb\_infer.py`. At line 480, `det\_f = open(det\_jsonl, "a")` is opened, and the `try/finally` at lines 481-527 ensures it's closed. That's handled. Let me look closer at the `extract\_frames` function? `cap.release()` happens but if `cv2.imwrite` raises, the cap isn't released. That's a minor leak in a WSL-only script. Let me note it but keep hunting for stronger findings.

Let me look at the validators (pts_continuity, format, resolution_fps) and the gemini segmenter.

27. user (2026-06-18T01:34:04.790Z)

```

1      import subprocess
2      from backend.pipeline.validators.base import VideoValidator, ValidationResult
3      from typing import Dict, Any
4
5      class PTSContinuityValidator(VideoValidator):
6          @property
7          def name(self) -> str:
8              return "pts_continuity_check"
9
10         @property
11         def description(self) -> str:
12             return "Analyzes Presentation Time Stamps (PTS) of keyframes to check for
13 timeline gaps, jumps, or editing splices."
14
15         def validate(self, video_path: str, config: Dict[str, Any], context: Dict[str, Any])
16 -> ValidationResult:
17             # Command to extract packet PTS and flags (extremely fast, works on H.264 &
18 H.265)
19
20             cmd = [
21                 "ffprobe",
22                 "-v", "error",
23                 "-select_streams", "v:0",
24                 "-show_entries", "packet=pts_time,flags",
25                 "-of", "csv=p=0",
26                 video_path
27             ]
28
29             try:
30                 result = subprocess.run(cmd, capture_output=True, text=True, check=True)
31                 output = result.stdout.strip()
32
33                 if not output:
34                     return ValidationResult(
35                         validator_name=self.name,
36                         passed=False,
37                         message="No packet timestamps could be extracted from the video
38 stream."
39                     )
40
41                 timestamps = []
42                 for line in output.split("\n"):
43                     line = line.strip()
44                     if line:
45                         parts = line.split(",")
46                         if len(parts) == 2:
47                             pts_str, flags = parts
48                             if "K" in flags: # Check if keyframe
49                                 try:
50                                     timestamps.append(float(pts_str))
51                                 except ValueError:
52                                     continue
53
54                 if len(timestamps) < 2:
55                     return ValidationResult(
56                         validator_name=self.name,
57                         passed=True,
58                         message="Video is too short to perform a PTS continuity check.",
59                         details={"keyframes_count": len(timestamps)}
60                     )
61
62                 # Analyze gaps
63                 max_gap = 0.0
64                 backward_jumps = 0
65                 gaps = []

```

```

62         for i in range(1, len(timestamps)):
63             gap = timestamps[i] - timestamps[i-1]
64             gaps.append(gap)
65             if gap < 0:
66                 backward_jumps += 1
67             if gap > max_gap:
68                 max_gap = gap
69
70         # An continuous GoPro recording usually has keyframe intervals (GOP size) of
71         1.0 to 4.0 seconds.
72         # If there's a keyframe gap larger than 8.0 seconds or a negative gap
73         (backward jump), it's highly likely spliced.
74         details = {
75             "keyframes_count": len(timestamps),
76             "max_keyframe_gap_seconds": round(max_gap, 2),
77             "backward_jumps_detected": backward_jumps,
78             "first_keyframe": timestamps[0],
79             "last_keyframe": timestamps[-1]
80         }
81
82         if backward_jumps > 0:
83             return ValidationResult(
84                 validator_name=self.name,
85                 passed=False,
86                 message="Video integrity compromise: Backward timeline jumps
87                 (splicing) detected.",
88                 details=details
89             )
90
91         # Threshold for sudden gap (e.g. 10.0 seconds)
92         if max_gap > 10.0:
93             return ValidationResult(
94                 validator_name=self.name,
95                 passed=False,
96                 message=f"Video integrity compromise: Detected a timestamp gap of
97                 {round(max_gap, 2)}s, indicating edited or missing content.",
98                 details=details
99             )
100
101         return ValidationResult(
102             validator_name=self.name,
103             passed=True,
104             message="PTS timeline is continuous with no abnormal gaps or backward
105             jumps.",
106             details=details
107         )
108
109     except subprocess.CalledProcessError as e:
110         return ValidationResult(
111             validator_name=self.name,
112             passed=False,
113             message="Failed to execute ffprobe PTS analysis.",
114             details={"error": e.stderr}
115         )
116
117     except Exception as e:
118         return ValidationResult(
119             validator_name=self.name,
120             passed=False,
121             message=f"Unexpected error in PTS validation: {str(e)}"
122         )

```

28. user (2026-06-18T01:34:05.446Z)

```
1 import subprocess
```

```

2     import json
3     import os
4     from backend.pipeline.validators.base import VideoValidator, ValidationResult
5     from typing import Dict, Any
6
7     class FormatValidator(VideoValidator):
8         @property
9         def name(self) -> str:
10             return "format_check"
11
12         @property
13         def description(self) -> str:
14             return "Verifies the video is a valid, uncorrupted MP4 container and extracts
metadata."
15
16         def validate(self, video_path: str, config: Dict[str, Any], context: Dict[str, Any])
-> ValidationResult:
17             if not os.path.exists(video_path):
18                 return ValidationResult(
19                     validator_name=self.name,
20                     passed=False,
21                     message=f"File not found: {video_path}"
22                 )
23
24             cmd = [
25                 "ffprobe",
26                 "-v", "error",
27                 "-select_streams", "v:0",
28                 "-show_format",
29                 "-show_streams",
30                 "-of", "json",
31                 video_path
32             ]
33
34             try:
35                 result = subprocess.run(cmd, capture_output=True, text=True, check=True)
36                 info = json.loads(result.stdout)
37
38                 if not info.get("streams") or not info.get("format"):
39                     return ValidationResult(
40                         validator_name=self.name,
41                         passed=False,
42                         message="Failed to retrieve valid video streams from the file."
43                     )
44
45                 # Store the metadata in context for subsequent validators to utilize
46                 context["ffprobe_meta"] = info
47
48                 stream = info["streams"][0]
49                 fmt = info["format"]
50
51                 details = {
52                     "format_name": fmt.get("format_name"),
53                     "duration": float(fmt.get("duration", 0)),
54                     "size_bytes": int(fmt.get("size", 0)),
55                     "codec_name": stream.get("codec_name"),
56                 }
57
58                 # Check if container name includes mp4
59                 if "mp4" not in fmt.get("format_name", "").lower():
60                     return ValidationResult(
61                         validator_name=self.name,
62                         passed=False,
63                         message=f"Invalid format: Found '{fmt.get('format_name')}', but MP4
is required.",

```



```

64         details=details
65     )
66
67     return ValidationResult(
68         validator_name=self.name,
69         passed=True,
70         message="Video format is valid MP4 container.",
71         details=details
72     )
73
74     except subprocess.CalledProcessError as e:
75         return ValidationResult(
76             validator_name=self.name,
77             passed=False,
78             message="Video stream is corrupted or unreadable by ffprobe.",
79             details={"error": e.stderr}
80         )
81     except Exception as e:
82         return ValidationResult(
83             validator_name=self.name,
84             passed=False,
85             message=f"Unexpected error running ffprobe: {str(e)}"
86         )
87

```

29. assistant (2026-06-18T01:34:08.184Z)

These are clean. Let me look at the main.py, run_telemetry.py, and the gemini segmenter (which extracts chunks and uploads). Also the yolo_hybrid segmenter.

30. user (2026-06-18T01:34:09.131Z)

```

1     """Structured run telemetry ? a compact, rotating JSONL "black box" for long/flaky runs.
2
3     A long GPU/CV run (the fusion-golden extraction, a WSL/WASB pass, an ffmpeg stitch) can
4     die with
5     **no Python traceback** ? the OS kills it (a console-control event, an OOM, a thermal
6     event). When
7     that happens the only forensic evidence is whatever was last flushed to disk. This
8     module keeps a
9     tiny (~10 KB) rotating runlog of *execution velocity + machine health* so a post-mortem
10    is a
11    bullseye instead of a guess: "it was at 26 fps, GPU 85 °C throttled to 1125/2100 MHz,
12    4.8 GB VRAM
13    free, CPU 60 %, RAM 21/32 GB ? then vanished."
14
15    It is the **observability** companion to the `ExecutionPolicy` *control* layer
16    (`docs/EXECUTION_POLICY.md`): the policy decides wait / abort / resume; this records the
17    signals
18    that explain *why* a run slowed or died, and lets us compare execution velocity across
19    quality
20    iterations.
21
22    Design:
23    - **Caller-driven snapshots** (no background thread): call ``snapshot(done, total)``
24    from an
25    existing progress callback; fps is derived from the delta since the previous snapshot.
26    ``event(kind, msg)`` records discrete moments (start, video-done, error) WITH a health
27    sample, so
28    the last line before a kill captures the machine state at death.
29    - **Capped + rotating** : a fixed ``meta`` header line (machine / GPU / start) is always
30    kept; recent
31    snapshot/event lines form an on-disk ring trimmed to ``max_bytes`` (~10 KB). The file
32    is rewritten
33    atomically (tmp + ``os.replace``) on each write, so a hard kill never leaves it half-

```

```

written.
23     - **Graceful degradation**: GPU stats come from ``nvidia-smi`` (absent -> ``None``);
system stats
24     from ``psutil`` (not installed -> ``None``). Collection never raises into the caller ?
a telemetry
25     hiccup must not break the run it observes.
26     - **Deterministic / testable**: the clock and the two collectors are injectable, so fps,
rotation,
27     and parsing are unit-tested with no real GPU, no psutil, and no wall-clock.
28
29     All stdlib + optional ``psutil`` (BSD-3, commercial-clean). No network, no training-data
path.
30     """
31     from __future__ import annotations
32
33     import json
34     import os
35     import shutil
36     import subprocess
37     import time
38     from typing import Any, Callable, Dict, List, Optional
39
40     GpuFn = Callable[[], Optional[Dict[str, Any]]]
41     SysFn = Callable[[], Optional[Dict[str, Any]]]
42
43     # nvidia-smi fields pulled in one query, in this order (see gpu_stats parsing).
44     _GPU_QUERY =
"temperature.gpu,utilization.gpu,power.draw,clocks.sm,clocks.max.sm,memory.used,memory.free"
45     _THROTTLE_CLOCK_FRAC = 0.8 # SM clock below this fraction of max => flagged throttled
46     _THROTTLE_TEMP_C = 84.0 # ... or temperature at/above this (laptop Max-Q throttle
territory)
47
48
49     def _num(s: str) -> Optional[float]:
50         try:
51             return float(s)
52         except (TypeError, ValueError):
53             return None
54
55
56     def gpu_stats(timeout: float = 4.0) -> Optional[Dict[str, Any]]:
57         """One ``nvidia-smi`` sample as a dict, or ``None`` if nvidia-smi is unavailable /
errors.
58
59         Returns util %, temperature, power (W), current + max SM clock (MHz), VRAM used /
free (MB), and a
60         derived ``throttled`` flag (SM clock well below max, or hot). Pure read; never
raises.
61         """
62         exe = shutil.which("nvidia-smi")
63         if not exe:
64             return None
65         try:
66             out = subprocess.run(
67                 [exe, f"--query-gpu={_GPU_QUERY}", "--format=csv,noheader,nounits"],
68                 capture_output=True, text=True, timeout=timeout, check=True,
69                 ).stdout.strip()
70         except (subprocess.SubprocessError, OSError):
71             return None
72         row = out.splitlines()[0] if out else ""
73         parts = [p.strip() for p in row.split(",")]
74         if len(parts) < 7:
75             return None
76         vals = [_num(p) for p in parts[:7]]
77         temp, util, power, sm, sm_max, used, free = vals

```

```

78     throttled = bool(temp is not None and temp >= _THROTTLE_TEMP_C)
79     if sm is not None and sm_max:
80         throttled = throttled or sm < _THROTTLE_CLOCK_FRAC * sm_max
81     return {
82         "util_pct": util, "temp_c": temp, "power_w": power,
83         "sm_mhz": sm, "sm_max_mhz": sm_max,
84         "vram_used_mb": used, "vram_free_mb": free,
85         "throttled": throttled,
86     }
87
88
89     def system_stats() -> Optional[Dict[str, Any]]:
90         """CPU % + RAM via ``psutil``, or ``None`` if psutil isn't installed. Never
91         raises."""
92         try:
93             import psutil # optional dep (BSD-3); only used when present
94         except ImportError:
95             return None
96         try:
97             vm = psutil.virtual_memory()
98             return {
99                 "cpu_pct": psutil.cpu_percent(interval=None),
100                 "ram_used_gb": round(vm.used / 1e9, 2),
101                 "ram_total_gb": round(vm.total / 1e9, 2),
102                 "ram_pct": vm.percent,
103             }
104         except Exception: # noqa: BLE001 - psutil edge cases must not break the run
105             return None
106
107     def gpu_name() -> Optional[str]:
108         """The GPU model string (for the runlog header), or ``None`` without nvidia-smi."""
109         exe = shutil.which("nvidia-smi")
110         if not exe:
111             return None
112         try:
113             out = subprocess.run([exe, "--query-gpu=name", "--format=csv,noheader"],
114                                 capture_output=True, text=True, timeout=4.0,
115                                 check=True).stdout
116         except (subprocess.SubprocessError, OSError):
117             return None
118         return out.splitlines()[0].strip() if out else None
119
120     class RunTelemetry:
121         """A small rotating JSONL runlog of execution velocity + machine health.
122
123         Call ``snapshot(done, total, phase=...)`` from a progress callback and ``event(kind,
124         msg)`` at
125         milestones. The file holds a ``meta`` header line plus the most recent snapshot /
126         event lines
127         that fit within ``max_bytes`` (~10 KB), rewritten atomically on every write.
128         """
129
130     def __init__(self, path: str, label: str = "run", max_bytes: int = 10_000,
131                 gpu_fn: GpuFn = gpu_stats, sys_fn: SysFn = system_stats,
132                 clock: Callable[[], float] = time.time) -> None:
133         self.path = path
134         self.label = label
135         self.max_bytes = max_bytes
136         self._gpu_fn = gpu_fn
137         self._sys_fn = sys_fn
138         self._clock = clock
139         self._t0 = clock()
140         self._last_done: Optional[int] = None

```

```

139         self._last_t: Optional[float] = None
140         self._lines: List[str] = [] # recent body lines (on-disk ring, trimmed to
max_bytes)
141         self._meta = json.dumps({"meta": {
142             "label": label, "started": self._iso(self._t0), "pid": os.getpid(),
143             "gpu_name": gpu_name(), "cpu_count": os.cpu_count(),
144         }}, separators=(",", ":"))
145         self._write()
146
147     # -- public API -----
148     def snapshot(self, done: Optional[int] = None, total: Optional[int] = None,
149                 phase: Optional[str] = None, **extra: Any) -> Dict[str, Any]:
150         """Record a velocity + health sample; ``fps`` is derived from the previous
snapshot."""
151         now = self._clock()
152         rec: Dict[str, Any] = {"t": self._iso(now), "elapsed_s": round(now - self._t0,
1)}
153         if phase is not None:
154             rec["phase"] = phase
155         if done is not None:
156             rec["done"] = done
157             if total:
158                 rec["total"] = total
159                 rec["pct"] = round(100.0 * done / total, 1)
160             if self._last_done is not None and self._last_t is not None and now >
self._last_t:
161                 rec["fps"] = round((done - self._last_done) / (now - self._last_t), 1)
162                 self._last_done, self._last_t = done, now
163         rec.update(extra)
164         self._attach_health(rec)
165         self._append(rec)
166         return rec
167
168     def event(self, kind: str, msg: str = "", **extra: Any) -> Dict[str, Any]:
169         """Record a milestone (start, video_done, error, restart) WITH a health
sample."""
170         now = self._clock()
171         rec: Dict[str, Any] = {"t": self._iso(now), "elapsed_s": round(now - self._t0,
1)},
172                                "event": kind}
173         if msg:
174             rec["msg"] = msg
175         rec.update(extra)
176         self._attach_health(rec)
177         self._append(rec)
178         return rec
179
180     # -- internals -----
181     def _attach_health(self, rec: Dict[str, Any]) -> None:
182         gpu = self._safe(self._gpu_fn)
183         sys_ = self._safe(self._sys_fn)
184         if gpu is not None:
185             rec["gpu"] = gpu
186         if sys_ is not None:
187             rec["sys"] = sys_
188
189     def _append(self, rec: Dict[str, Any]) -> None:
190         self._lines.append(json.dumps(rec, separators=(",", ":")))
191         budget = self.max_bytes - len(self._meta) - 1
192         while len(self._lines) > 1 and sum(len(s) + 1 for s in self._lines) > budget:
193             self._lines.pop(0) # drop oldest body line; the meta header is always kept
194         self._write()
195
196     def _write(self) -> None:
197         try:

```

```

198         tmp = self.path + ".tmp"
199         with open(tmp, "w", encoding="utf-8") as f:
200             f.write(self._meta + "\n")
201             for s in self._lines:
202                 f.write(s + "\n")
203         os.replace(tmp, self.path)
204     except OSError:
205         pass    # telemetry must never break the run it observes
206
207     @staticmethod
208     def _safe(fn: GpuFn) -> Optional[Dict[str, Any]]:
209         try:
210             return fn()
211         except Exception:    # noqa: BLE001 - a collector hiccup must not break the run
212             return None
213
214     @staticmethod
215     def _iso(t: float) -> str:
216         return time.strftime("%Y-%m-%dT%H:%M:%S", time.localtime(t))
217

```

31. assistant (2026-06-18T01:34:11.408Z)

Clean. Let me look at the gemini segmenter and main.py, then yolo_hybrid.

32. user (2026-06-18T01:34:12.335Z)

```

1     import os
2     import time
3     import logging
4     import subprocess
5     import concurrent.futures
6     from typing import List, Dict, Any, Tuple
7     from backend.pipeline.segmenters.base import BasePlaySegmenter, segmenter_registry
8     from backend.utils.video import get_video_duration, extract_video_chunk
9     from backend.sports import sport_registry
10    from backend.providers import provider_registry
11    from backend.reporting import run_signals
12
13    logger = logging.getLogger(__name__)
14
15
16    def _chunk_workdir(video_id: str) -> str:
17        """Per-video chunk staging dir under output/chunks/<video_id>.
18
19        Concurrency isolation: two runs for DIFFERENT videos must not share one chunk dir ?
20        otherwise one run finishing (and rmdir-ing the shared dir) can undo the other's
21        in-flight work. A per-video subdir means each run only ever creates/cleans its own.
22        """
23        return os.path.join("output", "chunks", video_id)
24
25
26    class GeminiPlaySegmenter(BasePlaySegmenter):
27        @property
28        def name(self) -> str:
29            return "gemini"
30
31        @property
32        def description(self) -> str:
33            return "High-precision AI-powered semantic play segment detection using Google Gemini Multimodal API."
34
35        def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
36            None, max_frames: int = None) -> List[Dict[str, Any]]:
37            """

```

```

37         Processes a video to detect play segments using Google Gemini Multimodal Video
API,
38         populates SQLite, and indexes detailed events.
39         """
40         # Get sport profile config
41         sport_name = self.config.get("sport", "badminton")
42         try:
43             sport_profile = sport_registry.get(sport_name)
44         except KeyError as e:
45             logger.error(str(e))
46             return []
47
48         # Get AI provider and model config
49         idx_cfg = self.config.get("indexing", {})
50         provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
"gemini"))
51         model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
"gemini-2.5-flash"))
52
53         # Get appropriate API key based on the provider
54         api_key_env_var = f"{provider_name.upper()}_API_KEY"
55         api_key = os.environ.get(api_key_env_var)
56         if not api_key:
57             logger.error(
58                 f"{api_key_env_var} environment variable is not set! "
59                 f"To run the {provider_name} segmenter, set your API key. "
60                 f"In PowerShell: $env:{api_key_env_var}=\"your_api_key_here\""
61             )
62             return []
63
64         try:
65             provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)
66         except KeyError as e:
67             logger.error(str(e))
68             return []
69
70         # A: Check for debug_duration to trim the video first
71         debug_duration = self.config.get("indexing", {}).get("debug_duration")
72         video_to_upload = video_path
73         is_trimmed = False
74         temp_trimmed_path = None
75
76         if debug_duration:
77             logger.info(f"DEBUG FEATURE: Trimming video to first {debug_duration}
seconds for fast processing...")
78             temp_trimmed_path = os.path.join("output", f"temp_trimmed_{video_id}.mp4")
79             os.makedirs("output", exist_ok=True)
80             if os.path.exists(temp_trimmed_path):
81                 try:
82                     os.remove(temp_trimmed_path)
83                 except Exception:
84                     pass
85
86             cmd = [
87                 "ffmpeg", "-y",
88                 "-ss", "0",
89                 "-t", str(debug_duration),
90                 "-i", video_path,
91                 "-c", "copy",
92                 temp_trimmed_path
93             ]
94             logger.debug(f"Running command: {' '.join(cmd)}")
95             try:
96                 subprocess.run(cmd, check=True, stdout=subprocess.DEVNULL,

```

```

stderr=subprocess.DEVNULL)
97         logger.info(f"Trimmed video successfully created at:
{temp_trimmed_path}")
98         video_to_upload = temp_trimmed_path
99         is_trimmed = True
100     except Exception as e:
101         logger.warning(f"Failed to trim video using ffmpeg: {e}. Falling back to
full video.")
102
103
104
105     # Helper to clean up local temp files
106     def cleanup_temp_files():
107         if is_trimmed and temp_trimmed_path and os.path.exists(temp_trimmed_path):
108             try:
109                 os.remove(temp_trimmed_path)
110             except Exception:
111                 pass
112
113     # 1. Determine actual video duration
114     video_duration = get_video_duration(video_to_upload)
115     if video_duration > 0:
116         logger.info(f"Detected video duration: {video_duration:.1f}s")
117     else:
118         logger.warning("Could not detect video duration. Prompt will not include
duration context.")
119
120     # 2. Decide whether to use chunked processing
121     idx_cfg = self.config.get("indexing", {})
122     chunk_duration = idx_cfg.get("chunk_duration", 300)    # 5 minutes default
123     chunk_overlap = idx_cfg.get("chunk_overlap", 30)      # 30 seconds overlap
124     use_chunking = video_duration > 0 and video_duration > chunk_duration
125     # Re-encode + downscale chunks for upload (0 = legacy stream-copy). Stream-
copied
126     # 4K chunks (~1.3GB/90s) exceed the provider's MAX_UPLOAD_MB and are refused, so
127     # every chunk of a 4K source gets skipped; Gemini analyzes at low res anyway.
128     ai_scale_width = int(idx_cfg.get("ai_chunk_scale_width", 1280) or 0)
129
130     ai_segments = []
131     # Per-video chunk workdir (concurrency isolation): a concurrent run for another
132     # video keeps its own dir, so cleanup here never removes another run's chunks.
133     chunks_dir = _chunk_workdir(video_id)
134
135     if use_chunking:
136         # --- CHUNKED PROCESSING ---
137         step = chunk_duration - chunk_overlap
138         chunk_starts = []
139         t = 0.0
140         while t < video_duration:
141             chunk_starts.append(t)
142             t += step
143         num_chunks = len(chunk_starts)
144         logger.info(
145             f"Video is {video_duration:.1f}s ? splitting into {num_chunks}
overlapping chunks "
146             f"({chunk_duration}s each, {chunk_overlap}s overlap).")
147         )
148
149         os.makedirs(chunks_dir, exist_ok=True)
150
151         def process_single_chunk(ci: int, chunk_start: float) -> Tuple[str, list,
dict]:
152             chunk_end = min(chunk_start + chunk_duration, video_duration)
153             actual_chunk_dur = chunk_end - chunk_start
154             chunk_label = f"Chunk {ci+1}/{num_chunks}"

```

```

155         chunk_path = os.path.join(chunks_dir, f"chunk_{video_id}_{ci}.mp4")
156
157         # Extract chunk with ffmpeg
158         logger.info(f"Extracting {chunk_label}: {chunk_start:.1f}s -
{chunk_end:.1f}s ({actual_chunk_dur:.1f}s)")
159         success = extract_video_chunk(
160             video_to_upload, chunk_start, actual_chunk_dur, chunk_path,
161             reencode=ai_scale_width > 0,
162             scale_width=ai_scale_width if ai_scale_width > 0 else None,
163         )
164         if not success:
165             logger.error(f"Failed to extract {chunk_label}. Skipping.")
166             return chunk_label, [], {}
167
168         # Measure chunk size
169         try:
170             chunk_size_mb = os.path.getsize(chunk_path) / (1024 * 1024)
171         except Exception:
172             chunk_size_mb = 0.0
173
174         # Get prompt from sport profile
175         prompt = sport_profile.get_prompt(chunk_duration=actual_chunk_dur)
176
177         # Process this chunk through the provider
178         chunk_segments, metrics = provider.analyze_video(
179             chunk_path, prompt, chunk_duration=actual_chunk_dur,
label=chunk_label
180         )
181         metrics["file_size_mb"] = chunk_size_mb
182         metrics["segments_found"] = len(chunk_segments)
183
184         # Offset timestamps back to full-video coordinates
185         for segment in chunk_segments:
186             if "start_time" in segment:
187                 try:
188                     segment["start_time"] = float(segment["start_time"]) +
chunk_start
189                 except (ValueError, TypeError):
190                     pass
191             if "end_time" in segment:
192                 try:
193                     segment["end_time"] = float(segment["end_time"]) +
chunk_start
194                 except (ValueError, TypeError):
195                     pass
196
197         logger.info(f"{chunk_label} returned {len(chunk_segments)} play
segments.")
198
199         # Clean up chunk file
200         try:
201             os.remove(chunk_path)
202         except Exception:
203             pass
204
205         return chunk_label, chunk_segments, metrics
206
207     # Run chunks in parallel. Workers come from indexing.ai_max_workers (same
knob
208     # the hybrids honor; was hardcoded 5). ai_max_workers=1 = serial ? needed to
209     # gently ramp a cold/prepaid Gemini project, whose burst throttling returns
a
210     # misleading "credits depleted" 429 under 5 simultaneous uploads.
211     all_metrics = {}
212     max_workers = max(1, int(self.config.get("indexing",

```



```

{}).get("ai_max_workers", 5)))
213         logger.info("Starting parallel processing with up to %d concurrent
workers...", max_workers)
214         t_chunking_start = time.time()
215         with concurrent.futures.ThreadPoolExecutor(max_workers=max_workers) as
executor:
216             futures = [executor.submit(process_single_chunk, ci, start) for ci,
start in enumerate(chunk_starts)]
217             for future in concurrent.futures.as_completed(futures):
218                 label, segments, metrics = future.result()
219                 ai_segments.extend(segments)
220                 if metrics:
221                     all_metrics[label] = metrics
222
223             # Oversize refusals must be LOUD: each one is a time range with NO analysis,
224             # so "0 segments / success" here would be a silent failure, not "no
rallies".
225             oversize_skips = sum(1 for m in all_metrics.values() if
m.get("oversize_skipped"))
226             if oversize_skips:
227                 logger.error(
228                     f"{oversize_skips}/{num_chunks} chunks exceeded the provider upload
cap and were "
229                     f"SKIPPED WITHOUT ANALYSIS ? segments are missing for those time
ranges. "
230                     f"Set indexing.ai_chunk_scale_width (default 1280) so chunks are re-
encoded/"
231                     f"downscaled before upload."
232                 )
233             run_signals.incr(video_id, "oversize_clips_skipped", oversize_skips)
234
235             # Print Summary Log
236             summary_lines = [
237                 "",
238                 "=" * 60,
239                 "                GEMINI CHUNKING SUMMARY",
240                 "=" * 60,
241                 f" Total time: {time.time() - t_chunking_start:.1f}s",
242                 f" Total chunks: {num_chunks}",
243                 f" Total play segments found: {len(ai_segments)}",
244                 "-" * 60,
245                 f" {'Chunk':<12} | {'Size':<8} | {'Upload':<8} | {'Process':<8} |
{'Gen.':<8} | {'Segments'}",
246                 "-" * 60,
247             ]
248             for i in range(num_chunks):
249                 lbl = f"Chunk {i+1}/{num_chunks}"
250                 m = all_metrics.get(lbl, {})
251                 sz = f"{m.get('file_size_mb', 0):.1f}MB" if m else "N/A"
252                 upl = f"{m.get('upload_time', 0):.1f}s" if m else "N/A"
253                 prc = f"{m.get('process_time', 0):.1f}s" if m else "N/A"
254                 gen = f"{m.get('gen_time', 0):.1f}s" if m else "N/A"
255                 seg = m.get('segments_found', 0) if m else 0
256                 summary_lines.append(f" {lbl:<12} | {sz:<8} | {upl:<8} | {prc:<8} |
{gen:<8} | {seg}")
257             summary_lines.append("=" * 60)
258             logger.info("\n".join(summary_lines))
259
260             # Clean up local temp files
261             cleanup_temp_files()
262             try:
263                 os.rmdir(chunks_dir)
264             except Exception:
265                 pass
266

```

```

267         logger.info(f"All chunks processed. {len(ai_segments)} total play segments
before deduplication.")
268
269     else:
270         # --- SINGLE-FILE PROCESSING (short videos) ---
271         try:
272             file_sz = os.path.getsize(video_to_upload) / (1024 * 1024)
273         except Exception:
274             file_sz = 0.0
275
276         # Get prompt from sport profile
277         prompt = sport_profile.get_prompt(chunk_duration=video_duration)
278         ai_segments, metrics = provider.analyze_video(
279             video_to_upload, prompt, chunk_duration=video_duration,
label="SingleFile"
280         )
281         if metrics.get("oversize_skipped"):
282             logger.error(
283                 f"The video ({file_sz:.0f}MB) exceeded the provider upload cap and
was "
284                 f"SKIPPED WITHOUT ANALYSIS ? 0 segments here does NOT mean 'no
rallies'."
285             )
286             run_signals.incr(video_id, "oversize_clips_skipped", 1)
287             summary_lines = [
288                 "",
289                 "=" * 50,
290                 "                GEMINI SINGLE FILE SUMMARY",
291                 "=" * 50,
292                 f" File Size: {file_sz:.1f} MB",
293                 f" Upload time: {metrics.get('upload_time', 0):.1f}s",
294                 f" API Processing time: {metrics.get('process_time', 0):.1f}s",
295                 f" Prompt Generation time: {metrics.get('gen_time', 0):.1f}s",
296                 f" Play segments found: {len(ai_segments)}",
297                 "=" * 50,
298             ]
299             logger.info("\n".join(summary_lines))
300             cleanup_temp_files()
301
302         if not ai_segments:
303             return []
304
305         # Populate SQLite DB with parsed play segments
306         segments_data = []
307         logger.info(f"Processing {len(ai_segments)} play segments through
validation...")
308
309         def parse_time(val) -> float:
310             """Converts a timestamp value to total float seconds.
311             Handles float/int pass-through, plain numeric strings, and
312             colon-separated formats (MM:SS, HH:MM:SS). Logs warnings
313             when unexpected formats are encountered."""
314             if isinstance(val, (int, float)):
315                 return float(val)
316             if isinstance(val, str):
317                 val = val.strip()
318                 if ":" in val:
319                     # Gemini returned MM:SS or HH:MM:SS despite being asked for decimal
seconds.
320                     # Convert it, but warn so the issue is visible in logs.
321                     parts = val.split(":")
322                     parts.reverse()
323                     total = 0.0
324                     for i, p in enumerate(parts):
325                         try:

```

```

326             total += float(p) * (60 ** i)
327         except ValueError:
328             pass
329         logger.warning(f"Timestamp '{val}' was in colon-separated format
(MM:SS/HH:MM:SS) instead of decimal seconds. Converted to {total:.2f}s.")
330         return total
331     try:
332         return float(val)
333     except ValueError:
334         logger.warning(f"Could not parse timestamp string '{val}' as a
number. Defaulting to 0.0.")
335         return 0.0
336     logger.warning(f"Unexpected timestamp type {type(val).__name__} for value
'{val}'. Defaulting to 0.0.")
337     return 0.0
338
339     # 7. Parse timestamps and build candidate segment list
340     idx_cfg = self.config.get("indexing", {})
341     min_segment_duration = idx_cfg.get("min_segment_duration", 3.0)
342     max_segment_duration = 90.0 # Singles play segments almost never exceed this
343
344     candidates = []
345     for idx, segment in enumerate(ai_segments):
346         start = parse_time(segment.get("start_time", 0.0))
347         end = parse_time(segment.get("end_time", 0.0))
348         if start >= end:
349             logger.info(f"Skipping segment #{idx+1}: start_time ({start:.2f}s) >=
end_time ({end:.2f}s)")
350             continue
351         candidates.append({
352             "idx": idx,
353             "start": start,
354             "end": end,
355             "raw": segment
356         })
357
358     # 8. Post-hoc validation pass
359     logger.info(f"Running validation on {len(candidates)} candidate play
segments...")
360     validated = []
361     rejected = []
362
363     for c in candidates:
364         label = f"Segment #{c['idx']+1} [{c['start']:.2f}s - {c['end']:.2f}s]"
365
366         # A: Reject segments that start beyond the video
367         if video_duration > 0 and c["start"] >= video_duration:
368             rejected.append((label, f"start_time ({c['start']:.2f}s) is beyond video
duration ({video_duration:.1f}s)"))
369             continue
370
371         # B: Clamp end_time to video duration
372         if video_duration > 0 and c["end"] > video_duration:
373             logger.info(f"{label}: end_time ({c['end']:.2f}s) exceeds video duration
({video_duration:.1f}s). Clamping.")
374             c["end"] = video_duration
375
376         dur = c["end"] - c["start"]
377
378         # C: Reject segments shorter than min_segment_duration
379         if dur < min_segment_duration:
380             rejected.append((label, f"duration ({dur:.2f}s) is below minimum
({min_segment_duration}s)"))
381             continue
382

```

```

383         # D: Flag suspiciously long segments
384         if dur > max_segment_duration:
385             logger.warning(
386                 f"{label}: duration ({dur:.1f}s) is unusually long
387 (>{max_segment_duration}s). "
388             )
389             f"This may be a detection error. Keeping but flagging."
390         validated.append(c)
391
392     # E: Resolve overlapping segments (keep the longer one)
393     if len(validated) > 1:
394         # Sort by start time
395         validated.sort(key=lambda r: r["start"])
396         non_overlapping = [validated[0]]
397         for curr in validated[1:]:
398             prev = non_overlapping[-1]
399             # Check if current segment overlaps with the previous one
400             if curr["start"] < prev["end"]:
401                 prev_dur = prev["end"] - prev["start"]
402                 curr_dur = curr["end"] - curr["start"]
403                 kept, dropped = (prev, curr) if prev_dur >= curr_dur else (curr,
404 prev)
405                 kept_label = f"Segment #{kept['idx']+1} [{kept['start']:.2f}s -
406 {kept['end']:.2f}s]"
407                 dropped_label = f"Segment #{dropped['idx']+1}
408 [{dropped['start']:.2f}s - {dropped['end']:.2f}s]"
409                 rejected.append((dropped_label, f"overlaps with {kept_label} ? kept
410 the longer segment"))
411                 if kept == curr:
412                     non_overlapping[-1] = curr # Replace previous with current
413                 else:
414                     non_overlapping.append(curr)
415                 validated = non_overlapping
416
417     # Log validation summary
418     if rejected:
419         reject_lines = [f" ? {label}: {reason}" for label, reason in rejected]
420         logger.info(f"{len(rejected)} play segments were REJECTED:\n" +
421 "\n".join(reject_lines))
422         logger.info(f"{len(validated)} play segments passed validation (out of
423 {len(ai_segments)} from Gemini).")
424
425     stitch_cfg = self.config.get("stitching", {})
426     begin_pad = stitch_cfg.get("begin_padding", 0.5)
427     end_pad = stitch_cfg.get("end_padding", 0.5)
428
429     total_highlight_dur = 0.0
430     for c in validated:
431         total_highlight_dur += (c["end"] - c["start"]) + begin_pad + end_pad
432
433     mins = int(total_highlight_dur // 60)
434     secs = int(total_highlight_dur % 60)
435     logger.info(
436         f"Estimated final highlight video duration: {mins:02d}:{secs:02d} "
437         f"({total_highlight_dur:.1f}s total; includes {begin_pad}s begin +
438 {end_pad}s end padding per segment)"
439     )
440
441     # 9. Populate SQLite DB with validated play segments
442     for c in validated:
443         start = c["start"]
444         end = c["end"]
445         segment = c["raw"]
446         dur = end - start

```

```

440
441         # Extract shot_count from Gemini or compute estimate if not present
442         shot_count = segment.get("shuttle_exchange_count")
443         if shot_count is None:
444             shot_count = max(3, int(dur / 0.8)) # 1 shot every 0.8 seconds as
fallback estimate
445         else:
446             try:
447                 shot_count = int(shot_count)
448             except Exception:
449                 shot_count = max(3, int(dur / 0.8))
450
451         # Only store metrics we actually know ? no fabricated data
452         metadata = {
453             "shot_count": shot_count,
454             "shot_count_confidence": segment.get("shot_count_confidence",
"Unknown"),
455             "detector": model_name
456         }
457
458         segment_id = self.db.add_play_segment(video_id, start, end, None, None,
metadata)
459
460         if segment_id:
461             segments_data.append({
462                 "id": segment_id,
463                 "start_time": start,
464                 "end_time": end,
465                 "duration": dur,
466                 "metadata": metadata
467             })
468
469         return segments_data
470
471     # Register the segmenter
472     segmenter_registry.register(GeminiPlaySegmenter)
473

```

33. assistant (2026-06-18T01:34:16.027Z)

This is reasonable. Let me look at main.py and the yolo_hybrid segmenter, then the remaining files: cleanup_caches.py, doctor.py, harvest.py, resolution_fps.py.

34. user (2026-06-18T01:34:17.417Z)

```

1     import os
2     import sys
3     import argparse
4     import logging
5     import threading
6     import traceback
7     import uuid
8     from concurrent.futures import ThreadPoolExecutor
9     import uvicorn
10    from dotenv import load_dotenv
11
12    load_dotenv()
13
14    # Centralized logging setup for the indexer (hot paths use logger, CLI output uses print
for direct user-facing messages).
15    logging.basicConfig(
16        level=logging.INFO,
17        format='%(asctime)s [%(levelname)s] %(name)s: %(message)s',
18        datefmt='%H:%M:%S'
19    )

```

```

20     logger = logging.getLogger(__name__)
21     from fastapi import FastAPI, HTTPException, Request
22     from fastapi.middleware.cors import CORSMiddleware
23     from pydantic import BaseModel
24     from typing import List, Dict, Any, Optional
25
26     from fastapi.staticfiles import StaticFiles
27     from fastapi.responses import FileResponse, HTMLResponse, JSONResponse
28     from backend.pipeline.validators.base import registry
29     from backend.infrastructure.database import Database
30     from backend.pipeline.segmenters.base import segmenter_registry
31     from backend.pipeline.stitcher.compiler import VideoCompiler
32     from backend.utils.hashing import compute_video_id # canonical file-identity hashing
33     from backend.utils.paths import resolve_under_root, safe_output_filename,
validate_input_path
34     from backend.config import load_config as load_app_config, AppConfig, StitchingConfig #
new typed config
35     from backend.results import Failure # standardized error/result contract
36     from backend.reporting import emit_indexer_report
37
38     app = FastAPI(title="Sports Highlight Indexer API")
39
40     # Enable CORS for the local web interface. allow_origins='' with allow_credentials=True
is
41     # rejected by browsers per the fetch spec and is an unsafe default to carry into the
42     # auth/tenancy work; this tool uses no cookies, so credentials stay off.
43     app.add_middleware(
44         CORSMiddleware,
45         allow_origins=["*"],
46         allow_credentials=False,
47         allow_methods=["*"],
48         allow_headers=["*"],
49     )
50
51     # Serves static frontend files directly (now under api/ per approved structure)
52     os.makedirs("backend/api/static", exist_ok=True)
53     app.mount("/static", StaticFiles(directory="backend/api/static"), name="static")
54
55     @app.get("/", response_class=HTMLResponse)
56     def serve_index():
57         index_path = "backend/api/static/index.html"
58         if os.path.exists(index_path):
59             with open(index_path, "r", encoding="utf-8") as f:
60                 return f.read()
61         return "<h3>Sports Highlight Indexer Server is Running. Please create index.html in
backend/api/static/</h3>"
62
63     # Global variables initialized at startup
64     db_instance: Optional[Database] = None
65     global_config: Optional[AppConfig] = None # now typed (Pydantic model)
66
67     # Async job executor (DEFERRED_HARDENING_PLAN #16). max_workers=1 on purpose: a single
68     # self-hosted box serializes GPU work, and the Gemini provider is rate-fragile under
69     # parallel uploads (2026-06-10 ops learning: serial uploads + backoff). The segmenters
70     # already parallelize their AI handoff internally via their own pools.
71     _job_executor: Optional[ThreadPoolExecutor] = None
72
73     def _get_executor() -> ThreadPoolExecutor:
74         """Lazy module-global executor; tests monkeypatch this for inline execution."""
75         global _job_executor
76         if _job_executor is None:
77             _job_executor = ThreadPoolExecutor(max_workers=1,
thread_name_prefix="jobworker")
78         return _job_executor
79

```

```

80 # Legacy thin wrapper for any remaining direct calls during transition.
81 # New code should import load_app_config / AppConfig from backend.config directly.
82 def load_config(config_path: str = "config.json") -> Dict[str, Any]:
83     cfg = load_app_config(config_path)
84     return cfg.model_dump(mode="json")
85
86 # --- API Models ---
87 class ProcessRequest(BaseModel):
88     video_path: str
89     player_pool: Optional[List[str]] = None
90     max_frames: Optional[int] = None
91     segmenter: Optional[str] = None
92
93 class QueryRequest(BaseModel):
94     video_id: str
95     server: Optional[str] = None
96     receiver: Optional[str] = None
97     duration_min: Optional[float] = None
98     event_type: Optional[str] = None
99
100 class CompileRequest(BaseModel):
101     video_id: str
102     segment_ids: List[int]
103     output_filename: str
104
105 def _finalize_reingest(video_id: str, segments, play_wm: int, cand_wm: int) -> None:
106     """Commit a (re)segmentation with destroy-AFTER-confirm semantics.
107
108     On a non-empty run, drop the PRIOR rows (id <= the pre-run watermark) so only the
109     fresh segments remain. On an empty run, drop the NEW partial rows (id > watermark)
110     and
111     keep the prior good results intact ? a failed/empty re-run never destroys prior
112     data.
113     """
114     if segments:
115         db_instance.prune_segments(video_id, play_upto=play_wm, cand_upto=cand_wm)
116     else:
117         db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
118
119 # --- API Endpoints ---
120 @app.get("/api/config")
121 def get_config():
122     return global_config
123
124 def _execute_processing(req: ProcessRequest, progress=None) -> Dict[str, Any]:
125     """The full hash ? validate ? segment ? report pipeline for one video.
126
127     This is the former synchronous /api/process body, unchanged in behavior, now run on
128     the job worker thread (DEFERRED_HARDENING_PLAN #16). Returns the same response dict
129     the sync endpoint used to return (UI compatibility); the per-video observability
130     report is still emitted in `finally:` and grafted as response["reports"].
131
132     `progress` is an optional callable(stage: str) used to surface coarse job progress.
133     Raises on unexpected internal errors ? the caller records those as a job failure
134     (after this function has already restored the pre-run segments, see below).
135     """
136     notify = progress or (lambda stage: None)
137
138     notify("hashing")
139     video_id = compute_video_id(req.video_path)
140     was_reingest = db_instance.get_video(video_id) is not None
141
142     # 1. Run pluggable validators (with-context variant surfaces ffprobe_meta for the
143     report)

```

```

142     notify("validating")
143     logger.info(f"Running validations for {req.video_path}...")
144     val_results, val_context = registry.run_all_with_context(req.video_path,
global_config)
145     failures = [r for r in val_results if not r.passed]
146
147     # typed AppConfig: attribute access for the default segmenter (with safe fallback)
148     default_seg = getattr(getattr(global_config, "indexing", None), "default_segmenter",
"motion")
149     seg_name = req.segmenter or default_seg
150     segmenter_actual = None
151     segmentation_ran = False
152     response: Optional[Dict[str, Any]] = None
153     try:
154         if failures:
155             # add_video is an UPSERT (no cascade) ? a failed re-validation only updates
156             # status; it no longer destroys the video's prior good segments.
157             db_instance.add_video(video_id, req.video_path, "failed", [r.model_dump()
for r in val_results])
158             response = {
159                 "video_id": video_id,
160                 "status": "failed",
161                 "message": "Video failed validation checks.",
162                 "results": [r.model_dump() for r in val_results],
163             }
164             return response
165
166         # 2. Run segmenter & indexer
167         logger.info("[API] Video passed checks. Segmenting and indexing...")
168         db_instance.add_video(video_id, req.video_path, "processed", [r.model_dump() for
r in val_results])
169
170         segmenter_cls = segmenter_registry.get(seg_name)
171         if not segmenter_cls:
172             # Submit-time validation already 400s unknown names; this is the defensive
173             # in-worker path (e.g. config default pointing at an unregistered
segmenter).
174             available = list(segmenter_registry.list_available().keys())
175             response = {
176                 "video_id": video_id,
177                 "status": "failed",
178                 "message": f"Segmenter '{seg_name}' not found. Available segmenters:
{available}",
179                 "results": [r.model_dump() for r in val_results],
180                 "play_segments": [],
181             }
182             return response
183
184         logger.info(f"[API] Using segmenter: {seg_name}")
185         notify(f"segmenting ({seg_name})")
186         segmenter_actual = seg_name
187         # Destroy-AFTER-confirm: remember the pre-run rows, then keep new + drop old
only
188         # if the run yields segments; otherwise drop the new partial rows and keep the
old.
189         play_wm, cand_wm = db_instance.segment_watermarks(video_id)
190         segmenter = segmenter_cls(db_instance, global_config)
191         try:
192             result = segmenter.process_video(video_id, req.video_path, req.player_pool,
req.max_frames)
193         except Exception:
194             # A raising segmenter must not leave half-written rows: restore the pre-run
195             # state (same destroy-after-confirm contract as the Failure branch), then
let
196             # the job worker record the failure.

```



```

197         db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
198         raise
199     segmentation_ran = True
200
201     if isinstance(result, Failure):
202         logger.error(f"[API] Segmenter failed: {result.message}")
203         db_instance.prune_segments(video_id, play_after=play_wm, cand_after=cand_wm)
204         response = {
205             "video_id": video_id,
206             "status": "failed",
207             "message": f"Segmenter '{seg_name}' failed: {result.message}",
208             "results": [r.model_dump() for r in val_results],
209             "play_segments": [],
210         }
211         return response
212
213     segments = result.value if hasattr(result, "value") else result
214     notify("finalizing")
215     _finalize_reingest(video_id, segments, play_wm, cand_wm)
216
217     response = {
218         "video_id": video_id,
219         "status": "success",
220         "message": f"Successfully indexed {len(segments)} play segments using
221 '{seg_name}' segmenter.",
222         "results": [r.model_dump() for r in val_results],
223         "play_segments": segments,
224     }
225     return response
226
227     finally:
228         # Always emit the per-video observability report (next to the video, or to
229         # report.output_dir). Never raises. Surface the paths in the response.
230         paths = emit_indexer_report(
231             db=db_instance, video_id=video_id, video_path=req.video_path,
232             config=global_config,
233             val_results=val_results, val_context=val_context,
234             segmenter_requested=req.segmenter, segmenter_actual=segmenter_actual,
235             was_reingest=was_reingest, segmentation_ran=segmentation_ran,
236         )
237         if paths and isinstance(response, dict):
238             response["reports"] = paths
239
240
241     class JobWaiting(Exception):
242         """Raised inside the pipeline to SELF-SCHEDULE a job (EXECUTION_POLICY.md P3,
243         WAIT_AND_RESUME). It means "I can't finish now ? park me and resume in `delay_sec`,
244         NOT a failure. The worker persists `next_poll_at` + `progress` (via
245         `set_job_waiting`),
246         releases the thread, and re-claims the job when due (`claim_due_job`). No work to
247         date
248         is the segmenter's `_execute_processing`, run UNDER the destroy-after-confirm
249         contract.
250
251         The wiring is additive: the production segmenter path never raises this today (zero
252         behaviour change), so a job runs exactly as before. The hook is what a later slice
253         (resumable AI handoff) raises once a long op exhausts its in-process wait budget.
254         """
255
256         def __init__(self, delay_sec: float, progress: Optional[str] = None) -> None:
257             super().__init__(f"job parked for {delay_sec:.0f}s")
258             self.delay_sec = max(0.0, float(delay_sec))
259             self.progress = progress
260
261         def _job_to_request(job: Dict[str, Any]) -> ProcessRequest:

```

```

257     """Reconstruct the submit-time ProcessRequest from a claimed job row, so a job
258     re-claimed by `claim_due_job` (queued OR resumed-from-waiting) runs identically to
259     the original submit. The row stores exactly the ProcessRequest fields."""
260     return ProcessRequest(
261         video_path=job["video_path"],
262         player_pool=job.get("player_pool"),
263         max_frames=job.get("max_frames"),
264         segmenter=job.get("segmenter"),
265     )
266
267
268 def _run_claimed_job(job: Dict[str, Any]) -> None:
269     """Run ONE job already claimed (`status='running'`) by `claim_due_job`, recording
270     its
271     terminal state ? OR parking it for resume on `JobWaiting` (WAIT_AND_RESUME).
272
273     Job `status` is the EXECUTION state (queued|waiting|running|done|failed): 'done'
274     means
275     the worker finished ? the pipeline's own verdict (validation failure etc.) lives in
276     result["status"]; 'failed' means the worker itself crashed/raised; 'waiting' means
277     the
278     job self-scheduled and will be re-claimed when `next_poll_at` is due.
279     """
280     job_id = job["id"]
281     req = _job_to_request(job)
282     try:
283         result = _execute_processing(
284             req, progress=lambda stage: db_instance.set_job_progress(job_id, stage))
285         if result.get("video_id"):
286             db_instance.set_job_video_id(job_id, result["video_id"])
287             db_instance.mark_job_done(job_id, result)
288     except JobWaiting as w:
289         # Cooperative re-enqueue: persist progress + next_poll_at, release the worker.
290         logger.info(f"Job {job_id} parked for {w.delay_sec:.0f}s (resume when due).")
291         db_instance.set_job_waiting(job_id, w.delay_sec, progress=w.progress)
292     except Exception as e:
293         logger.error(f"Job {job_id} crashed: {e}\n{traceback.format_exc()}")
294         db_instance.mark_job_failed(job_id, f"{type(e).__name__}: {e}")
295
296
297 def _drain_due_jobs() -> int:
298     """One worker tick: claim and run every currently-runnable job (queued, or waiting
299     whose
300     `next_poll_at` is due), earliest-due first, until the table has nothing runnable.
301
302     Each parked job (`JobWaiting` ? status 'waiting') drops out of this drain and is
303     picked
304     up by a FUTURE drain once due; we schedule that follow-up drain via
305     `_schedule_drain` so
306     a single-worker box keeps making progress with no central timer (the DB is the
307     clock).
308
309     Returns the number of jobs that reached a terminal/parked state this tick.
310     """
311     ran = 0
312     while True:
313         job = db_instance.claim_due_job()
314         if job is None:
315             break
316         ran += 1
317         _run_claimed_job(job)
318     # If anything is parked-but-not-yet-due, arrange a wake-up so it resumes without a
319     new
320     # submit. On the single-box executor this is a delayed re-drain; multi-worker just
321     relies
322     # on the next submit/drain. Best-effort ? never raises into the worker.

```

```

313     _schedule_next_drain_if_waiting()
314     return ran
315
316
317     def _schedule_next_drain_if_waiting() -> None:
318         """If any job is parked in 'waiting', arm a re-drain for when the soonest one is
due, so a
319         self-scheduled job resumes with no further client submit. The drain itself re-checks
320         due-ness, so an early wake is harmless and a clamped wake for a far-future job is
fine."""
321         try:
322             delay = db_instance.seconds_until_next_due()
323         except Exception as e: # pragma: no cover - defensive; never wedge the worker
324             logger.error(f"Could not compute next-due delay: {e}")
325             return
326         if delay is None:
327             return # nothing waiting
328         # Clamp so a far-future job doesn't pin a thread forever; the drain re-checks due-
ness.
329         _arm_wake_timer(max(0.0, min(delay, _MAX_DRAIN_WAKE_SEC)))
330
331
332     def _arm_wake_timer(delay_sec: float) -> None:
333         """Arm a single daemon timer that re-submits a drain after ``delay_sec`` (no central
334         scheduler ? the DB is the clock). Isolated as the sole timer-creating seam so tests
can
335         monkeypatch it to a no-op and assert the *scheduling decision* without real
waiting."""
336         timer = threading.Timer(delay_sec, lambda: _get_executor().submit(_drain_due_jobs))
337         timer.daemon = True
338         timer.start()
339
340
341     #: A parked job further out than this is re-checked by a capped wake rather than one
long
342     #: sleeping timer ? keeps the wake cadence bounded without a central scheduler.
343     _MAX_DRAIN_WAKE_SEC = 60.0
344
345
346     @app.post("/api/process", status_code=202)
347     def process_video(req: ProcessRequest):
348         """Submit a processing job (202 + job_id). Poll GET /api/jobs/{job_id} for state.
349
350         Fast-fail checks (bad path, missing file, unknown segmenter) happen here so the
351         client gets an immediate 4xx; everything slow ? including MD5 hashing of the file ?
352         runs on the worker (DEFERRED_HARDENING_PLAN #16). The row is created 'queued'; the
353         worker drains it via `claim_due_job` (EXECUTION_POLICY.md P3 self-scheduling loop).
354         """
355         allowed_root = getattr(getattr(global_config, "security", None),
"allowed_video_root", None)
356         try:
357             validate_input_path(req.video_path, allowed_root=allowed_root)
358         except ValueError as e:
359             raise HTTPException(status_code=400, detail=str(e)) from e
360         if not os.path.exists(req.video_path):
361             raise HTTPException(status_code=404, detail=f"Video file not found at
'{req.video_path}'")
362         if req.segmenter and not segmenter_registry.get(req.segmenter):
363             available = list(segmenter_registry.list_available().keys())
364             raise HTTPException(
365                 status_code=400,
366                 detail=f"Segmenter '{req.segmenter}' not found. Available segmenters:
{available}"
367             )
368

```

```

369     job_id = uuid.uuid4().hex
370     if not db_instance.create_job(job_id, req.video_path, req.segmenter,
371                                 req.player_pool, req.max_frames):
372         raise HTTPException(status_code=500, detail="Failed to persist job record.")
373     _get_executor().submit(_drain_due_jobs)
374     return JSONResponse(
375         status_code=202,
376         content={"job_id": job_id, "status": "queued", "poll": f"/api/jobs/{job_id}"},
377     )
378
379
380 @app.get("/api/jobs/{job_id}")
381 def get_job(job_id: str):
382     """Job state: {status, progress, result, reports}. `status` = execution state
383     (queued|running|done|failed); the pipeline verdict is result["status"]."""
384     job = db_instance.get_job(job_id)
385     if not job:
386         raise HTTPException(status_code=404, detail=f"Unknown job_id '{job_id}'")
387     result = job.get("result")
388     return {
389         "job_id": job["id"],
390         "status": job["status"],
391         "progress": job.get("progress"),
392         "video_id": job.get("video_id"),
393         "error": job.get("error"),
394         "result": result,
395         "reports": (result or {}).get("reports"),
396         "created_at": job.get("created_at"),
397         "started_at": job.get("started_at"),
398         "finished_at": job.get("finished_at"),
399     }
400
401 # --- Chunked upload (B2, PR-2) -----
402 # Browser uploads arrive in sequential parts ? security.max_part_mb (free-tier edge
403 # proxies cap request bodies at ~100 MB ? HOSTING_PLAN $2), are assembled server-side
404 # under security.upload_dir, then submitted to /api/process by the returned path.
405 # Upload state is in-process by design (this is the single-box alpha): a server restart
406 # aborts partial uploads and the client re-inits. Per-user partitioning of upload_dir
407 # arrives with PR-3 identity scoping.
408
409 _uploads: Dict[str, Dict[str, Any]] = {}
410 _uploads_lock = threading.Lock()
411
412
413 class UploadInitRequest(BaseModel):
414     filename: str
415     total_size_bytes: Optional[int] = None
416
417
418 class UploadCompleteRequest(BaseModel):
419     total_parts: int
420
421
422 def _upload_cfg():
423     sec = getattr(global_config, "security", None)
424     upload_dir = getattr(sec, "upload_dir", None) or "output/uploads"
425     max_bytes = int(getattr(sec, "max_upload_mb", 4096) or 4096) * 1024 * 1024
426     part_bytes = int(getattr(sec, "max_part_mb", 95) or 95) * 1024 * 1024
427     exts = [e.lower().lstrip(".") for e in (getattr(sec, "allowed_upload_extensions",
428     None) or ["mp4", "mov"])]
429     return upload_dir, max_bytes, part_bytes, exts
430
431
432 def _abort_upload(upload_id: str) -> None:
433     with _uploads_lock:

```

```

433         st = _uploads.pop(upload_id, None)
434     if st:
435         try:
436             os.remove(st["tmp_path"])
437         except OSError:
438             pass
439
440
441     @app.post("/api/upload/init")
442     def upload_init(req: UploadInitRequest):
443         """Start a chunked upload. Returns upload_id; PUT sequential parts next."""
444         upload_dir, max_bytes, part_bytes, exts = _upload_cfg()
445         try:
446             name = safe_output_filename(req.filename)
447         except ValueError as e:
448             raise HTTPException(status_code=400, detail=str(e)) from e
449         ext = os.path.splitext(name)[1].lower().lstrip(".")
450         if ext not in exts:
451             raise HTTPException(status_code=400,
452                                 detail=f"Extension '{ext}' not allowed. Allowed:
{sorted(exts)}")
453         if req.total_size_bytes is not None and req.total_size_bytes > max_bytes:
454             raise HTTPException(status_code=413,
455                                 detail=f"File exceeds the {max_bytes // (1024 * 1024)} MB
upload limit.")
456         os.makedirs(upload_dir, exist_ok=True)
457         upload_id = uuid.uuid4().hex
458         tmp_path = os.path.join(upload_dir, f".partial-{upload_id}")
459         open(tmp_path, "wb").close()
460         with _uploads_lock:
461             _uploads[upload_id] = {"filename": name, "tmp_path": tmp_path, "parts": 0,
"bytes": 0}
462         return {"upload_id": upload_id, "max_part_mb": part_bytes // (1024 * 1024),
463                 "next_part": f"/api/upload/{upload_id}/part/0"}
464
465
466     @app.put("/api/upload/{upload_id}/part/{index}")
467     async def upload_part(upload_id: str, index: int, request: Request):
468         """Append one part (raw bytes body). Parts are strictly sequential (0,1,2,...)."""
469         _, max_bytes, part_bytes, _ = _upload_cfg()
470         with _uploads_lock:
471             st = _uploads.get(upload_id)
472             if st is None:
473                 raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
474             if index != st["parts"]:
475                 raise HTTPException(status_code=409,
476                                     detail=f"Out-of-order part: expected {st['parts']}, got
{index} (parts are sequential)")
477
478             received = 0
479             overflow = None
480             with open(st["tmp_path"], "ab") as fh:
481                 async for chunk in request.stream():
482                     received += len(chunk)
483                     if received > part_bytes:
484                         overflow = f"Part exceeds the {part_bytes // (1024 * 1024)} MB per-part
limit."
485                         break
486                     if st["bytes"] + received > max_bytes:
487                         overflow = f"Upload exceeds the {max_bytes // (1024 * 1024)} MB total
limit."
488                         break
489                     fh.write(chunk)
490             if overflow:
491                 _abort_upload(upload_id)

```

```

492         raise HTTPException(status_code=413, detail=overflow + " Upload aborted ? re-
init to retry.")
493
494     with _uploads_lock:
495         st["parts"] += 1
496         st["bytes"] += received
497     return {"upload_id": upload_id, "received_parts": st["parts"], "received_bytes":
st["bytes"]}
498
499
500     @app.post("/api/upload/{upload_id}/complete")
501     def upload_complete(upload_id: str, req: UploadCompleteRequest):
502         """Finalize: verify part count, move into place, return {path, video_id} for
/api/process."""
503         upload_dir, _, _, _ = _upload_cfg()
504         with _uploads_lock:
505             st = _uploads.get(upload_id)
506             if st is None:
507                 raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
508             if st["parts"] == 0 or st["parts"] != req.total_parts:
509                 raise HTTPException(status_code=409,
510                                     detail=f"Expected {req.total_parts} part(s), received
{st['parts']}.")
511             final_path = os.path.join(upload_dir, st["filename"])
512             if os.path.exists(final_path): # never overwrite an earlier upload with the same
name
513                 stem, ext = os.path.splitext(st["filename"])
514                 final_path = os.path.join(upload_dir, f"{stem}-{upload_id[:8]}{ext}")
515             os.replace(st["tmp_path"], final_path)
516             with _uploads_lock:
517                 _uploads.pop(upload_id, None)
518             video_id = compute_video_id(final_path)
519             return {"path": os.path.abspath(final_path), "video_id": video_id,
520                     "size_bytes": os.path.getsize(final_path),
521                     "next": "POST /api/process with this path"}
522
523
524     @app.delete("/api/upload/{upload_id}")
525     def upload_abort(upload_id: str):
526         """Abort an in-flight upload and delete its partial file."""
527         with _uploads_lock:
528             known = upload_id in _uploads
529             if not known:
530                 raise HTTPException(status_code=404, detail=f"Unknown upload_id '{upload_id}'")
531             _abort_upload(upload_id)
532             return {"status": "aborted", "upload_id": upload_id}
533
534
535     # --- Highlight download (B3, PR-2) -----
536
537     @app.get("/api/highlights/{video_id}/{filename}")
538     def download_highlight(video_id: str, filename: str):
539         """Stream a compiled highlight reel ? `filename` is the bare name given to
/api/compile
540         (which only writes into output_dir; the browser previously got back an unreachable
541         server-local path)."""
542         try:
543             name = safe_output_filename(filename)
544         except ValueError as e:
545             raise HTTPException(status_code=400, detail=str(e)) from e
546         if not db_instance.get_video(video_id):
547             raise HTTPException(status_code=404, detail="Indexed video record not found.")
548         output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output")
549         or "output"
550         path = os.path.join(output_dir, name)

```

```

550         try:
551             path = resolve_under_root(path, output_dir)
552         except ValueError as e:
553             raise HTTPException(status_code=400, detail=str(e)) from e
554     if not os.path.isfile(path):
555         raise HTTPException(status_code=404,
556                             detail=f"No compiled file named '{name}'. Compile first via
/api/compile.")
557     media = "video/quicktime" if name.lower().endswith(".mov") else "video/mp4"
558     return FileResponse(path, media_type=media, filename=name)
559
560
561 @app.post("/api/query")
562 def query_play_segments(req: QueryRequest):
563     filters = {
564         "server": req.server,
565         "receiver": req.receiver,
566         "duration_min": req.duration_min,
567         "event_type": req.event_type
568     }
569     segments = db_instance.query_play_segments(req.video_id, filters)
570     return {"play_segments": segments}
571
572 @app.post("/api/compile")
573 def compile_highlights(req: CompileRequest):
574     video = db_instance.get_video(req.video_id)
575     if not video:
576         raise HTTPException(status_code=404, detail="Indexed video record not found.")
577
578     # Retrieve matching play segments from db
579     all_segments = db_instance.query_play_segments(req.video_id, {})
580     matched_segments = [s for s in all_segments if s["id"] in req.segment_ids]
581
582     if not matched_segments:
583         raise HTTPException(status_code=400, detail="No matching play segments found to
stitch.")
584
585     # Reject path traversal / absolute output names (os.path.join would otherwise let an
586     # absolute or '..'-laden output_filename write anywhere on disk).
587     try:
588         out_name = safe_output_filename(req.output_filename)
589     except ValueError as e:
590         raise HTTPException(status_code=400, detail=str(e)) from e
591
592     # The configured shot-count floor (stitching.min_shot_count) applies on the API
593     # path too ? parity with scripts/run_process_and_stitch.py. Segments WITHOUT
shot_count
594     # metadata are exempt: the floor filters known counts only, so explicitly
595     # requested manual/legacy segments can't silently vanish under the default floor.
596     stitching = getattr(global_config, "stitching", None) or StitchingConfig()
597     kept = []
598     for s in matched_segments:
599         meta = s.get("metadata") or {}
600         sc = meta.get("shot_count") if isinstance(meta, dict) else None
601         try:
602             if sc is not None and int(sc) < stitching.min_shot_count:
603                 continue
604             except (TypeError, ValueError):
605                 pass # unparseable count -> treat as unknown, keep
606             kept.append(s)
607     if len(kept) < len(matched_segments):
608         logger.info(f"[compile] stitching.min_shot_count={stitching.min_shot_count}:
dropped "
609                    f"{len(matched_segments) - len(kept)}/{len(matched_segments)}
segments below the floor.")

```

```

610         if not kept:
611             raise HTTPException(status_code=400,
612                                 detail=f"All {len(matched_segments)} matching segments fall
below "
613
614         f"stitching.min_shot_count={stitching.min_shot_count}.")
615
616         # Extract start/end time pairs
617         time_pairs = [(s["start_time"], s["end_time"]) for s in kept]
618
619         # Run compiler with the configured stitching paddings + optional branding watermark
620         output_dir = getattr(getattr(global_config, "output", None), "output_dir", "output")
621         or "output"
622         compiler = VideoCompiler(output_dir, begin_padding=stitching.begin_padding,
623                                 end_padding=stitching.end_padding,
624                                 watermark=stitching.watermark)
625         try:
626             out_path = compiler.compile_highlights(video["filepath"], time_pairs, out_name)
627             return {"status": "success", "filepath": out_path}
628         except Exception as e:
629             raise HTTPException(status_code=500, detail=f"Highlight stitching failed:
{str(e)}") from e
630
631     # --- CLI Debug Utility & Orchestration ---
632     def run_cli_diagnose_clip(video_path: str, timestamp: float):
633         """CLI debugging utility to examine segment properties around a timestamp."""
634         print("=" * 60)
635         print(f"DEBUGGING VIDEO CLIP: {video_path} at {timestamp}s")
636         print("=" * 60)
637
638         cfg = load_config() # legacy wrapper still works, returns dict for now
639
640         # 1. Validation check diagnostics
641         print("[DIAGNOSTIC] Running validation framework...")
642         results = registry.run_all(video_path, cfg)
643         for r in results:
644             status_symbol = "[PASS]" if r.passed else "[FAIL]"
645             print(f" {status_symbol} {r.validator_name}: {r.message}")
646             if r.details:
647                 print(f"      Details: {r.details}")
648
649         # 2. Local segment analysis around the timestamp (sampling +/- 5 seconds)
650         print("-" * 60)
651         print("[DIAGNOSTIC] Analyzing local motion around the timestamp...")
652         import cv2
653         cap = cv2.VideoCapture(video_path)
654         if not cap.isOpened():
655             print("[FAIL] OpenCV could not open video.")
656             return
657
658         fps = cap.get(cv2.CAP_PROP_FPS)
659         total_frames = cap.get(cv2.CAP_PROP_FRAME_COUNT)
660
661         start_frame = max(0, int((timestamp - 3.0) * fps))
662         end_frame = min(int(total_frames), int((timestamp + 3.0) * fps))
663
664         # Measure frame-by-frame changes in sample region
665         cap.set(cv2.CAP_PROP_POS_FRAMES, start_frame)
666         prev_gray = None
667         motions = []
668
669         for _ in range(start_frame, end_frame):
670             ret, frame = cap.read()
671             if not ret:
672                 break

```



```

670         gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
671         gray = cv2.GaussianBlur(gray, (21, 21), 0)
672
673         if prev_gray is not None:
674             diff = cv2.absdiff(prev_gray, gray)
675             _, thresh = cv2.threshold(diff, 25, 255, cv2.THRESH_BINARY)
676             motion_ratio = cv2.countNonZero(thresh) / (thresh.shape[0] *
thresh.shape[1])
677             motions.append(motion_ratio)
678             prev_gray = gray
679
680         cap.release()
681
682         if motions:
683             avg_motion = sum(motions) / len(motions)
684             # Support both legacy dict (from wrapper) and future direct model
685             if hasattr(cfg, "indexing"):
686                 threshold = getattr(cfg.indexing, "motion_threshold", 0.08)
687             else:
688                 threshold = cfg.get("indexing", {}).get("motion_threshold", 0.08) # type:
ignore[attr-defined]
689             print(f" Sample Frame Count: {len(motions)}")
690             print(f" Average motion index around timestamp: {round(avg_motion, 4)}
(threshold: {threshold})")
691             if avg_motion > threshold:
692                 print(" Result: [ACTIVE PLAY] High motion detected. Segment classifies as
active play.")
693             else:
694                 print(" Result: [DEAD TIME] Idle movement. Segment classified as dead
time.")
695         else:
696             print(" [ERROR] No visual frames were extracted in the sample range.")
697
698         print("=" * 60)
699
700     def main():
701         parser = argparse.ArgumentParser(description="Sports Highlight Indexer
Orchestrator")
702         parser.add_argument("--video-path", help="Local path to the MP4 sports video file")
703         parser.add_argument("--output-dir", default="output", help="Directory where
processed clips/highlights are saved")
704         parser.add_argument("--setup", action="store_true", help="Launch dependency checker
and setup utility")
705         parser.add_argument("--debug-clip", type=float, help="Timestamp (in seconds) of a
video clip to run detailed diagnostics on")
706         parser.add_argument("--server", action="store_true", help="Run the FastAPI local API
server")
707         parser.add_argument("--port", type=int, default=8000, help="Port to run the FastAPI
server on")
708         parser.add_argument("--max-frames", type=int, help="Limit processing to the first N
sub-sampled frames for debugging")
709
710         available_segmenters = list(segmenter_registry.list_available().keys())
711         parser.add_argument("--segmenter", choices=available_segmenters, help=f"Choose rally
detection segmenter. Available: {available_segmenters}")
712         parser.add_argument("--trim-start", type=float, help="Start time in seconds for the
debug trim segment feature")
713         parser.add_argument("--trim-end", type=float, help="End time in seconds for the
debug trim segment feature")
714         parser.add_argument("--trim-output", help="Output filename for the debug trimmed
segment (saves in output-dir unless absolute path)")
715
716         args = parser.parse_args()
717
718         if args.setup:

```

```

719         # Invoke the diagnostics/bootstrap script (renamed from the old root
720         # setup.py to scripts/doctor.py so it no longer shadows the PEP 517
721         # build system). Resolve its path from this file so `indexer --setup`
722         # works regardless of the caller's cwd.
723         import subprocess
724         doctor_path = os.path.join(
725             os.path.dirname(os.path.abspath(__file__)),
726             "scripts", "doctor.py",
727         )
728         print(f"[INFO] Invoking diagnostics: {doctor_path} ...")
729         subprocess.run([sys.executable, doctor_path])
730         return
731
732     if args.trim_start is not None and args.trim_end is not None:
733         if not args.video_path:
734             print("[ERROR] Please provide --video-path to extract a segment.")
735             return
736
737         # Determine output path
738         out_filename = args.trim_output or "trimmed_segment.mp4"
739         if os.path.isabs(out_filename):
740             out_path = out_filename
741             out_dir = os.path.dirname(out_path)
742             out_name = os.path.basename(out_path)
743         else:
744             out_dir = args.output_dir
745             out_path = os.path.join(out_dir, out_filename)
746             out_name = out_filename
747
748         os.makedirs(out_dir, exist_ok=True)
749         print(f"[CLI] Extracting debug segment from {args.trim_start}s to
{args.trim_end}s from {args.video_path}...")
750
751         # global_config isn't initialized this early; load the typed config so the trim
752         # clip carries the same configured stitching paddings + watermark as a real
753         # highlight (watermark default-on).
754         stitching = load_app_config().stitching
755         compiler = VideoCompiler(out_dir, begin_padding=stitching.begin_padding,
756                                 end_padding=stitching.end_padding,
watermark=stitching.watermark)
757         try:
758             res_path = compiler.compile_highlights(args.video_path, [(args.trim_start,
args.trim_end)], out_name)
759             print(f"[SUCCESS] Trimmed segment saved to: {res_path}")
760         except Exception as e:
761             print(f"[ERROR] Failed to compile trimmed segment: {e}")
762         return
763
764     if args.debug_clip is not None:
765         if not args.video_path:
766             print("[ERROR] Please provide --video-path to debug a specific clip.")
767             return
768         run_cli_diagnose_clip(args.video_path, args.debug_clip)
769         return
770
771     # Initialize Global Config and DB
772     global global_config, db_instance
773     global_config = load_app_config() # returns typed AppConfig
774
775     # The CLI --output-dir overrides the configured output directory. It now lives
776     # on the typed model (OutputConfig.output_dir), so every consumer ? including
777     # /api/compile ? reads the same value instead of a write-only runtime hack.
778     global_config.output.output_dir = args.output_dir
779     os.makedirs(global_config.output.output_dir, exist_ok=True)
780

```

```

781         db_path = os.path.join(global_config.output.output_dir,
global_config.output.db_name)
782         db_instance = Database(db_path)
783
784         # Crash recovery (#16): jobs left queued/running by a previous process are dead ?
785         # the executor is in-process. Mark them failed so pollers see a terminal state.
786         swept = db_instance.fail_stale_jobs()
787         if swept:
788             logger.warning(f"[JOBS] Swept {swept} stale job(s) from a previous run ?
failed.")
789
790         # CLI processing mode (no web UI needed)
791         if args.video_path and not args.server:
792             print(f"[CLI] Processing video file: {args.video_path}")
793             if not os.path.exists(args.video_path):
794                 print(f"[FAIL] Video file not found: {args.video_path}")
795                 return
796
797             video_id = compute_video_id(args.video_path)
798             was_reingest = db_instance.get_video(video_id) is not None
799
800             # Validate (with-context variant surfaces ffprobe_meta for the report)
801             print("[CLI] Validating...")
802             val_results, val_context = registry.run_all_with_context(args.video_path,
global_config)
803             failures = [r for r in val_results if not r.passed]
804
805             # Save video status
806             status = "failed" if failures else "processed"
807             db_instance.add_video(video_id, args.video_path, status, [r.model_dump() for r
in val_results])
808
809             seg_name = args.segmenter or getattr(getattr(global_config, "indexing", None),
"default_segmenter", "motion")
810             segmenter_actual = None
811             segmentation_ran = False
812             try:
813                 print("\n" + "="*40)
814                 print("VALIDATION SUMMARY")
815                 print("="*40)
816                 for r in val_results:
817                     status_symbol = "[PASS]" if r.passed else "[FAIL]"
818                     print(f"{status_symbol} {r.validator_name}: {r.message}")
819                 print("="*40 + "\n")
820
821                 if failures:
822                     print("[FAIL] Video failed checks. Indexing halted.")
823                     return
824
825                 # Index
826                 segmenter_cls = segmenter_registry.get(seg_name)
827                 if not segmenter_cls:
828                     print(f"[FAIL] Segmenter '{seg_name}' not found.")
829                     return
830
831                 print(f"[CLI] Indexing play segments using segmenter: {seg_name}...")
832                 segmenter_actual = seg_name
833                 play_wm, cand_wm = db_instance.segment_watermarks(video_id)
834                 segmenter = segmenter_cls(db_instance, global_config)
835                 result = segmenter.process_video(video_id, args.video_path,
max_frames=args.max_frames)
836                 segmentation_ran = True
837
838                 if isinstance(result, Failure):
839                     print(f"[FAIL] Segmenter '{seg_name}' failed: {result.message}")

```

```

840         print("[FAIL] Indexing halted for video.")
841         segments = []
842     else:
843         segments = result.value if hasattr(result, "value") else result
844         print(f"[SUCCESS] Process complete. Indexed {len(segments)} play
segments inside database: {db_path}")
845         # Destroy-AFTER-confirm: keep new on success, restore prior on
empty/failure.
846         _finalize_reingest(video_id, segments, play_wm, cand_wm)
847     finally:
848         # Always emit the per-video observability report. Never raises.
849         paths = emit_indexer_report(
850             db=db_instance, video_id=video_id, video_path=args.video_path,
config=global_config,
851             val_results=val_results, val_context=val_context,
852             segmenter_requested=args.segmenter, segmenter_actual=segmenter_actual,
853             was_reingest=was_reingest, segmentation_ran=segmentation_ran,
854         )
855         if paths:
856             print(f"[REPORT] Technical report : {paths.get('technical_md')}")
857             print(f"[REPORT] Customer summary : {paths.get('customer_md')}")
858             print(f"[REPORT] Machine JSON      : {paths.get('json')}")
859         return
860
861     # Start Local Server Mode
862     if args.server or not args.video_path:
863         print(f"[SERVER] Starting local FastAPI server on
http://localhost:{args.port}...")
864         # Resume self-scheduled work orphaned by the restart (EXECUTION_POLICY.md P3):
the
865         # in-process wake-timer for any parked 'waiting' job died with the old process,
so
866         # kick one drain ? it reclaims due jobs and re-arms the wake-timer for the rest.
867         _get_executor().submit(_drain_due_jobs)
868         uvicorn.run(app, host="127.0.0.1", port=args.port)
869
870     if __name__ == "__main__":
871         main()
872

```

35. assistant (2026-06-18T01:34:24.214Z)

I found a potential issue in `main.py`. Let me examine `upload\_init` at line 459 closely:

```

```python
open(tmp_path, "wb").close()
```

```

This creates and immediately closes the file ? no leak since `.close()` is called. That's fine.

Let me look at the `upload\_part` more carefully. At line 480, the file is opened with `with open(...)` and writes happen. But there's a subtle concern: when overflow happens and `\_abort\_upload` is called inside `upload\_part`, the file `st["tmp\_path"]` is removed. But the `with` block at line 480 already closed the file by the time we reach line 491. Good.

Let me look at the remaining subprocess files: cleanup_caches.py, doctor.py, harvest.py, resolution_fps.py, and the splits.py/regression.py in eval.

36. user (2026-06-18T01:34:24.930Z)

```

1     """Cache-hygiene tool for the GPU highlight pipeline.
2
3     Scans the regenerable intermediates the pipeline leaves behind ? in the WSL stage
4     dir (`~/clips`) and the Windows `output/` tree ? classifies each entry as
5     **purge** (regenerable: frame caches, staged video copies, handoff clips, temp dirs)

```

```

6      or **keep** (durable: trajectory CSVs, the SQLite DB, final reels) and reports the
7      reclaimable space.
8
9      DRY-RUN BY DEFAULT ? it only *shows* what it would delete. Pass ``--apply`` to delete.
10
11         python -m tools.cleanup_caches                        # report only (safe)
12         python -m tools.cleanup_caches --apply                # delete purge candidates
13         python -m tools.cleanup_caches --keep-video 89605e80ed01 # protect one video's
cache
14         python -m tools.cleanup_caches --older-than 7         # only purge entries >7 days old
15         python -m tools.cleanup_caches --include-wasbcache    # also drop detector resume
caches
16
17     Pairs with the pipeline's own self-cleaning (indexing.{wasb,tracknet}.keep_frames=false,
18     which deletes frames after a SUCCESSFUL run); this tool reclaims what failed/old runs
19     and pre-self-cleaning videos left behind. Stdlib only; safe to run anywhere (the WSL
20     scan is skipped automatically when ``wsl`` is unavailable).
21     """
22     from __future__ import annotations
23
24     import argparse
25     import os
26     import shutil
27     import subprocess
28     import sys
29     from dataclasses import dataclass
30     from typing import List, Optional, Tuple
31
32     # ---- classification (pure; unit-tested) ----- #
33
34     # (category, regenerable, default_purge). default_purge=True ? deleted unless a flag
35     # protects it. wasbcache is regenerable but default_purge=False (it enables resume and
36     # is tiny) ? opt in with --include-wasbcache.
37     _KEEP = ("keep", False, False)
38
39
40     def classify(name: str, is_dir: bool, location: str) -> Tuple[str, bool, bool]:
41         """Classify a cache entry by name. Returns (category, regenerable,
default_purge)."""
42         low = name.lower()
43         if is_dir:
44             if low.endswith("_frames"):
45                 return ("frame cache", True, True)
46             if low.endswith("_wasbcache"):
47                 return ("detector resume cache", True, False) # opt-in via --include-
wasbcache
48             if low.startswith("chunks_") and low.endswith("_handoff"):
49                 return ("ai handoff clips", True, True)
50             if low.startswith("tmp"):
51                 return ("temp dir", True, True)
52             return ("other dir", False, False)
53         # files
54         if low.endswith((".csv",)):
55             return ("trajectory csv", False, False) # durable analysis output
56         if low.endswith((".db", ".sqlite", ".sqlite3")):
57             return ("database", False, False) # cached rallies ? never purge
58         if low.endswith((".log", ".sh", ".txt", ".json", ".md")):
59             return ("log/script", False, False)
60         if low.endswith((".mp4", ".avi", ".mov", ".mkv")):
61             # In the WSL stage dir, every video is a regenerable staged copy.
62             if location == "wsl":
63                 return ("staged video", True, True)
64             # In output/, a 1080p/proxy is regenerable; anything else is a produced reel.
65             if "_1080p" in low or "_proxy" in low or low.endswith("_proxy.mp4"):
66                 return ("proxy video", True, True)

```

```

67         return ("highlight reel", False, False)          # durable output ? keep
68         return ("other", False, False)
69
70
71 @dataclass
72 class Entry:
73     name: str
74     path: str          # display path (WSL ~/clips/<name> or windows abs)
75     size: int           # bytes
76     mtime: float        # epoch seconds (0 if unknown)
77     is_dir: bool
78     location: str        # "wsl" | "windows"
79
80     @property
81     def category(self) -> Tuple[str, bool, bool]:
82         return classify(self.name, self.is_dir, self.location)
83
84
85     def plan_deletions(entries: List[Entry], *, keep_video: Optional[str], older_than_days:
float,
86                        include_wasbcache: bool, now: float) -> Tuple[List[Entry],
List[Entry]]:
87         """Split entries into (to_delete, kept) per the flags. Pure ? no I/O."""
88         to_delete: List[Entry] = []
89         kept: List[Entry] = []
90         cutoff = older_than_days * 86400.0
91         for e in entries:
92             cat, _regen, default_purge = e.category
93             purge = default_purge or (include_wasbcache and cat == "detector resume cache")
94             if purge and keep_video and keep_video in e.name:
95                 purge = False
96             if purge and older_than_days > 0 and e.mtime > 0 and (now - e.mtime) < cutoff:
97                 purge = False
98             (to_delete if purge else kept).append(e)
99         return to_delete, kept
100
101
102 # ---- scanning (I/O) ----- #
103
104 def human(n: int) -> str:
105     f = float(n)
106     for unit in ("B", "K", "M", "G", "T"):
107         if f < 1024 or unit == "T":
108             return f"{f:.1f}{unit}" if unit != "B" else f"{int(f)}B"
109         f /= 1024
110     return f"{f:.1f}T"
111
112
113 def _wsl_lines(distro: str, cmd: str) -> List[str]:
114     """Run a bash command in WSL and return stdout lines ([] on any failure)."""
115     try:
116         res = subprocess.run(["wsl", "-d", distro, "bash", "-lc", cmd],
117                               capture_output=True, text=True, timeout=300)
118     except (OSError, subprocess.TimeoutExpired):
119         return []
120     return (res.stdout or "").splitlines()
121
122
123 def scan_wsl(distro: str, stage_dir: str) -> List[Entry]:
124     """List ~/clips entries with recursive size + mtime + type. [] if WSL absent.
125
126     Uses ``du -sb`` (recursive byte sizes, native-fast on ext4) merged with ``find
127     -printf`` (type + mtime) by basename. Deliberately avoids a ``for`` loop ? the
128     user's login shell mangles loop variables, but du/find arg-globbing is reliable.
129     """

```

```

130     if not shutil.which("wsl"):
131         return []
132     size_lines = _wsl_lines(distro, f"du -sb {stage_dir}/* {stage_dir}/.[!.]*
2>/dev/null || true")
133     meta_lines = _wsl_lines(distro, f"find {stage_dir}/ -maxdepth 1 -mindepth 1 "
134                             f"-printf '%y\\t%T@\\t%f\\n' 2>/dev/null || true")
135     meta = {} # basename -> (is_dir, mtime)
136     for line in meta_lines:
137         parts = line.split("\\t")
138         if len(parts) == 3:
139             y, mt, name = parts
140             try:
141                 meta[name] = (y == "d", float(mt))
142             except ValueError:
143                 continue
144     entries: List[Entry] = []
145     for line in size_lines:
146         if "\\t" not in line:
147             continue
148         sz, path = line.split("\\t", 1)
149         name = path.rsplit("/", 1)[-1]
150         try:
151             size = int(sz)
152         except ValueError:
153             continue
154         is_dir, mtime = meta.get(name, (False, 0.0))
155         entries.append(Entry(name=name, path=f"{stage_dir}/{name}", size=size,
156                             mtime=mtime, is_dir=is_dir, location="wsl"))
157     return entries
158
159
160 def _dir_size(path: str) -> int:
161     total = 0
162     for root, _dirs, files in os.walk(path):
163         for fn in files:
164             try:
165                 total += os.path.getsize(os.path.join(root, fn))
166             except OSError:
167                 pass
168     return total
169
170
171 def scan_windows(output_dir: str) -> List[Entry]:
172     """List output/ entries (top level) plus the output/wasb sub-tree."""
173     entries: List[Entry] = []
174     for base in (output_dir, os.path.join(output_dir, "wasb")):
175         if not os.path.isdir(base):
176             continue
177         for de in os.scandir(base):
178             # 'wasb' is scanned as its own base below ? don't also list it as a dir
179             here.
180             if base == output_dir and de.name == "wasb":
181                 continue
182             try:
183                 is_dir = de.is_dir()
184                 size = _dir_size(de.path) if is_dir else de.stat().st_size
185                 mtime = de.stat().st_mtime
186             except OSError:
187                 continue
188             entries.append(Entry(name=de.name, path=de.path, size=size,
189                                 mtime=mtime, is_dir=is_dir, location="windows"))
189     return entries
190
191
192 def _delete(e: Entry, distro: str) -> bool:

```

```

193     try:
194         if e.location == "wsl":
195             subprocess.run(["wsl", "-d", distro, "bash", "-lc", f'rm -rf {_q(e.path)}'],
196                             capture_output=True, text=True, timeout=600, check=True)
197         elif e.is_dir:
198             shutil.rmtree(e.path, ignore_errors=True)
199         else:
200             os.remove(e.path)
201         return True
202     except (OSError, subprocess.SubprocessError):
203         return False
204
205
206     def _q(path: str) -> str:
207         if path.startswith("~/"):
208             return "~/'" + path[2:].replace("'", "'\\'") + "'"
209         return "'" + path.replace("'", "'\\'") + "'"
210
211
212     def _report(title: str, rows: List[Entry]) -> int:
213         print(f"\n{title}")
214         if not rows:
215             print(" (none)")
216             return 0
217         total = 0
218         for e in sorted(rows, key=lambda x: x.size, reverse=True):
219             cat = e.category[0]
220             print(f" {human(e.size):>8} {cat:<22} {e.path}")
221             total += e.size
222         print(f" {'-' * 8}")
223         print(f" {human(total):>8} TOTAL ({len(rows)} items)")
224         return total
225
226
227     def main(argv: Optional[List[str]] = None) -> int:
228         ap = argparse.ArgumentParser(description="Report/clean regenerable pipeline caches
229 (dry-run by default).")
230         ap.add_argument("--apply", action="store_true", help="actually delete (default:
231 report only)")
232         ap.add_argument("--keep-video", default=None, help="protect entries whose name
233 contains this id/substring")
234         ap.add_argument("--older-than", type=float, default=0.0, help="only purge entries
235 older than N days")
236         ap.add_argument("--include-wasbcache", action="store_true", help="also purge
237 detector resume caches")
238         ap.add_argument("--stage-dir", default="~/clips", help="WSL stage dir (default
239 ~/clips)")
240         ap.add_argument("--distro", default="Ubuntu", help="WSL distro (default Ubuntu)")
241         ap.add_argument("--output-dir", default="output", help="Windows output dir (default
242 output)")
243         ap.add_argument("--summary", action="store_true",
244                         help="print a single-line nudge (only if reclaimable >= --threshold-
245 gb); for hooks")
246         ap.add_argument("--threshold-gb", type=float, default=5.0,
247                         help="--summary stays silent below this many reclaimable GB (default
248 5)")
249         args = ap.parse_args(argv)
250
251         import time
252         now = time.time()
253         entries = scan_wsl(args.distro, args.stage_dir) + scan_windows(args.output_dir)
254         to_delete, kept = plan_deletions(
255             entries, keep_video=args.keep_video, older_than_days=args.older_than,
256             include_wasbcache=args.include_wasbcache, now=now)

```



```

249         if args.summary:
250             reclaim = sum(e.size for e in to_delete)
251             if reclaim >= args.threshold_gb * (1024 ** 3):
252                 print(f"? {human(reclaim)} of regenerable pipeline cache reclaimable "
253                       f"? run `python -m tools.cleanup_caches` to review, `--apply` to
delete.")
254             return 0
255
256         reclaim = _report("PURGE candidates (regenerable):", to_delete)
257         _report("KEEP (durable / protected):", kept)
258         print(f"\nReclaimable: {human(reclaim)}")
259
260         if not args.apply:
261             print("\nDRY RUN ? nothing deleted. Re-run with --apply to delete the PURGE
list.")
262             return 0
263         if not to_delete:
264             print("\nNothing to delete.")
265             return 0
266
267         print(f"\nDeleting {len(to_delete)} items...")
268         freed, failed = 0, 0
269         for e in to_delete:
270             if _delete(e, args.distro):
271                 freed += e.size
272             else:
273                 failed += 1
274                 print(f" FAILED: {e.path}")
275         print(f"Freed ~{human(freed)}" + (f" ({failed} failed)" if failed else ""))
276         return 0
277
278
279 if __name__ == "__main__":
280     sys.exit(main())
281

```

37. assistant (2026-06-18T01:34:28.481Z)

Clean. Let me look at doctor.py, harvest.py, resolution_fps.py, and the eval splits.py.

38. user (2026-06-18T01:34:28.969Z)

```

1      """Build an observability report for one indexer run.
2
3      Pure-ish harvest: given the validation results/context, the persisted segments,
4      and run metadata, populate an `obsreport.RunRecorder` with the raw signals. The
5      *footgun rules* (which turn signals into flags) live declaratively in spec.yaml,
6      so this module only records facts faithfully ? it does not decide warnings.
7
8      Imported only after `backend.reporting` confirms obsreport is installed, so it may
9      import obsreport freely.
10     """
11     from __future__ import annotations
12
13     import os
14     from pathlib import Path
15     from typing import Any, Dict, List, Optional
16
17     try:
18         import obsreport # soft dep; only needed for RunRecorder / load_spec at runtime
19     except ImportError:
20         obsreport = None # type: ignore[assignment]
21
22     _AI_SEGMENTERS = {"yolo_hybrid", "tracknet_hybrid", "wasb_hybrid", "gemini"}
23     _SPEC_PATH = Path(__file__).resolve().parent / "spec.yaml"

```

```

24
25
26 def load_spec() -> "obsreport.ReportSpec":
27     return obsreport.load_spec(_SPEC_PATH)
28
29
30 # --- media metadata -----
31 def video_meta_from_context(val_context: Optional[Dict[str, Any]], video_path:
Optional[str]) -> Dict[str, Any]:
32     """Derive VideoMeta fields from the validators' shared ffprobe_meta. fps_source
33     distinguishes a real probe from the absence of one (the fallback footgun)."""
34     meta: Dict[str, Any] = {"fps_source": "unknown"}
35     if video_path and os.path.exists(video_path):
36         try:
37             meta["size_mb"] = round(os.path.getsize(video_path) / (1024 * 1024), 1)
38         except OSError:
39             pass
40
41     ffmata = (val_context or {}).get("ffprobe_meta") or {}
42     streams = ffmata.get("streams") or []
43     fmt = ffmata.get("format") or {}
44     if streams:
45         s = streams[0]
46         w, h = s.get("width"), s.get("height")
47         meta["width"], meta["height"] = w, h
48         if w and h:
49             meta["resolution"] = f"{w}x{h}"
50         fps_str = s.get("r_frame_rate", "0/1")
51         try:
52             num, den = (int(x) for x in fps_str.split("/"))
53             if den:
54                 meta["fps"] = round(num / den, 2)
55                 meta["fps_source"] = "probed"
56             except (ValueError, ZeroDivisionError):
57                 pass
58         if s.get("codec_name"):
59             meta["container"] = fmt.get("format_name") or s.get("codec_name")
60     if fmt.get("duration"):
61         try:
62             meta["duration_s"] = round(float(fmt["duration"]), 1)
63         except (ValueError, TypeError):
64             pass
65     if fmt.get("format_name"):
66         meta["container"] = fmt.get("format_name")
67
68     # Audio readiness signal (for Input Quality UX extension per review item 2).
69     # Guidelines make "record with audio on, mic unobstructed" a first-class requirement
70     # (ADR-007) because shuttle thwacks are a strong rally cue for recall.
71     # We only record presence here (cheap, from ffprobe already in context); the actual
72     # thwack onset probe (pipeline/audio/hit_detector) remains a diagnostic /
calibration
73     # tool for now.
74     audio_streams = [s for s in streams if (s or {}).get("codec_type") == "audio"]
75     meta["audio_present"] = len(audio_streams) > 0
76     return meta
77
78
79 # --- stage builders -----
80 def _validation_stage(rec: "obsreport.RunRecorder", val_results: List[Any]) -> None:
81     with rec.stage("validation") as st:
82         total = len(val_results)
83         passed = sum(1 for r in val_results if getattr(r, "passed", False))
84         st.add_metric("checks_passed", passed)
85         st.add_metric("checks_total", total)
86         failures = [r for r in val_results if not getattr(r, "passed", True)]

```

```

87         for r in failures:
88             st.messages.append(f"FAILED {getattr(r, 'validator_name', '?')}: {getattr(r,
'message', '')}")
89             st.status = "error" if failures else ("ok" if total else "not_run")
90
91
92     def _segmentation_stage(
93         rec: "obsreport.RunRecorder",
94         play_segments: List[Dict[str, Any]],
95         candidates: List[Dict[str, Any]],
96         segmenter_requested: Optional[str],
97         segmenter_actual: Optional[str],
98         config_default: Optional[str],
99         was_reingest: bool,
100        ran: bool,
101    ) -> None:
102        with rec.stage("segmentation") as st:
103            if not ran:
104                st.status = "not_run"
105            seg_count = len(play_segments)
106            total_play = round(sum(float(s.get("duration", 0.0)) for s in play_segments), 1)
107            detector_tag = None
108            if play_segments:
109                detector_tag = (play_segments[0].get("metadata") or {}).get("detector")
110            st.add_metric("candidate_windows", len(candidates))
111            st.add_metric("segment_count", seg_count, audience="both")
112            st.add_metric("total_play_s", total_play, unit="s")
113            # "diverges" only when a segmenter actually RAN, the user did NOT explicitly
114            # pick it, and it differs from config's default (the genuine surprise case).
115            # An explicit --segmenter override or a halted run is not a divergence.
116            matches_default = True
117            if ran and segmenter_actual and config_default:
118                matches_default = (segmenter_actual == config_default)
119            st.details.update({
120                "segmenter_requested": segmenter_requested,
121                "segmenter_actual": segmenter_actual,
122                "config_default": config_default,
123                "segmenter_explicitly_requested": segmenter_requested is not None,
124                "matches_config_default": matches_default,
125                "was_reingest": was_reingest,
126                "detector": detector_tag,
127            })
128            # Surface metadata trustworthiness at the data level (not just in a warning):
129            # the motion baseline fabricates per-segment metadata; everything else measures
130            it.
131            if ran and segmenter_actual:
132                st.details["metadata_source"] = (
133                    "synthetic/heuristic ? NOT measured" if segmenter_actual == "motion"
134                    else "measured (detector/AI)"
135                )
136
137     def _ai_stage(rec: "obsreport.RunRecorder", config: Dict[str, Any], segmenter_actual:
Optional[str],
138                play_segments: List[Dict[str, Any]], ran: bool,
139                run_signals: Optional[Dict[str, Any]] = None) -> None:
140         """Only meaningful for AI-based segmenters. Records whether the provider key was
141         present ? the signal the silent_provider_noop rule keys on ? plus any in-process
142         run signals (backend.reporting.run_signals), e.g. oversize chunk refusals."""
143         ai_based = segmenter_actual in _AI_SEGMENTERS
144         provider = (config.get("ai_provider") or "gemini")
145         key_env = f"{provider.upper()}_API_KEY"
146         with rec.stage("ai_handoff") as st:
147             if not ran or not ai_based:
148                 st.status = "not_run" if not ran else "skipped"

```

```

149         st.details.update({
150             "ai_based": ai_based,
151             "provider": provider if ai_based else None,
152             "model": config.get("ai_model") if ai_based else None,
153             "api_key_present": bool(os.environ.get(key_env)) if ai_based else None,
154             # Chunks the provider refused as over its upload cap (no analysis ran for
155             # those time ranges). Detail name couples to the oversize_clip_skipped rule.
156             "oversize_clips_skipped": int((run_signals or
157         {}).get("oversize_clips_skipped", 0)) if ai_based else None,
158         })
159         if ai_based:
160             st.add_metric("confirmed_segments", len(play_segments))
161
162     def _highlights(rec: "obsreport.RunRecorder", play_segments: List[Dict[str, Any]],
163                   video_meta: Dict[str, Any]) -> None:
164         total = round(sum(float(s.get("duration", 0.0)) for s in play_segments), 1)
165         coverage = None
166         dur = video_meta.get("duration_s")
167         if dur:
168             coverage = round(100.0 * total / dur, 1)
169         notes = []
170         if not play_segments:
171             notes.append("No highlights were produced for this video.")
172         rec.set_highlights(segment_count=len(play_segments), total_highlight_s=total,
173                           coverage_pct=coverage, notes=notes)
174
175     # --- top-level -----
176     def record_run(
177         *,
178         video_id: str,
179         video_path: Optional[str],
180         config: Dict[str, Any],
181         val_results: List[Any],
182         val_context: Optional[Dict[str, Any]],
183         play_segments: List[Dict[str, Any]],
184         candidates: List[Dict[str, Any]],
185         segmenter_requested: Optional[str] = None,
186         segmenter_actual: Optional[str] = None,
187         was_reingest: bool = False,
188         segmentation_ran: bool = True,
189         tool_version: Optional[str] = None,
190         spec: Optional["obsreport.ReportSpec"] = None,
191         run_signals: Optional[Dict[str, Any]] = None,
192         **recorder_kwargs: Any,
193     ) -> "obsreport.RunRecorder":
194         """Assemble a finalized recorder for an indexer run (does NOT write)."""
195         rec = obsreport.RunRecorder(
196             "badminton-highlight-indexer",
197             tool_version=tool_version,
198             video_id=video_id,
199             video_id_kind="file_md5",
200             video_path=video_path,
201             spec=spec if spec is not None else load_spec(),
202             **recorder_kwargs,
203         )
204         vm = video_meta_from_context(val_context, video_path)
205         # If we never probed an fps but segmentation still ran, the segmenter must have
206         # used its internal fallback (30fps / 1280px) ? surface that as the footgun it is,
207         # AND show the assumed numeric value so the report reads "30 (fallback)" not "?
208         # (fallback)".
209         if vm.get("fps_source") == "unknown" and segmentation_ran:
210             vm["fps_source"] = "fallback"
211             if vm.get("fps") is None:

```

```

212         vm["fps"] = 30.0 # matches the runners' fallback (_probe_fps_width -> 30.0
/ 1280)
213     rec.set_video_meta(**vm)
214
215     _validation_stage(rec, val_results)
216     config_default = (config.get("indexing") or {}).get("default_segmenter")
217     _segmentation_stage(rec, play_segments, candidates, segmenter_requested,
218                         segmenter_actual, config_default, was_reingest,
segmentation_ran)
219     _ai_stage(rec, config, segmenter_actual, play_segments, segmentation_ran,
run_signals)
220     _highlights(rec, play_segments, vm)
221
222     rec.finalize()
223     return rec
224

```

39. assistant (2026-06-18T01:34:32.227Z)

No subprocess/file/DB here. Let me look at doctor.py and resolution_fps.py, plus check the yolo_hybrid which uses ffmpeg/cv2 for frame extraction.

40. user (2026-06-18T01:34:33.224Z)

```

1     import sys
2     import os
3     import subprocess
4     import shutil
5
6     # This diagnostics/bootstrap script lives in scripts/ but operates on the repo
7     # root (requirements.txt, config.json). Resolve the root from this file's
8     # location so it works regardless of the caller's cwd.
9     REPO_ROOT = os.path.dirname(os.path.dirname(os.path.abspath(__file__)))
10
11     def check_command(cmd):
12         """Check if a system command is available."""
13         return shutil.which(cmd) is not None
14
15     def run_pip_install():
16         """Install required Python packages."""
17         print("Installing Python packages from requirements.txt...")
18
19         # Check if NVIDIA GPU is present
20         has_gpu = check_command("nvidia-smi")
21
22         requirements_path = os.path.join(REPO_ROOT, "requirements.txt")
23         pip_cmd = [sys.executable, "-m", "pip", "install", "--user", "-r",
requirements_path]
24
25         if has_gpu:
26             print("[INFO] NVIDIA GPU detected. Configuring PyTorch to download CUDA-enabled
wheels.")
27             pip_cmd.extend(["--extra-index-url", "https://download.pytorch.org/whl/cu124"])
28         else:
29             print("[INFO] No NVIDIA GPU detected. Defaulting to PyTorch CPU wheels.")
30             pip_cmd.extend(["--extra-index-url", "https://download.pytorch.org/whl/cpu"])
31
32         try:
33             subprocess.check_call(pip_cmd)
34             print("[SUCCESS] Python packages installed.")
35             return True
36         except subprocess.CalledProcessError as e:
37             print(f"[ERROR] Failed to install Python packages: {e}")
38             return False
39

```

```

40     def init_default_config():
41         """Initialize a default config.json if it doesn't exist.
42
43         Now delegates to the canonical backend.config loader so that defaults
44         stay in sync with the Pydantic models (single source of truth).
45         """
46         try:
47             from backend.config import save_default_config
48         except Exception as e:
49             print(f"[ERROR] Could not import backend.config loader: {e}")
50             print("[INFO] Falling back to no-op for config initialization.")
51             return
52
53         config_path = os.path.join(REPO_ROOT, "config.json")
54         if os.path.exists(config_path):
55             print(f"[INFO] {config_path} already exists. Skipping initialization.")
56             return
57
58         try:
59             saved_path = save_default_config(config_path)
60             print(f"[SUCCESS] config.json initialized with default settings at {saved_path}.")
61         except Exception as e:
62             print(f"[ERROR] Failed to create config.json via loader: {e}")
63
64     def run_diagnostics():
65         """Perform system-wide checks for FFmpeg and PyTorch/CUDA."""
66         print("-" * 50)
67         print("SPORTS HIGHLIGHT INDEXER SETUP DIAGNOSTICS")
68         print("-" * 50)
69
70         # 1. Check Python Version
71         print(f"Python Version: {sys.version_info.major}.{sys.version_info.minor}.{sys.version_info.micro}")
72         if sys.version_info.major < 3 or (sys.version_info.major == 3 and sys.version_info.minor < 8):
73             print("[WARNING] Python 3.8+ is highly recommended!")
74         else:
75             print("[PASS] Python version check passed.")
76
77         print("-" * 50)
78
79         # 2. Check FFmpeg / FFprobe
80         ffmpeg_ok = check_command("ffmpeg")
81         ffprobe_ok = check_command("ffprobe")
82
83         if ffmpeg_ok:
84             print("[PASS] FFmpeg is installed and available in PATH.")
85         else:
86             print("[FAIL] FFmpeg is NOT found in your PATH.")
87
88         if ffprobe_ok:
89             print("[PASS] FFprobe is installed and available in PATH.")
90         else:
91             print("[FAIL] FFprobe is NOT found in your PATH.")
92
93         if not (ffmpeg_ok and ffprobe_ok):
94             print("\n" + "!" * 50)
95             print("CRITICAL: FFmpeg and FFprobe are REQUIRED to run this application.")
96             print("How to install FFmpeg on Windows:")
97             print("  Option 1: Open PowerShell as Administrator and run:")
98             print("      winget install Gyan.FFmpeg")
99             print("  Option 2: Download the build from https://ffmpeg.org/download.html")
100             print("      and add its 'bin' directory to your System PATH environment variable.")

```

```

101         print("!" * 50 + "\n")
102     else:
103         print("[SUCCESS] All essential media command line tools are ready.")
104
105     print("-" * 50)
106
107     # 3. Check ML / GPU (Optional but good)
108     try:
109         import torch
110         cuda_ok = torch.cuda.is_available()
111         if cuda_ok:
112             print(f"[PASS] PyTorch loaded successfully. CUDA (GPU) is AVAILABLE:
{torch.cuda.get_device_name(0)}")
113         else:
114             print("[INFO] PyTorch loaded successfully, but CUDA (GPU) is NOT available.
Falling back to CPU.")
115     except ImportError:
116         print("[INFO] PyTorch is not installed. Relevance checks will run using
lightweight visual heuristics.")
117
118     print("=" * 50)
119     return ffmpeg_ok and ffprobe_ok
120
121 if __name__ == "__main__":
122     init_default_config()
123     media_ok = run_diagnostics()
124     run_pip_install()
125     print("\n[INFO] Setup complete. If diagnostics passed, you can now run the app!")
126

```

41. assistant (2026-06-18T01:34:37.348Z)

Clean. Let me look at the yolo_hybrid segmenter and resolution_fps.py, then the eval splits.py and per_user_regression.py.

42. user (2026-06-18T01:34:38.170Z)

```

1      """Two-stage rally segmenter: torchvision person detection + ByteTrack tracking
2      (Stage 1, local) feeding an AI provider for semantic boundaries (Stage 2).
3
4      Stage-1 detection now lives in `backend/pipeline/detectors/person_detector.py`
5      (`PersonDetector`, a reusable rally-cue producer ? fusion P1) so the SAME person cue
6      can be consumed by the future fusion segmenter; this segmenter just orchestrates
7      chunking, the AI-provider handoff, and DB population around it. Behaviour is unchanged.
8
9      Stage 1 originally used the AGPL-licensed ultralytics YOLO. To keep the hosted
10     service free of copyleft obligations (see docs/MONETIZATION_AUDIT.md and ADR-005),
11     detection uses torchvision's permissively-licensed (BSD) detectors and the
12     association/tracking that YOLO.track() used to bundle in is provided by
13     supervision's MIT-licensed ByteTrack.
14
15     The registry key stays "yolo_hybrid" for back-compat with existing configs and the
16     DB `detector` tags, even though no YOLO is involved anymore.
17     """
18     import os
19     import time
20     import logging
21     import concurrent.futures
22     from typing import List, Dict, Any, Tuple
23
24     from backend.pipeline.segmenters.base import BasePlaySegmenter, segmenter_registry
25     from backend.pipeline.detectors.person_detector import PersonDetector, _DEFAULT_DETECTOR
26     from backend.utils.video import get_video_duration, extract_video_chunk
27     from backend.sports import sport_registry
28     from backend.providers import provider_registry

```

```

29
30     logger = logging.getLogger(__name__)
31
32
33     def _fmt_ts(seconds: float) -> str:
34         """Format seconds as HH:MM:SS for cross-verifying a timestamp against the video."""
35         seconds = max(0, int(seconds))
36         h, rem = divmod(seconds, 3600)
37         m, s = divmod(rem, 60)
38         return f"{h:02d}:{m:02d}:{s:02d}"
39
40
41     class HybridYoloSegmenter(BasePlaySegmenter):
42         def __init__(self, db, config):
43             super().__init__(db, config)
44             # Stage-1 person cue (detection + tracking + motion windows) ? a reusable
producer
45             # (fusion P1). This segmenter orchestrates chunking + AI handoff + DB around it.
46             self.person = PersonDetector(config)
47
48             @property
49             def name(self) -> str:
50                 return "yolo_hybrid"
51
52             @property
53             def description(self) -> str:
54                 return ("Two-stage pipeline: torchvision person detection + ByteTrack for fast "
55                     "candidate filtering, then an AI provider for high-precision semantic
boundaries.")
56
57             def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None, max_frames: int = None) -> List[Dict[str, Any]]:
58                 self.person.lazy_load_model()
59
60                 # Get sport profile config
61                 sport_name = self.config.get("sport", "badminton")
62                 try:
63                     sport_profile = sport_registry.get(sport_name)
64                 except KeyError as e:
65                     logger.error(str(e))
66                     return []
67
68                 # Get AI provider and model config
69                 idx_cfg = self.config.get("indexing", {})
70                 provider_name = self.config.get("ai_provider", idx_cfg.get("ai_provider",
"gemini"))
71                 model_name = self.config.get("ai_model", idx_cfg.get("ai_model",
"gemini-2.5-flash"))
72
73                 # Get appropriate API key based on the provider
74                 api_key_env_var = f"{provider_name.upper()}_API_KEY"
75                 api_key = os.environ.get(api_key_env_var)
76                 if not api_key:
77                     logger.error(
78                         f"{api_key_env_var} environment variable is not set! "
79                         f"To run the {provider_name} segmenter, set your API key. "
80                         f"In PowerShell: $env:{api_key_env_var}=\"your_api_key_here\"")
81                 )
82                 return []
83
84                 try:
85                     provider = provider_registry.get(provider_name, api_key=api_key,
model_name=model_name)
86                 except KeyError as e:
87                     logger.error(str(e))

```



```

88         return []
89
90     logger.info(f"Initializing Hybrid Segmenter (torchvision + ByteTrack) for video:
{video_path}")
91
92     video_duration = get_video_duration(video_path)
93     if video_duration <= 0:
94         logger.error("Invalid video duration.")
95         return []
96
97     chunk_duration = idx_cfg.get("chunk_duration", 300)
98     chunk_overlap = idx_cfg.get("chunk_overlap", 30)
99     velocity_thresh = idx_cfg.get("yolo_velocity_threshold", 0.15) # Fraction of
frame width per second
100     merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
101     frame_skip = idx_cfg.get("yolo_frame_skip", 3)
102     tracker_config = idx_cfg.get("yolo_tracker", "bytetrack.yaml")
103     handoff_padding = idx_cfg.get("ai_handoff_padding",
idx_cfg.get("gemini_handoff_padding", 2.0))
104     ai_max_workers = idx_cfg.get("ai_max_workers", idx_cfg.get("gemini_max_workers",
5))
105     log_candidates = idx_cfg.get("log_yolo_candidates", True)
106     store_candidates = idx_cfg.get("store_yolo_candidates", True)
107
108     # Snapshot of the params that produced these detections ? stored alongside each
109     # candidate so a fine-tuning run can reproduce / correlate against them.
110     detection_params = {
111         "velocity_thresh": velocity_thresh,
112         "merge_gap": merge_gap,
113         "frame_skip": frame_skip,
114         "tracker": tracker_config,
115         "detector_model": idx_cfg.get("detector_model", _DEFAULT_DETECTOR),
116         "detector_score_threshold": idx_cfg.get("detector_score_threshold", 0.5),
117         "min_action_window_duration": idx_cfg.get("min_action_window_duration",
2.0),
118     }
119
120     chunks_dir = os.path.join("output", "chunks_yolo")
121     os.makedirs(chunks_dir, exist_ok=True)
122
123     step = chunk_duration - chunk_overlap
124     chunk_starts = []
125     t = 0.0
126     while t < video_duration:
127         chunk_starts.append(t)
128         t += step
129
130     num_chunks = len(chunk_starts)
131     logger.info(f"Video duration: {video_duration:.1f}s. Splitting into {num_chunks}
chunks.")
132
133     all_candidate_windows = []
134
135     t_start = time.time()
136     prefetch_workers = idx_cfg.get("yolo_prefetch_workers", 2)
137
138     if num_chunks > 1 and prefetch_workers > 0:
139         # --- PIPELINED PROCESSING ---
140         # GPU inference must stay sequential (single GPU), but ffmpeg chunk
141         # extraction is pure I/O. We pre-extract upcoming chunks in background
142         # threads so extraction of chunk N+1 overlaps with inference on chunk N.
143         logger.info(f"Running detection Phase 2 with pipelined extraction
({prefetch_workers} prefetch workers)...")
144
145         extract_executor =

```

```

concurrent.futures.ThreadPoolExecutor(max_workers=prefetch_workers)
146
147     def _extract_chunk(ci: int, cs: float) -> Tuple[int, float, str, bool]:
148         chunk_end = min(cs + chunk_duration, video_duration)
149         actual_dur = chunk_end - cs
150         chunk_path = os.path.join(chunks_dir, f"chunk_{video_id}_{ci}.mp4")
151         success = extract_video_chunk(video_path, cs, actual_dur, chunk_path)
152         return (ci, cs, chunk_path, success)
153
154     # Submit all extractions upfront ? they'll queue and run as workers free up
155     extract_futures = [
156         extract_executor.submit(_extract_chunk, ci, cs)
157         for ci, cs in enumerate(chunk_starts)
158     ]
159
160     # Process results in order: wait for extraction, then run inference on GPU
161     for future in extract_futures:
162         ci, chunk_start, chunk_path, success = future.result()
163         chunk_end = min(chunk_start + chunk_duration, video_duration)
164         logger.info(f"Processing Chunk {ci+1}/{num_chunks} ({chunk_start:.1f}s -
{chunk_end:.1f}s)...")
165
166         if not success:
167             logger.warning(f"Chunk {ci+1} extraction failed. Skipping.")
168             continue
169
170         windows = self.person.extract_action_windows_from_chunk(
171             chunk_path, chunk_start, velocity_thresh, merge_gap,
172             frame_skip=frame_skip, chunk_index=ci
173         )
174         logger.info(f"Found {len(windows)} candidate action windows in chunk
{ci+1}.")
175         all_candidate_windows.extend(windows)
176
177         try:
178             os.remove(chunk_path)
179         except Exception:
180             pass
181
182     extract_executor.shutdown(wait=False)
183 else:
184     # --- SEQUENTIAL PROCESSING (single chunk or prefetch disabled) ---
185     for ci, chunk_start in enumerate(chunk_starts):
186         chunk_end = min(chunk_start + chunk_duration, video_duration)
187         actual_dur = chunk_end - chunk_start
188         chunk_path = os.path.join(chunks_dir, f"chunk_{video_id}_{ci}.mp4")
189
190         logger.info(f"Processing Chunk {ci+1}/{num_chunks} ({chunk_start:.1f}s -
{chunk_end:.1f}s)...")
191         success = extract_video_chunk(video_path, chunk_start, actual_dur,
chunk_path)
192         if not success:
193             continue
194
195         windows = self.person.extract_action_windows_from_chunk(
196             chunk_path, chunk_start, velocity_thresh, merge_gap,
197             frame_skip=frame_skip, chunk_index=ci
198         )
199         logger.info(f"Found {len(windows)} candidate action windows in chunk
{ci+1}.")
200         all_candidate_windows.extend(windows)
201
202         try:
203             os.remove(chunk_path)
204         except Exception:

```

```

205         pass
206
207     # Deduplicate overlapping candidate windows across chunks (chunks overlap by
208     # chunk_overlap, so the same rally can surface in two adjacent chunks).
209     def _merge_stats(a: Dict[str, Any], b: Dict[str, Any]) -> Dict[str, Any]:
210         fa, fb = a["active_frame_count"], b["active_frame_count"]
211         total = fa + fb or 1
212         return {
213             "start": a["start"],
214             "end": max(a["end"], b["end"]),
215             "active_frame_count": fa + fb,
216             "peak_velocity": max(a["peak_velocity"], b["peak_velocity"]),
217             # Frame-weighted mean so longer sub-windows dominate proportionally.
218             "mean_velocity": (a["mean_velocity"] * fa + b["mean_velocity"] * fb) /
total,
219             "track_id_count": max(a["track_id_count"], b["track_id_count"]),
220             "chunk_index": a["chunk_index"],
221         }
222
223     if all_candidate_windows:
224         all_candidate_windows.sort(key=lambda w: w["start"])
225         merged_windows = [all_candidate_windows[0]]
226         for current in all_candidate_windows[1:]:
227             last = merged_windows[-1]
228             if current["start"] <= last["end"]: # Overlap
229                 merged_windows[-1] = _merge_stats(last, current)
230             else:
231                 merged_windows.append(current)
232         all_candidate_windows = merged_windows
233
234     logger.info(f"Phase 2 tracking completed in {time.time() - t_start:.1f}s.")
235     logger.info(f"Total candidate windows: {len(all_candidate_windows)}")
236
237     # Per-rally console log (final, full-video timestamps) for video cross-
verification.
238     if log_candidates:
239         for w in all_candidate_windows:
240             logger.info(
241                 f"[rally] {_fmt_ts(w['start'])}-{_fmt_ts(w['end'])} "
242                 f"({w['end'] - w['start']:.1f}s) | frames={w['active_frame_count']}"
243                 f"peak_v={w['peak_velocity']:.3f} mean_v={w['mean_velocity']:.3f} "
244                 f"tracks={w['track_id_count']}"
245             )
246
247     # Persist raw candidates for the fine-tuning dataset. Map window index ->
candidate_id
248     # so confirmed AI segments can be linked back in Phase 4.
249     candidate_ids = [None] * len(all_candidate_windows)
250     if store_candidates:
251         for i, w in enumerate(all_candidate_windows):
252             stats = {
253                 "peak_velocity": w["peak_velocity"],
254                 "mean_velocity": w["mean_velocity"],
255                 "active_frame_count": w["active_frame_count"],
256                 "track_id_count": w["track_id_count"],
257             }
258             # Use peak velocity (capped at 1.0) as a coarse generic confidence
score.
259             confidence = min(1.0, w["peak_velocity"])
260             candidate_ids[i] = self.db.add_candidate_segment(
261                 video_id, detector="yolo_hybrid",
262                 start_time=w["start"], end_time=w["end"],
263                 chunk_index=w["chunk_index"], confidence=confidence,
264                 stats=stats, detection_params=detection_params,

```

```

265         )
266
267     try:
268         os.rmdir(chunks_dir)
269     except Exception:
270         pass
271
272     # Phase 3: AI Provider Handoff
273     gemini_segments = []
274     gemini_dir = os.path.join("output", f"chunks_{provider_name}_handoff")
275     os.makedirs(gemini_dir, exist_ok=True)
276
277     logger.info(f"Starting Phase 3: AI Provider ({provider_name}) Validation on
{len(all_candidate_windows)} candidate windows...")
278
279     def process_candidate_window(wi: int, window: Dict[str, Any]) -> List[dict]:
280         w_start = max(0, window["start"] - handoff_padding)
281         w_end = min(video_duration, window["end"] + handoff_padding)
282         w_dur = w_end - w_start
283
284         w_path = os.path.join(gemini_dir, f"handoff_{video_id}_{wi}.mp4")
285         # Re-encode short handoff clips to avoid keyframe-boundary drift
286         success = extract_video_chunk(video_path, w_start, w_dur, w_path,
reencode=True)
287         if not success:
288             return []
289
290         prompt = sport_profile.get_prompt(chunk_duration=w_dur)
291         segments, metrics = provider.analyze_video(w_path, prompt,
chunk_duration=w_dur, label=f"Handoff {wi+1}")
292
293         # Coordinate mapping ? shift local timestamps back to full-video time
294         valid_segments = []
295         for segment in segments:
296             try:
297                 segment["start_time"] = float(segment["start_time"]) + w_start
298                 segment["end_time"] = float(segment["end_time"]) + w_start
299                 # Stamp the source candidate so Phase 4 can link
prediction->confirmation.
300                 segment["_candidate_id"] = candidate_ids[wi]
301                 valid_segments.append(segment)
302             except (ValueError, TypeError, KeyError) as e:
303                 logger.warning(f"Dropping segment with unparseable timestamps:
{segment} ? {e}")
304
305         try:
306             os.remove(w_path)
307         except OSError:
308             pass
309
310         return valid_segments
311
312     with concurrent.futures.ThreadPoolExecutor(max_workers=ai_max_workers) as
executor:
313         futures = [executor.submit(process_candidate_window, wi, w) for wi, w in
enumerate(all_candidate_windows)]
314         # Iterate in submission order to preserve temporal ordering
315         for future in futures:
316             gemini_segments.extend(future.result())
317
318     try:
319         os.rmdir(gemini_dir)
320     except OSError:
321         pass
322

```

```

323         logger.info(f"Phase 3 completed. AI confirmed {len(gemini_segments)} semantic
play segments.")
324
325         # Phase 4: DB Population
326         segments_data = []
327         for segment in gemini_segments:
328             try:
329                 start = float(segment["start_time"])
330                 end = float(segment["end_time"])
331                 dur = end - start
332
333                 shot_count = segment.get("shuttle_exchange_count")
334                 if shot_count is None:
335                     shot_count = max(3, int(dur / 0.8))
336                 else:
337                     try:
338                         shot_count = int(shot_count)
339                     except (ValueError, TypeError):
340                         shot_count = max(3, int(dur / 0.8))
341
342                 metadata = {
343                     "shot_count": shot_count,
344                     "shot_count_confidence": segment.get("shot_count_confidence",
"Unknown"),
345                     "detector": f"yolo-hybrid-{model_name}"
346                 }
347
348                 segment_id = self.db.add_play_segment(video_id, start, end, None, None,
metadata)
349                 if segment_id:
350                     # Link back to the candidate that produced this segment. A candidate
351                     # may yield multiple AI segments (one-to-many); confirmed_segment_id
keeps
352                     # the last one ? the link table records "this candidate was
confirmed".
353                     candidate_id = segment.get("_candidate_id")
354                     if candidate_id is not None:
355                         self.db.link_candidate_to_segment(candidate_id, segment_id)
356                     segments_data.append({
357                         "id": segment_id,
358                         "start_time": start,
359                         "end_time": end,
360                         "duration": dur,
361                         "metadata": metadata
362                     })
363             except Exception as e:
364                 logger.warning(f"Skipping invalid segment struct: {e}")
365
366         return segments_data
367
368     # Register the segmenter
369     segmenter_registry.register(HybridYoloSegmenter)
370

```

43. assistant (2026-06-18T01:34:41.208Z)

This is well-handled. Let me look at the eval splits.py, regression.py, per_user_regression.py, and the export_wasb_segments / gt_loader which open files. Let me also check resolution_fps.py.

44. user (2026-06-18T01:34:42.103Z)

```

1     from backend.pipeline.validators.base import VideoValidator, ValidationResult
2     from typing import Dict, Any
3
4     class ResolutionFPSValidator(VideoValidator):

```

```

5         @property
6         def name(self) -> str:
7             return "resolution_fps_check"
8
9         @property
10        def description(self) -> str:
11            return "Checks if the video meets minimum resolution (e.g. 720p) and frame rate
(e.g. 30fps) requirements."
12
13        def validate(self, video_path: str, config: Dict[str, Any], context: Dict[str, Any])
-> ValidationResult:
14            meta = context.get("ffprobe_meta")
15            if not meta or not meta.get("streams"):
16                return ValidationResult(
17                    validator_name=self.name,
18                    passed=False,
19                    message="Video metadata is missing. Run format_check validator first."
20                )
21
22            stream = meta["streams"][0]
23            width = int(stream.get("width", 0))
24            height = int(stream.get("height", 0))
25
26            # Parse frame rate (e.g. "30/1" or "30000/1001")
27            fps_str = stream.get("r_frame_rate", "0/1")
28            try:
29                num, den = map(int, fps_str.split("/"))
30                fps = num / den if den != 0 else 0
31            except ValueError:
32                fps = 0.0
33
34            details = {
35                "width": width,
36                "height": height,
37                "fps": round(fps, 2)
38            }
39
40            # Check bounds from config
41            val_cfg = config.get("validation", {})
42            min_w = val_cfg.get("min_resolution_width", 1280)
43            min_h = val_cfg.get("min_resolution_height", 720)
44            min_fps = val_cfg.get("min_fps", 29.9)
45
46            if width < min_w or height < min_h:
47                return ValidationResult(
48                    validator_name=self.name,
49                    passed=False,
50                    message=f"Low resolution: Found {width}x{height}, but minimum is
{min_w}x{min_h}.",
51                    details=details
52                )
53
54            if fps < min_fps:
55                return ValidationResult(
56                    validator_name=self.name,
57                    passed=False,
58                    message=f"Low frame rate: Found {round(fps, 2)} FPS, but minimum is
{min_fps} FPS.",
59                    details=details
60                )
61
62            return ValidationResult(
63                validator_name=self.name,
64                passed=True,
65                message="Video resolution and frame rate meet minimum standards.",

```

```
66         details=details
67     )
68
```

45. user (2026-06-18T01:34:42.754Z)

```
1     """Train/eval split guard (pure-logic core, in-container).
2
3     Spec from BACKLOG.md "Train/eval separation guardrails" / DEFERRED #13
4     and plan M0:
5     - Split unit = match (never clip / file MD5).
6     - load_split(manifest) -> (train_ids, eval_ids) keyed by match id.
7     - assert_disjoint(train_ids, eval_ids) raises on any overlap.
8     - This would have caught train/eval leakage in training/eval paths.
9     - Cross-repo collector manifest `split` field wiring + wiring into
10       distill_local / run_regression deferred to later PR.
11
12     Usage (once wired):
13         train, eval_ = load_split(manifest)
14         assert_disjoint(train, eval_)
15     """
16     from __future__ import annotations
17
18     import json
19     import os
20     from typing import Any, Dict, Iterable, List, Set, Tuple, Union, cast
21
22
23     def load_split(manifest: Union[str, Iterable[Dict[str, Any]], os.PathLike[str]]) ->
24     Tuple[Set[str], Set[str]]:
25         """Load (train_ids, eval_ids) sets keyed by match id.
26
27         manifest: path to JSON (str/bytes/PathLike) or iterable of dicts. Each entry should
28         have 'match_id'
29         (or 'id' fallback; must be non-empty str for real match key; name/video_id
30         deliberately not used as they are weak/MD5).
31
32         Raises ValueError on:
33         - non-list manifest after load
34         - no real match-level key (empty/falsy after explicit check)
35         - split label not exactly "train" or "eval"
36
37         The collector 'split' field and cross-repo wiring is deferred; this supports
38         synthetic or pre-split manifests for the pure-logic guard and tests.
39         """
40         if isinstance(manifest, (str, bytes, os.PathLike)):
41             with open(manifest, "r", encoding="utf-8-sig") as f:
42                 raw = json.load(f)
43                 if not isinstance(raw, list):
44                     raise ValueError(f"manifest must be a list of entries, got {type(raw)}")
45                 data = cast(List[Dict[str, Any]], raw)
46         else:
47             if isinstance(manifest, dict):
48                 raise ValueError(f"manifest must be a list of entries, got
49 {type(manifest)}")
50             data = list(manifest)
51             if not isinstance(data, list):
52                 raise ValueError(f"manifest must be a list of entries, got {type(data)}")
53
54         train_ids: Set[str] = set()
55         eval_ids: Set[str] = set()
56
57         for entry in data:
58             if not isinstance(entry, dict):
59                 continue
```

```

56         # Explicit not-None + non-empty for real match key; no silent fallback to
name/video_id (weak/MD5)
57         mid = entry.get("match_id")
58         if mid is None or str(mid).strip() == "":
59             mid = entry.get("id")
60         if mid is None or str(mid).strip() == "":
61             raise ValueError(f"no real match-level key (match_id or id) in entry:
{entry}")
62         mid = str(mid).strip()
63         sp = str(entry.get("split", "")).strip().lower()
64         if sp not in ("train", "eval"):
65             raise ValueError(f"split label must be exactly 'train' or 'eval', got {sp!r}
in entry: {entry}")
66         if sp == "eval":
67             eval_ids.add(mid)
68         else:
69             train_ids.add(mid)
70
71     return train_ids, eval_ids
72
73
74     def assert_disjoint(train_ids: Iterable[str], eval_ids: Iterable[str]) -> None:
75         """Raise if any match-id overlap between train and eval sets.
76
77         This would have caught train/eval leakage (e.g. a video used for both
78         training a segmenter and for regression/eval baselines).
79         """
80         t = set(str(x) for x in train_ids)
81         e = set(str(x) for x in eval_ids)
82         overlap = t & e
83         if overlap:
84             raise ValueError(
85                 f"Train/eval leakage detected on match ids: {sorted(overlap)}. "
86                 "This would have caught train/eval leakage."
87             )
88

```

46. assistant (2026-06-18T01:34:47.148Z)

These are clean. Let me look at the remaining eval files that touch ffmpeg/files: export_wasb_segments.py, gt_loader.py, regression.py, per_user_regression.py, serve_contrast.py, served_gate_a.py, classifier.py, distill_local.py, calibrate_local.py, golden_fixtures.py, fusion_golden.py, rally_seq_proto.py, rally_seg_eval.py, annotations.py, experiment.py.

Let me focus on the ones that do subprocess/file work. Let me grep for `subprocess` and `open` specifically in the eval dir (excluding the excluded files).

47. user (2026-06-18T01:34:49.754Z)

```

.claude\worktrees\hunt-base\backend\eval\ablation.py:409:         with open(path,
encoding="utf-8") as f:
.claude\worktrees\hunt-base\backend\eval\ablation.py:410:             d = json.load(f)
.claude\worktrees\hunt-base\backend\eval\ablation.py:623:         with open(args.json, "w",
encoding="utf-8") as f:
.claude\worktrees\hunt-base\backend\eval\annotations.py:62:         with open(csv_path, "w",
newline="") as f:
.claude\worktrees\hunt-base\backend\eval\annotations.py:63:             writer = csv.writer(f)
.claude\worktrees\hunt-base\backend\eval\calibrate_local.py:120:         with open(args.manifest,
encoding="utf-8") as f:
.claude\worktrees\hunt-base\backend\eval\calibrate_local.py:121:             specs = json.load(f)
.claude\worktrees\hunt-base\backend\eval\classifier.py:108:         with open(path, "w") as f:
.claude\worktrees\hunt-base\backend\eval\classifier.py:113:         with open(path) as f:
.claude\worktrees\hunt-base\backend\eval\classifier.py:114:             return
cls.from_dict(json.load(f))
.claude\worktrees\hunt-base\backend\eval\distill_local.py:159:         with open(args.manifest,

```



```

encoding="utf-8") as f:
.claude\worktrees\hunt-base\backend\eval\distill_local.py:160:         specs = json.load(f)
.claude\worktrees\hunt-base\backend\eval\experiment.py:531:         with open(path,
encoding="utf-8") as f:
.claude\worktrees\hunt-base\backend\eval\experiment.py:532:         entries = json.load(f)
.claude\worktrees\hunt-base\backend\eval\experiment.py:535:         with open(path, "w",
encoding="utf-8") as f:
.claude\worktrees\hunt-base\backend\eval\fusion_audio.py:30:         with open(manifest_path,
encoding="utf-8") as f:
.claude\worktrees\hunt-base\backend\eval\fusion_audio.py:31:         specs = json.load(f)
.claude\worktrees\hunt-base\backend\eval\fusion_audio.py:41:         with open(jp,
encoding="utf-8") as f:
.claude\worktrees\hunt-base\backend\eval\fusion_audio.py:42:         data = json.load(f)
.claude\worktrees\hunt-base\backend\eval\fusion_audio.py:45:         with open(jp, "w",
encoding="utf-8") as f:
.claude\worktrees\hunt-base\backend\eval\fusion_audio.py:57:         with open(path,
encoding="utf-8") as f:
.claude\worktrees\hunt-base\backend\eval\fusion_audio.py:58:         d = json.load(f)
.claude\worktrees\hunt-base\backend\eval\fusion_golden.py:131:         with open(manifest_path,
encoding="utf-8") as f:
.claude\worktrees\hunt-base\backend\eval\fusion_golden.py:132:         specs = json.load(f)
.claude\worktrees\hunt-base\backend\eval\fusion_golden.py:168:         with open(out, "w",
encoding="utf-8") as f:
.claude\worktrees\hunt-base\backend\eval\gemini_refine.py:12:calibrate_wasb.make_predict_fn,
export_wasb_segments.write_segments_csv.
.claude\worktrees\hunt-base\backend\eval\fusion_compare.py:40:         with open(path,
encoding="utf-8") as f:
.claude\worktrees\hunt-base\backend\eval\fusion_compare.py:41:         d = json.load(f)
.claude\worktrees\hunt-base\backend\eval\export_wasb_segments.py:77:         with open(path, "w",
newline="", encoding="utf-8") as f:
.claude\worktrees\hunt-base\backend\eval\export_wasb_segments.py:78:         w =
csv.DictWriter(f, fieldnames=SEG_FIELDS)
.claude\worktrees\hunt-base\backend\eval\export_wasb_segments.py:127:         with open(path, "w",
newline="", encoding="utf-8") as f:
.claude\worktrees\hunt-base\backend\eval\export_wasb_segments.py:128:         w =
csv.DictWriter(f, fieldnames=COMP_FIELDS)
.claude\worktrees\hunt-base\backend\eval\golden_fixtures.py:443:         if not traj_csv.is_file() or
not labels_csv.is_file():
.claude\worktrees\hunt-base\backend\eval\golden_fixtures.py:960:         with open(path,
encoding="utf-8") as f:
.claude\worktrees\hunt-base\backend\eval\golden_fixtures.py:961:         return json.load(f)
.claude\worktrees\hunt-base\backend\eval\gt_loader.py:60:         with open(csv_path, newline="",
encoding="utf-8-sig") as f:
.claude\worktrees\hunt-base\backend\eval\gt_loader.py:61:         for lineno, row in
enumerate(csv.reader(f), start=1):
.claude\worktrees\hunt-base\backend\eval\per_user_regression.py:141:         with open(path,
encoding="utf-8") as f:
.claude\worktrees\hunt-base\backend\eval\per_user_regression.py:142:         return json.load(f)
.claude\worktrees\hunt-base\backend\eval\per_user_regression.py:156:         with open(path, "w",
encoding="utf-8") as f:
.claude\worktrees\hunt-base\backend\eval\rally_seg_eval.py:105:         with open(manifest_path) as
fh:
.claude\worktrees\hunt-base\backend\eval\rally_seg_eval.py:106:         manifest = json.load(fh)
.claude\worktrees\hunt-base\backend\eval\rally_seg_eval.py:112:         with open(feats) as
fh:
.claude\worktrees\hunt-base\backend\eval\rally_seg_eval.py:113:         gts = [(float(s),
float(e)) for s, e in (json.load(fh).get("gts") or [])]
.claude\worktrees\hunt-base\backend\eval\rally_seg_eval.py:374:         with open(args.json_out,
"w", encoding="utf-8") as fh:
.claude\worktrees\hunt-base\backend\eval\served_gate_a.py:182:         with open(manifest_path,
encoding="utf-8") as f:
.claude\worktrees\hunt-base\backend\eval\served_gate_a.py:183:         specs = json.load(f)
.claude\worktrees\hunt-base\backend\eval\regression.py:72:         with open(path) as f:
.claude\worktrees\hunt-base\backend\eval\regression.py:73:         return json.load(f)
.claude\worktrees\hunt-base\backend\eval\regression.py:97:         with open(path, "w") as f:

```

```
.claude\worktrees\hunt-base\backend\eval\serve_contrast.py:155:    with open(args.manifest,
encoding="utf-8") as f:
.claude\worktrees\hunt-base\backend\eval\serve_contrast.py:156:        specs = json.load(f)
.claude\worktrees\hunt-base\backend\eval\serve_contrast.py:161:        with open(args.json, "w",
encoding="utf-8") as f:
.claude\worktrees\hunt-base\backend\eval\splits.py:38:        with open(manifest, "r",
encoding="utf-8-sig") as f:
.claude\worktrees\hunt-base\backend\eval\splits.py:39:        raw = json.load(f)
.claude\worktrees\hunt-base\backend\eval\rally_seq_proto.py:156:    with open(args.manifest,
encoding="utf-8") as f:
.claude\worktrees\hunt-base\backend\eval\rally_seq_proto.py:157:        run(json.load(f))
```

48. assistant (2026-06-18T01:34:52.789Z)

These all use context managers. Let me look at the export_wasb_segments.py which runs ffmpeg/ffprobe, and gt_loader.py and gemini_refine.py.

49. user (2026-06-18T01:34:53.587Z)

```
1      """Export tuned WASB rally segments for manual eyeball verification.
2
3      After calibration picks a tuned threshold config, this turns a cached WASB
4      trajectory into the actual rally [start,end] segments that config produces, so the
5      user can answer "are the rallies actually better?" ? by scrubbing the video to each
6      segment, by diffing tuned-vs-golden quantitatively, or by cutting clips.
7
8      Reuses (no new windowing/metrics/cutting logic):
9      - calibrate_wasb.make_predict_fn -> trajectory -> segments for a given config
10     - eval.metrics.match_intervals/score -> tuned-vs-golden IoU comparison
11     - tools.rally_slicer.slice_rallies -> optional clip cutting (the exported CSV is
12       written in the golden/slicer schema, so it feeds straight back in)
13
14     Run (uses the tuned config from the first calibration by default):
15     python -m backend.eval.export_wasb_segments --trajectory
~/clips/sample_long_traj.csv \
16         --labels "Annotation Setup/golden_labels_badminton.csv" --fps 59.94 --frame-
width 1920
17         # add --video <mp4> --slice to also cut one padded clip per tuned rally
18     """
19     from __future__ import annotations
20
21     import csv
22     import os
23     import os.path as osp
24     import argparse
25     from typing import List, Tuple, Dict, Optional
26
27     from backend.eval.calibrate_wasb import make_predict_fn, load_golden_gts, BASELINE
28     from backend.eval.metrics import match_intervals, score
29     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
30     from backend.pipeline.detectors import rally_gate
31
32     Interval = Tuple[float, float]
33     Config = Tuple[float, float, float]
34
35     # The tuned config from the first per-video calibration (velocity, merge_gap, min_dur);
36     # F1 0.588 (baseline) -> 0.746. Override on the CLI to export a different operating
point.
37     TUNED: Config = (0.05, 1.0, 1.0)
38
39     SEG_FIELDS = ["rally_number", "start_time", "end_time", "duration", "start_mmss",
"end_mmss"]
40     COMP_FIELDS = ["type", "ref", "start_time", "end_time", "mmss", "status",
41                   "iou", "pred_start", "pred_end", "start_err_s", "end_err_s"]
42
```

```

43
44 def seconds_to_mmss(t: float) -> str:
45     """Human-scrubbable timestamp, e.g. 106.031 -> '1:46.031'."""
46     t = max(0.0, float(t))
47     m = int(t // 60)
48     return f"{m}:{t - m * 60:06.3f}"
49
50
51 def segments_to_rows(segments: List[Interval]) -> List[Dict[str, str]]:
52     """Number segments 1..N in time order; emit the golden/slicer CSV schema."""
53     rows = []
54     for i, (s, e) in enumerate(segments, start=1):
55         rows.append({
56             "rally_number": str(i),
57             "start_time": f"{s:.3f}",
58             "end_time": f"{e:.3f}",
59             "duration": f"{e - s:.3f}",
60             "start_mmss": seconds_to_mmss(s),
61             "end_mmss": seconds_to_mmss(e),
62         })
63     return rows
64
65
66 def _ensure_parent(path: str) -> None:
67     os.makedirs(osp.dirname(osp.abspath(path)) or ".", exist_ok=True)
68
69
70 def write_segments_csv(path: str, segments: List[Interval]) -> str:
71     """Write segments in the golden schema (rally_number/start_time/end_time + mm:ss).
72
73     The schema is what tools.rally_slicer.load_rally_labels consumes, so the file
74     doubles as input to clip cutting.
75     """
76     _ensure_parent(path)
77     with open(path, "w", newline="", encoding="utf-8") as f:
78         w = csv.DictWriter(f, fieldnames=SEG_FIELDS)
79         w.writeheader()
80         w.writerows(segments_to_rows(segments))
81     return path
82
83
84 def build_comparison(preds: List[Interval], gts: List[Interval],
85                     iou_threshold: float = 0.5) -> dict:
86     """Golden-centric comparison: per golden rally, did a tuned segment match (IoU),
87     plus the boundary errors and any extra (false-positive) tuned segments."""
88     mr = match_intervals(preds, gts, iou_threshold)
89     sc = score(preds, gts, iou_threshold, match_result=mr)
90     by_gt = {m.gt_index: m for m in mr.matches}
91
92     rows: List[Dict[str, str]] = []
93     for gi, (gs, ge) in enumerate(gts, start=0):
94         m = by_gt.get(gi)
95         if m is not None:
96             ps, pe = preds[m.pred_index]
97             rows.append({
98                 "type": "golden", "ref": str(gi + 1),
99                 "start_time": f"{gs:.3f}", "end_time": f"{ge:.3f}",
100                 "mmss": f"{seconds_to_mmss(gs)}-{seconds_to_mmss(ge)}",
101                 "status": "matched", "iou": f"{m.iou:.3f}",
102                 "pred_start": f"{ps:.3f}", "pred_end": f"{pe:.3f}",
103                 "start_err_s": f"{ps - gs:+.3f}", "end_err_s": f"{pe - ge:+.3f}",
104             })
105         else:
106             rows.append({
107                 "type": "golden", "ref": str(gi + 1),

```

```

108         "start_time": f"{gs:.3f}", "end_time": f"{ge:.3f}",
109         "mmss": f"{seconds_to_mmss(gs)}-{seconds_to_mmss(ge)}",
110         "status": "MISS", "iou": "0.000",
111         "pred_start": "", "pred_end": "", "start_err_s": "", "end_err_s": "",
112     })
113     for pi in mr.unmatched_pred_indices: # extra tuned segments = false
positives
114         ps, pe = preds[pi]
115         rows.append({
116             "type": "pred_extra", "ref": str(pi + 1),
117             "start_time": f"{ps:.3f}", "end_time": f"{pe:.3f}",
118             "mmss": f"{seconds_to_mmss(ps)}-{seconds_to_mmss(pe)}",
119             "status": "false_positive", "iou": "",
120             "pred_start": "", "pred_end": "", "start_err_s": "", "end_err_s": "",
121         })
122     return {"score": sc, "rows": rows}
123
124
125     def write_comparison_csv(path: str, comparison: dict) -> str:
126         _ensure_parent(path)
127         with open(path, "w", newline="", encoding="utf-8") as f:
128             w = csv.DictWriter(f, fieldnames=COMP_FIELDS)
129             w.writeheader()
130             w.writerows(comparison["rows"])
131         return path
132
133
134     def _with_suffix(path: str, suffix: str) -> str:
135         root, ext = osp.splitext(path)
136         return root + suffix + ext
137
138
139     def run(trajecory_csv: str, fps: float, frame_width: float, cfg: Config = TUNED,
140            out_csv: str = "output/wasb_tuned_segments.csv", labels_csv: Optional[str] =
None,
141            iou: float = 0.5, baseline: bool = False,
142            comparison_out: Optional[str] = None, min_crossings: int = 0) -> dict:
143         predict_fn = make_predict_fn(trajecory_csv, fps, frame_width)
144         # Gate A: drop between-point false fires (single service-feed tosses) by requiring
145         # the shuttle to cross the net >= min_crossings times within the window.
146         points = TrackNetRunner.parse_trajectory_csv(trajecory_csv) if min_crossings > 0
else None
147
148         def _gate(segs: List[Interval], label: str) -> List[Interval]:
149             if not points:
150                 return segs
151             kept = rally_gate.apply_gate(points, segs, fps, min_crossings=min_crossings)
152             print(f"[gate] {label}: {len(segs)} -> {len(kept)} segments ")
153             f"(removed {len(segs) - len(kept)} with <{min_crossings} net-crossings)")
154             return kept
155
156         tuned_segs = _gate(predict_fn(cfg), "tuned")
157         write_segments_csv(out_csv, tuned_segs)
158         total = sum(e - s for s, e in tuned_segs)
159         print("=== WASB tuned-segment export ===")
160         print(f"tuned cfg {cfg}: {len(tuned_segs)} rally segments, {total:.1f}s of play ->
{out_csv}")
161         result: dict = {"tuned_cfg": cfg, "num_tuned": len(tuned_segs), "out_csv": out_csv}
162
163         if baseline:
164             base_segs = _gate(predict_fn(BASELINE), "baseline")
165             base_out = _with_suffix(out_csv, "_baseline")
166             write_segments_csv(base_out, base_segs)
167             print(f"baseline cfg {BASELINE}: {len(base_segs)} segments -> {base_out}")
168             result["num_baseline"] = len(base_segs)

```

```

169         result["baseline_csv"] = base_out
170
171     if labels_csv:
172         gts = load_golden_gts(labels_csv)
173         if not gts:
174             raise SystemExit(f"no usable golden rallies in {labels_csv}")
175         comp = build_comparison(tuned_segs, gts, iou)
176         sc = comp["score"]
177         comp_out = comparison_out or _with_suffix(out_csv, "_vs_golden")
178         write_comparison_csv(comp_out, comp)
179         print(f"\n--- tuned vs golden ({len(gts)} rallies, IoU>={iou}) ---")
180         print(f"matched (TP) {sc.true_positives} | misses (FN) {sc.false_negatives} | "
181               f"extra (FP) {sc.false_positives}")
182         print(f"precision {sc.precision:.3f} | recall {sc.recall:.3f} | f1 {sc.f1:.3f} | "
183               f"meanIoU {sc.mean_iou:.3f}")
184         print(f"mean boundary error: start {sc.mean_start_error:.2f}s | end "
185               f"{sc.mean_end_error:.2f}s")
186         print(f"per-rally detail -> {comp_out}")
187         result["comparison"] = sc.as_dict()
188         result["comparison_csv"] = comp_out
189     return result
190
191 def _maybe_slice(video: str, segments_csv: str, clips_dir: str) -> None:
192     """Cut one padded clip per exported rally (reuses the rally slicer)."""
193     from backend.tools.rally_slicer import slice_rallies
194     written = slice_rallies(video, segments_csv, ["all"], clips_dir)
195     print(f"\ncut {len(written)} clip(s) -> {clips_dir}")
196
197
198 def main() -> None:
199     ap = argparse.ArgumentParser(description="Export tuned WASB rally segments for "
200     eyeballing.")
201     ap.add_argument("--trajectory", required=True, help="WASB trajectory CSV "
202     (Frame,Visibility,X,Y)")
203     ap.add_argument("--fps", type=float, default=59.94)
204     ap.add_argument("--frame-width", type=float, default=1920.0)
205     ap.add_argument("--velocity-thresh", type=float, default=TUNED[0])
206     ap.add_argument("--merge-gap", type=float, default=TUNED[1])
207     ap.add_argument("--min-duration", type=float, default=TUNED[2])
208     ap.add_argument("--labels", default=None, help="golden labels CSV -> tuned-vs-golden "
209     comparison")
210     ap.add_argument("--iou", type=float, default=0.5)
211     ap.add_argument("--out", default="output/wasb_tuned_segments.csv")
212     ap.add_argument("--comparison-out", default=None)
213     ap.add_argument("--baseline", action="store_true", help="also export baseline-config "
214     segments")
215     ap.add_argument("--min-crossings", type=int, default=0,
216     help="gate A: drop windows whose shuttle crosses the net < N times "
217     "(kills single-toss service-feed false fires; 0=off, 2 "
218     recommended)")
219     ap.add_argument("--video", default=None, help="source video (with --slice) to cut "
220     clips from")
221     ap.add_argument("--slice", action="store_true", help="cut one padded clip per tuned "
222     rally")
223     ap.add_argument("--clips-dir", default="output/wasb_tuned_clips")
224     args = ap.parse_args()
225
226     cfg = (args.velocity_thresh, args.merge_gap, args.min_duration)
227     res = run(args.trajectory, args.fps, args.frame_width, cfg=cfg, out_csv=args.out,
228     labels_csv=args.labels, iou=args.iou, baseline=args.baseline,
229     comparison_out=args.comparison_out, min_crossings=args.min_crossings)
230     if args.slice:
231         if not args.video:

```

```

225         raise SystemExit("--slice requires --video")
226         _maybe_slice(args.video, res["out_csv"], args.clips_dir)
227
228
229     if __name__ == "__main__":
230         main()
231

```

50. assistant (2026-06-18T01:34:56.396Z)

Clean. Let me look at gt_loader.py and golden_fixtures.py (which run ffmpeg subprocess). Let me check golden_fixtures.py for subprocess use.

51. user (2026-06-18T01:34:58.519Z)

No matches found

52. assistant (2026-06-18T01:35:02.019Z)

No subprocess in golden_fixtures. Let me read gt_loader.py and check the gemini_refine.py for subprocess.

53. user (2026-06-18T01:35:02.279Z)

```

1     """Canonical ground-truth interval loader ? THE one GT/golden-CSV reader (ADR-008
prereq).
2
3     The repo historically carried two incompatible loaders:
4     - ``annotations.load_annotations``: positional ``start,end`` (the collector's
5       ``curate export`` format), raises on bad rows;
6     - ``calibrate_wasb.load_golden_gts``: DictReader requiring ``start_time/end_time``
7       (the rally-annotator golden format), silently skipping bad rows ? fed 7 modules.
8
9     A file in one convention fed to the other loader yielded zero/garbage intervals with no
10    error. This module accepts BOTH conventions (header-mapped, else positional), so every
11    quantity the ship-gate depends on (gt_hash, LOVO scores, the future consent fingerprint)
12    flows through one reader. Both legacy entry points now delegate here.
13
14    Accepted shapes (one rally per row; blank lines and ``#`` comments ignored):
15    - header with ``start``/``end`` OR ``start_time``/``end_time`` columns (any other
16      columns ? rally_number, ending_reason, sport, ? ? are ignored);
17    - no header: columns 0,1 positionally.
18    Timestamps: decimal seconds or ``MM:SS`` / ``HH:MM:SS``
19    (``annotations.parse_timestamp``).
20    """
21    import csv
22    import logging
23    from typing import List, Optional, Tuple
24
25    from backend.eval.annotations import parse_timestamp
26
27    logger = logging.getLogger(__name__)
28
29    Interval = Tuple[float, float]
30
31    _START_NAMES = ("start", "start_time")
32    _END_NAMES = ("end", "end_time")
33
34    def _header_columns(row: List[str]) -> Optional[Tuple[int, int]]:
35        """If `row` is a header naming start/end columns, return their indices, else
None."""
36        cells = [(c or "").strip().lower() for c in row]
37        start_idx = next((i for i, c in enumerate(cells) if c in _START_NAMES), None)

```

```

38     end_idx = next((i for i, c in enumerate(cells) if c in _END_NAMES), None)
39     if start_idx is not None and end_idx is not None:
40         return start_idx, end_idx
41     return None
42
43
44 def load_gt_intervals(csv_path: str, *, strict: bool = True) -> List[Interval]:
45     """Load GT rally intervals from either golden-CSV convention.
46
47     strict=True (corpus/gate use): raise ValueError with file:line on any malformed row
48     (bad timestamp, end <= start, negative start) so labeling mistakes fail loudly.
49     strict=False (legacy tuning-driver behavior): skip bad rows, but LOG how many were
50     skipped ? a silently shrinking GT set flatters precision.
51
52     Returns intervals sorted by start; overlapping GT intervals are logged as a warning
53     (they break one-to-one matching and usually indicate a labeling error).
54     """
55     intervals: List[Interval] = []
56     skipped = 0
57     cols: Optional[Tuple[int, int]] = None # (start_idx, end_idx); None until detected
58     saw_header = False
59
60     with open(csv_path, newline="", encoding="utf-8-sig") as f:
61         for lineno, row in enumerate(csv.reader(f), start=1):
62             if not row or not (row[0] or "").strip() or row[0].strip().startswith("#"):
63                 continue
64             if cols is None and not saw_header:
65                 header = _header_columns(row)
66                 if header is not None:
67                     cols = header
68                     saw_header = True
69                     continue # consume the header row
70             cols = (0, 1) # headerless: positional start,end
71             assert cols is not None # every branch above assigns it
72             si, ei = cols
73             try:
74                 if len(row) <= max(si, ei):
75                     raise ValueError(f"expected at least {max(si, ei) + 1} columns, got
{row!r}")
76                 start = parse_timestamp(row[si])
77                 end = parse_timestamp(row[ei])
78                 if start < 0:
79                     raise ValueError(f"negative start ({start})")
80                 if start >= end:
81                     raise ValueError(f"start ({start}) >= end ({end})")
82             except ValueError as e:
83                 if strict:
84                     raise ValueError(f"{csv_path}:{lineno}: {e}") from e
85                 skipped += 1
86                 continue
87             intervals.append((start, end))
88
89     if skipped:
90         logger.warning("%s: skipped %d malformed GT row(s) (strict=False)", csv_path,
skipped)
91
92     intervals.sort(key=lambda iv: iv[0])
93     for (s1, e1), (s2, e2) in zip(intervals, intervals[1:]):
94         if s2 < e1:
95             logger.warning("%s: overlapping GT intervals (%.3f-%.3f vs %.3f-%.3f) ? "
96                             "labeling error? Overlaps break one-to-one matching.",
97                             csv_path, s1, e1, s2, e2)
98     return intervals
99

```

54. assistant (2026-06-18T01:35:07.476Z)

Clean. Now let me check `gemini\_refine.py`, `serve\_contrast.py`, `served\_gate\_a.py`, and `calibrate\_local.py`/`distill\_local.py` for subprocess use. These run actual WASB inference which could shell out. Let me grep across the eval files for subprocess/Popen and cv2.VideoCapture not yet covered.

55. user (2026-06-18T01:35:10.062Z)

```
.claude\worktrees\hunt-base\backend\main.py:436:         os.remove(st["tmp_path"])
.claude\worktrees\hunt-base\backend\main.py:650:     cap = cv2.VideoCapture(video_path)
.claude\worktrees\hunt-base\backend\main.py:680:     cap.release()
.claude\worktrees\hunt-base\backend\main.py:723:         import subprocess
.claude\worktrees\hunt-base\backend\main.py:729:         subprocess.run([sys.executable,
doctor_path])
.claude\worktrees\hunt-base\backend\pipeline\audio\hit_detector.py:6:strength) hit candidates.
Pure numpy + stdlib `wave` (NO scipy); ffmpeg (subprocess)
.claude\worktrees\hunt-base\backend\pipeline\audio\hit_detector.py:14:import tempfile
.claude\worktrees\hunt-base\backend\pipeline\audio\hit_detector.py:15:import subprocess
.claude\worktrees\hunt-base\backend\pipeline\audio\hit_detector.py:32:         fd, out_wav =
tempfile.mkstemp(suffix=".wav")
.claude\worktrees\hunt-base\backend\pipeline\audio\hit_detector.py:33:         os.close(fd)
.claude\worktrees\hunt-base\backend\pipeline\audio\hit_detector.py:36:         subprocess.run(cmd,
check=True, stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL)
.claude\worktrees\hunt-base\backend\pipeline\audio\hit_detector.py:42:         with wave.open(path,
"rb") as w:
.claude\worktrees\hunt-base\backend\pipeline\audio\hit_detector.py:144:
os.remove(wav)
.claude\worktrees\hunt-base\backend\pipeline\detectors\base.py:71:import subprocess
.claude\worktrees\hunt-base\backend\pipeline\detectors\base.py:106:     def _wsl_bash(self,
inner_cmd: str) -> subprocess.CompletedProcess:
.claude\worktrees\hunt-base\backend\pipeline\detectors\base.py:110:         return
subprocess.run(argv, capture_output=True, text=True,
.claude\worktrees\hunt-base\backend\pipeline\detectors\base.py:122:         except
(subprocess.TimeoutExpired, OSError):
.claude\worktrees\hunt-base\backend\pipeline\detectors\base.py:137:         except
(subprocess.TimeoutExpired, OSError) as e:
.claude\worktrees\hunt-base\backend\pipeline\detectors\base.py:153:         except
(subprocess.TimeoutExpired, OSError):
.claude\worktrees\hunt-base\backend\tools\rally_slicer.py:23:import subprocess
.claude\worktrees\hunt-base\backend\tools\rally_slicer.py:108:         out = subprocess.run(
.claude\worktrees\hunt-base\backend\tools\rally_slicer.py:114:         except
(subprocess.SubprocessError, OSError, ValueError):
.claude\worktrees\hunt-base\backend\tools\rally_slicer.py:147:         subprocess.run(cmd,
capture_output=True, check=True)
.claude\worktrees\hunt-base\backend\tools\rally_slicer.py:148:         except
subprocess.CalledProcessError as e:
.claude\worktrees\hunt-base\backend\utils\video.py:1:import subprocess
.claude\worktrees\hunt-base\backend\utils\video.py:11:         duration_output =
subprocess.check_output([
.claude\worktrees\hunt-base\backend\utils\video.py:47:         subprocess.run(cmd, check=True,
stdout=subprocess.DEVNULL, stderr=subprocess.DEVNULL)
.claude\worktrees\hunt-base\backend\pipeline\detectors\tracknet_runner.py:1:"""TrackNet shuttle-
trajectory runner (WSL2 subprocess adapter).
.claude\worktrees\hunt-base\backend\pipeline\detectors\tracknet_runner.py:21:import subprocess
.claude\worktrees\hunt-base\backend\pipeline\detectors\tracknet_runner.py:111:         except
subprocess.TimeoutExpired:
.claude\worktrees\hunt-base\backend\pipeline\detectors\tracknet_runner.py:191:         except
subprocess.TimeoutExpired:
.claude\worktrees\hunt-base\backend\pipeline\detectors\tracknet_runner.py:226:         except
subprocess.TimeoutExpired:
.claude\worktrees\hunt-base\backend\utils\run_telemetry.py:36:import subprocess
.claude\worktrees\hunt-base\backend\utils\run_telemetry.py:66:         out = subprocess.run(
.claude\worktrees\hunt-base\backend\utils\run_telemetry.py:70:         except
(subprocess.SubprocessError, OSError):
```



```

.claude\worktrees\hunt-base\backend\utils\run_telemetry.py:113:         out =
subprocess.run([exe, "--query-gpu=name", "--format=csv,noheader"],
.claude\worktrees\hunt-base\backend\utils\run_telemetry.py:115:     except
(subprocess.SubprocessError, OSError):
.claude\worktrees\hunt-base\backend\pipeline\detectors\__init__.py:13:Keeping the heavy TF/torch
deps quarantined in WSL (or a native subprocess)
.claude\worktrees\hunt-base\backend\pipeline\detectors\wasb_runner.py:1:"""WASB shuttle-
trajectory runner (WSL2 subprocess adapter).
.claude\worktrees\hunt-base\backend\pipeline\detectors\wasb_runner.py:15:import subprocess
.claude\worktrees\hunt-base\backend\pipeline\detectors\wasb_runner.py:81:         except
subprocess.TimeoutExpired:
.claude\worktrees\hunt-base\backend\pipeline\detectors\wasb_runner.py:98:         except
subprocess.TimeoutExpired:
.claude\worktrees\hunt-base\backend\pipeline\detectors\wasb_runner.py:173:         except
subprocess.TimeoutExpired:
.claude\worktrees\hunt-base\backend\pipeline\detectors\person_detector.py:155:         cap =
cv2.VideoCapture(chunk_path)
.claude\worktrees\hunt-base\backend\pipeline\detectors\person_detector.py:221:
cap.release()
.claude\worktrees\hunt-base\backend\pipeline\detectors\person_detector.py:276:         cap =
cv2.VideoCapture(video_path)
.claude\worktrees\hunt-base\backend\pipeline\detectors\person_detector.py:301:
cap.release()
.claude\worktrees\hunt-base\backend\pipeline\detectors\person_detector.py:322:         cap =
cv2.VideoCapture(video_path)
.claude\worktrees\hunt-base\backend\pipeline\detectors\person_detector.py:331:
cap.release()
.claude\worktrees\hunt-base\backend\pipeline\detectors\person_detector.py:356:
cap.release()
.claude\worktrees\hunt-base\backend\pipeline\segmenters\motion.py:28:         cap =
cv2.VideoCapture(video_path)
.claude\worktrees\hunt-base\backend\pipeline\segmenters\motion.py:111:         cap.release()
.claude\worktrees\hunt-base\backend\pipeline\detectors\wasb_infer.py:150:
os.remove(p)
.claude\worktrees\hunt-base\backend\pipeline\detectors\wasb_infer.py:293:     """Convert an in-
memory BGR uint8 frame (as ``cv2.VideoCapture`` yields) to the EXACT
.claude\worktrees\hunt-base\backend\pipeline\detectors\wasb_infer.py:557:         cap =
cv2.VideoCapture(video_path)
.claude\worktrees\hunt-base\backend\pipeline\detectors\wasb_infer.py:563:         cap.release()
.claude\worktrees\hunt-base\backend\pipeline\detectors\wasb_infer.py:573:         cap.release()
.claude\worktrees\hunt-base\backend\pipeline\detectors\wasb_infer.py:581:         cap =
cv2.VideoCapture(video_path)
.claude\worktrees\hunt-base\backend\pipeline\detectors\wasb_infer.py:585:         cap.release()
.claude\worktrees\hunt-base\backend\pipeline\detectors\wasb_infer.py:593:         long-lived
``cv2.VideoCapture``. It MUST be called with non-decreasing indices (the

```

[Showing results with pagination = limit: 60]

56. assistant (2026-06-18T01:35:13.704Z)

Let me examine person_detector.py and motion.py ? these open `cv2.VideoCapture` and there's a potential leak concern around the `cap.release()` calls. Let me look at the person_detector closely, especially the regions around lines 155-356.

57. user (2026-06-18T01:35:15.116Z)

```

140         def extract_action_windows_from_chunk(self, chunk_path: str, chunk_start: float,
141                                             velocity_thresh: float, merge_gap: float,
142                                             frame_skip: int = 3,
143                                             chunk_index: Optional[int] = None) ->
List[Dict[str, Any]]:
144         """
145         Runs person detection + tracking on a video chunk and extracts high-motion
146         action windows. Returns a list of dicts in the full video's timeframe, each
with:

```

```

147         start, end (seconds), active_frame_count, peak_velocity, mean_velocity,
148         track_id_count, chunk_index.
149
150     Args:
151         frame_skip: Process every Nth frame (1 = every frame). Higher values
152                     dramatically reduce inference cost with minimal quality loss.
153         chunk_index: Index of the source chunk (recorded on each window for
traceability).
154     """
155     cap = cv2.VideoCapture(chunk_path)
156     if not cap.isOpened():
157         logger.error(f"Could not open {chunk_path}")
158         return []
159
160     fps = cap.get(cv2.CAP_PROP_FPS)
161     if fps <= 0:
162         fps = 30.0
163
164     frame_width = cap.get(cv2.CAP_PROP_FRAME_WIDTH)
165     if frame_width <= 0:
166         frame_width = 1280.0 # Sensible fallback
167
168     # Effective sampling rate after frame-skipping
169     effective_fps = fps / max(frame_skip, 1)
170
171     # A fresh tracker per chunk ? track IDs only need to be consistent within a
172     # chunk (windows are computed per chunk, then merged across chunks by time).
173     tracker = self.make_tracker(effective_fps)
174     score_thresh = self.config.get("indexing", {}).get("detector_score_threshold",
0.5)
175
176     # Per active frame we keep (frame_idx, peak_velocity, set_of_active_track_ids)
177     # so windows can be enriched with motion stats for logging + fine-tuning.
178     active_frames = []
179     frame_idx = 0
180     track_history: Dict[int, Tuple[float, float, float, float]] = {}
181
182     while True:
183         ret, frame = cap.read()
184         if not ret:
185             break
186
187         # Skip frames for performance ? at 30fps, every 3rd frame still
188         # gives ~10 samples/second, plenty for human-scale velocity.
189         if frame_idx % max(frame_skip, 1) != 0:
190             frame_idx += 1
191             continue
192
193         # Detect persons + assign persistent track IDs.
194         detections = self.detect_and_track(frame, tracker, score_thresh)
195
196         frame_peak_velocity = 0.0
197         frame_active_ids = set()
198         current_ids = set()
199
200         for track_id, bbox in detections:
201             current_ids.add(track_id)
202
203             if track_id in track_history:
204                 prev_bbox = track_history[track_id]
205                 velocity = get_velocity_normalised(prev_bbox, bbox, effective_fps,
frame_width)
206
207                 if velocity > velocity_thresh:
208                     frame_peak_velocity = max(frame_peak_velocity, velocity)
209                     frame_active_ids.add(track_id)

```

```

209         track_history[track_id] = bbox
210
211     # Prune stale IDs that are no longer visible ? prevents false
212     # velocity spikes when the tracker reassigns an old ID.
213     track_history = {k: v for k, v in track_history.items() if k in current_ids}
214
215     if frame_active_ids:
216         active_frames.append((frame_idx, frame_peak_velocity, frame_active_ids))
217
218     frame_idx += 1
219
220 cap.release()
221
222 # Group active frames into continuous windows
223 if not active_frames:
224     return []
225
226 max_gap_frames = int(merge_gap * fps)
227
228 def _finalize(group: List[Tuple[int, float, set]]) -> Dict[str, Any]:
229     velocities = [v for _, v, _ in group]
230     track_ids = set().union(*[ids for _, _, ids in group])
231     return {
232         "start": group[0][0] / fps + chunk_start,
233         "end": group[-1][0] / fps + chunk_start,
234         "active_frame_count": len(group),
235         "peak_velocity": max(velocities),
236         "mean_velocity": sum(velocities) / len(velocities),
237         "track_id_count": len(track_ids),
238         "chunk_index": chunk_index,
239     }
240
241 windows = []
242 group = [active_frames[0]]
243 for af in active_frames[1:]:
244     if af[0] - group[-1][0] <= max_gap_frames:
245         group.append(af)
246     else:
247         windows.append(_finalize(group))
248         group = [af]
249 windows.append(_finalize(group))
250
251 # Filter out extremely short action windows
252 min_window_duration = self.config.get("indexing",
253 {}).get("min_action_window_duration", 2.0)
254 windows = [w for w in windows if (w["end"] - w["start"]) >= min_window_duration]
255
256 return windows
257
258 def detections_per_frame(self, video_path: str, start_frame: int, end_frame: int,
259                          frame_skip: int = 3) -> Dict[str, Any]:
260     """Per-frame person detections over ``[start_frame, end_frame]`` of the ORIGINAL
261     video.
262
263     Used by the fusion segmenter (MULTI_SIGNAL_FUSION_PLAN P2). Unlike
264     ``extract_action_windows_from_chunk`` (which detects ITS OWN windows and
265     discards the
266     per-frame boxes), this returns the raw per-frame detections so the fusion
267     features can
268     align them with the shuttle trajectory. It reads the source by **absolute frame
269     index**
270     (seek + sequential read), NOT a re-extracted clip, so the returned frame indices
271     match
272     the WASB trajectory's ``.frame`` exactly ? that alignment is what

```

```

268         ``fusion_features.shuttle_player_colocation`` / ``two_sided_motion`` rely on.
269
270     Returns ``{"frames": {frame_idx: [(track_id, bbox), ...]}, "fps", "frame_width",
271     "frame_height"}``. A fresh window-local ByteTrack tracker is used. ? cv2 frame-
seek
272     accuracy varies by codec (it lands near the requested keyframe then decodes
forward);
273     treat the indices as ±a few frames on real footage ? fine for window-level
features.
274     """
275     self.lazy_load_model()
276     cap = cv2.VideoCapture(video_path)
277     if not cap.isOpened():
278         logger.error(f"Could not open {video_path}")
279         return {"frames": {}, "fps": 30.0, "frame_width": 0.0, "frame_height": 0.0}
280
281     fps = cap.get(cv2.CAP_PROP_FPS)
282     if fps <= 0:
283         fps = 30.0
284     frame_width = cap.get(cv2.CAP_PROP_FRAME_WIDTH)
285     frame_height = cap.get(cv2.CAP_PROP_FRAME_HEIGHT)
286     score_thresh = self.config.get("indexing", {}).get("detector_score_threshold",
0.5)
287     step = max(frame_skip, 1)
288     tracker = self.make_tracker(fps / step)
289
290     start_frame = max(0, int(start_frame))
291     frames: Dict[int, List[Tuple[int, Tuple[float, float, float, float]]]] = {}
292     cap.set(cv2.CAP_PROP_POS_FRAMES, start_frame)
293     frame_idx = start_frame
294     while frame_idx <= end_frame:
295         ret, frame = cap.read()
296         if not ret:
297             break
298         if (frame_idx - start_frame) % step == 0:
299             frames[frame_idx] = self.detect_and_track(frame, tracker, score_thresh)
300             frame_idx += 1
301     cap.release()
302     return {"frames": frames, "fps": fps,
303           "frame_width": float(frame_width), "frame_height": float(frame_height)}
304
305     def detections_over_windows(self, video_path: str, windows: List[Tuple[int, int]],
306                               frame_skip: int = 3,
307                               progress_cb: Optional[Callable[[int, int], None]] = None
308                               ) -> List[Dict[str, Any]]:
309         """Per-window person detections via ONE sequential pass over the video.
310
311         Opens the video ONCE and reads forward ? ``grab()`` to advance, ``retrieve()`` +
312         detect only on sampled frames inside a window ? instead of
``detections_per_frame``'s
313         per-window open + ``POS_FRAMES`` seek, which on a long video degrades to
314         decode-from-start per window (O(window_index) ? the fusion-P3 extraction
bottleneck;
315         172 windows took ~80 min). ``windows`` = ``[(start_frame, end_frame), ...]``;
returns a
316         list aligned to it, each ``{"frames": {idx: [(tid,bbox)]}, "fps", "frame_width",
317         "frame_height"}`` (same shape as ``detections_per_frame``). Each window gets its
own
318         window-local ByteTrack. ``progress_cb(frames_done, total_frames)`` (called every
~256
319         frames) drives progress logging.
320         """
321         self.lazy_load_model()
322         cap = cv2.VideoCapture(video_path)
323         n = len(windows)

```

```

324         fps = cap.get(cv2.CAP_PROP_FPS) if cap.isOpened() else 0.0
325         if fps <= 0:
326             fps = 30.0
327         fw = cap.get(cv2.CAP_PROP_FRAME_WIDTH) if cap.isOpened() else 0.0
328         fh = cap.get(cv2.CAP_PROP_FRAME_HEIGHT) if cap.isOpened() else 0.0
329         base = {"fps": fps, "frame_width": float(fw), "frame_height": float(fh)}
330         if not cap.isOpened() or n == 0:
331             cap.release()
332             return [dict(frames={}, **base) for _ in range(n)]
333
334         step = max(frame_skip, 1)
335         score_thresh = self.config.get("indexing", {}).get("detector_score_threshold",
0.5)
336         frames_per_win: List[Dict[int, Any]] = [{ } for _ in range(n)]
337         trackers = [self.make_tracker(fps / step) for _ in range(n)]
338         max_end = max(ef for _, ef in windows)
339
340         frame_idx = 0
341         while frame_idx <= max_end:
342             if not cap.grab():
343                 break
344             # Windows that want THIS frame (inside the range AND on that window's
sampling grid).
345             wis = [wi for wi, (sf, ef) in enumerate(windows)
346                   if sf <= frame_idx <= ef and (frame_idx - sf) % step == 0]
347             if wis:
348                 ok, frame = cap.retrieve()
349                 if ok:
350                     for wi in wis:
351                         frames_per_win[wi][frame_idx] = self.detect_and_track(
352                             frame, trackers[wi], score_thresh)
353                 frame_idx += 1
354                 if progress_cb is not None and frame_idx % 256 == 0:
355                     progress_cb(frame_idx, max_end)
356             cap.release()
357             return [dict(frames=frames_per_win[wi], **base) for wi in range(n)]
358

```

58. assistant (2026-06-18T01:35:22.146Z)

In `extract\_action\_windows\_from\_chunk` (line 155-221) and `detections\_per\_frame` (line 276-301), if `detect\_and\_track` raises mid-loop, `cap.release()` is skipped. But these run inside the segmenter where exceptions propagate up. These are GPU/cv2-only paths (can't run here). A leaked `cv2.VideoCapture` on exception is a minor finding at best; the code is consistent with the codebase style. Not a strong, concrete-trigger defect.

Let me now look at the wasb_runner.py (the WSL adapter), which I haven't read yet, and check the doctored areas. Let me also look at the `tests/` DB code is out of scope. Let me read wasb_runner.py.

59. user (2026-06-18T01:35:23.231Z)

```

1     """WASB shuttle-trajectory runner (WSL2 subprocess adapter).
2
3     Windows-side counterpart to backend/detectors/wasb_infer.py. It:
4         1. stages the video into the WSL filesystem (fast local reads, not /mnt),
5         2. copies wasb_infer.py into the WASB repo's src/ dir,
6         3. runs it in the WSL `wasb` conda env -> a Frame,Visibility,X,Y trajectory CSV,
7         4. reuses the (model-agnostic) trajectory->action-window logic from tracknet_runner.
8
9     So the only WASB-specific bits live here; the windowing + CSV parsing are shared
10    with the TrackNet path. Verified inference core: wasb_infer.py (see its docstring).
11    """
12    import os
13    import shlex

```

```

14     import logging
15     import subprocess
16     from dataclasses import dataclass
17     from typing import Dict, Any, Optional, Tuple
18
19     # Reuse the model-agnostic helpers ? trajectory shape + windowing are identical.
20     # parse_trajectory_csv / trajectory_to_action_windows are @staticmethods on
TrackNetRunner.
21     from backend.pipeline.detectors.tracknet_runner import to_wsl_mnt_path, TrackNetRunner
22     from backend.pipeline.detectors.base import DetectorRunner, WslCommandMixin,
wsl_tilde_quote
23
24     logger = logging.getLogger(__name__)
25
26     # Windows path to the inference wrapper we stage into WSL.
27     _INFER_WIN = os.path.join(os.path.dirname(os.path.abspath(__file__)), "wasb_infer.py")
28
29
30     @dataclass
31     class WasbConfig:
32         """Resolved settings for a WASB WSL run (built from config.json ->
indexing.wasb)."""
33         distro: str = "Ubuntu"
34         repo_dir: str = "~/models/WASB-SBDT"
35         conda_sh: str = "~/miniconda3/etc/profile.d/conda.sh"
36         conda_env: str = "wasb"
37         weights_path: str = "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"
38         sport: str = "badminton"
39         wsl_stage_dir: str = "~/clips"
40         timeout_sec: int = 1800 # 30 min; a hung GPU/conda call is killed (0 = no timeout)
41         keep_frames: bool = False # retain staged video + frame cache after success (debug
only)
42         min_free_gb: float = 0.0 # warn if WSL free space below this before a run (0 =
off)
43         # FAST PATH (default OFF until verified): decode the staged video on the fly inside
44         # WSL and feed frames straight to the detector, skipping the slow per-frame PNG
45         # extraction that dominates a long clip. Output is bit-identical to the PNG path
46         # (PNG is lossless), but this stays opt-in until the owner's side-by-side confirms
it.
47         stream_video: bool = False
48
49     @classmethod
50     def from_indexing_cfg(cls, idx_cfg: Dict[str, Any]) -> "WasbConfig":
51         w = (idx_cfg or {}).get("wasb", {}) or {}
52         return cls(
53             distro=w.get("wsl_distro", "Ubuntu"),
54             repo_dir=w.get("repo_dir", "~/models/WASB-SBDT"),
55             conda_sh=w.get("conda_sh", "~/miniconda3/etc/profile.d/conda.sh"),
56             conda_env=w.get("conda_env", "wasb"),
57             weights_path=w.get("weights_path",
58                               "~/models/WASB-
SBDT/pretrained_weights/wasb_badminton_best.pth.tar"),
59             sport=w.get("sport", "badminton"),
60             wsl_stage_dir=w.get("wsl_stage_dir", "~/clips"),
61             timeout_sec=int(w.get("timeout_sec", 1800)),
62             keep_frames=bool(w.get("keep_frames", False)),
63             min_free_gb=float(w.get("min_free_gb", 0.0)),
64             stream_video=bool(w.get("stream_video", False)),
65         )
66
67
68     class WasbRunner(WslCommandMixin, DetectorRunner):
69         def __init__(self, cfg: WasbConfig):
70             self.cfg = cfg

```

```

71
72 def healthcheck(self) -> Tuple[bool, str]:
73     """Verify the WSL `wasb` env imports torch and sees a GPU. Returns (ok, msg)."""
74     cmd = (f"conda activate {self.cfg.conda_env} && "
75           f"python -c \"import torch; print('torch', torch.__version__, "
76           f"'cuda', torch.cuda.is_available())\"")
77     try:
78         res = self._wsl_bash(cmd)
79     except FileNotFoundError:
80         return False, "`wsl` not found ? Windows + WSL2 required."
81     except subprocess.TimeoutExpired:
82         return False, "WSL healthcheck timed out."
83     out = (res.stdout or "").strip()
84     if res.returncode != 0:
85         return False, f"WSL `wasb` env check failed: {(res.stderr or out).strip()}"
86     return True, out
87
88 def _stage_video(self, video_win_path: str, dest_name: str) -> Optional[str]:
89     src_mnt = to_wsl_mnt_path(video_win_path)
90     reason = self.wsl_source_unreadable_reason(src_mnt)
91     if reason:
92         logger.error("Cannot stage video into WSL: %s", reason)
93         return None
94     dest = f"{self.cfg.wsl_stage_dir}/{dest_name}"
95     cmd = f"mkdir -p {self.cfg.wsl_stage_dir} && cp {shlex.quote(src_mnt)} "
96     {wsl_tilde_quote(dest)} && echo OK"
97     try:
98         res = self._wsl_bash(cmd)
99     except subprocess.TimeoutExpired:
100         logger.error("Staging video into WSL timed out.")
101         return None
102     if res.returncode != 0 or "OK" not in (res.stdout or ""):
103         logger.error("Failed to stage video into WSL: %s", (res.stderr or
104 res.stdout).strip())
105         return None
106     return dest
107
108 def _stage_infer_script(self) -> bool:
109     """Copy wasb_infer.py into the WASB repo src/ so it can import WASB modules."""
110     src_mnt = to_wsl_mnt_path(_INFER_WIN)
111     cmd = f"cp {shlex.quote(src_mnt)} {self.cfg.repo_dir}/src/wasb_infer.py && echo
112 OK"
113     res = self._wsl_bash(cmd)
114     return res.returncode == 0 and "OK" in (res.stdout or "")
115
116 def run_predict(self, video_win_path: str, output_win_dir: str,
117                 video_id: Optional[str] = None) -> Optional[str]:
118     """Run WASB on a video. Returns the Windows path to the trajectory CSV, or None.
119
120     All WSL/cache/output artifacts are namespaced by ``video_id`` (the file MD5) so
121     two videos that share a filename (e.g. two ``match.mp4`` from different folders)
122     can never reuse each other's staged frames, resumable cache, or CSV.
123
124     """
125     # Resolve relative dirs (the hybrids pass "output/wasb") BEFORE the /mnt
126     # translation: to_wsl_mnt_path passes relative paths through unchanged, so
127     # WSL resolved them against the inference cwd (~/.models/WASB-SBDT/src) and
128     # the CSV landed inside WSL while Windows polled the repo-relative path.
129     output_win_dir = os.path.abspath(output_win_dir)
130     os.makedirs(output_win_dir, exist_ok=True)
131     # Normalize for cross-platform basename (tests use Windows paths on Linux).
132     video_win_path = str(video_win_path).replace("\\", "/")
133     base = os.path.basename(video_win_path)
134     stem, ext = os.path.splitext(base)
135     # Unique, content-derived key: same filename + different bytes -> different key.
136     key = f"{stem}__{video_id[:12]}" if video_id else stem

```

```

133     wsl_out_dir = to_wsl_mnt_path(output_win_dir)
134     expected_csv_win = os.path.join(output_win_dir, f"{key}_wasb.csv")
135
136     if not self._stage_infer_script():
137         logger.error("Could not stage wasb_infer.py into WSL.")
138         return None
139     staged_video = self._stage_video(video_win_path, f"{key}{ext}")
140     if staged_video is None:
141         return None
142     frames_dir = f"{self.cfg.wsl_stage_dir}/{key}_frames"
143
144     # Disk guard: a 4K clip can extract ~100 GB of PNG frames. Warn early if the
145     # WSL stage filesystem is already low so the user can clean up before it fills.
146     # (Streaming never materializes the PNGs, so the guard only matters off the fast
147     path.)
148     if self.cfg.min_free_gb > 0 and not self.cfg.stream_video:
149         free = self._wsl_free_gb(self.cfg.wsl_stage_dir)
150         if free is not None and free < self.cfg.min_free_gb:
151             logger.warning(
152                 "Low WSL disk: %.1f GB free at %s (< min_free_gb=%.1f). Frame
153                 extraction "
154                 "may fill the disk ? consider running tools/cleanup_caches.py.",
155                 free, self.cfg.wsl_stage_dir, self.cfg.min_free_gb)
156
157     # Fast path: stream-decode the video in-process (no PNG extraction). Output is
158     # bit-identical to the PNG round-trip but avoids writing ~46k PNGs for a 12-min
159     clip.
160     if self.cfg.stream_video:
161         frame_args = "--stream-video "
162     else:
163         frame_args = f"--frames_out_dir {wsl_tilde_quote(frames_dir)} "
164     cmd = (
165         f"conda activate {self.cfg.conda_env} && cd {self.cfg.repo_dir}/src && "
166         f"python wasb_infer.py "
167         f"--video {wsl_tilde_quote(staged_video)} "
168         f"{frame_args}"
169         f"--weights {wsl_tilde_quote(self.cfg.weights_path)} "
170         f"--sport {shlex.quote(self.cfg.sport)} "
171         f"--out {wsl_tilde_quote(wsl_out_dir + '/' + key + '_wasb.csv')}"
172     )
173     logger.info("Running WASB inference on %s ...", base)
174     try:
175         res = self._wsl_bash(cmd)
176     except subprocess.TimeoutExpired:
177         logger.error("WASB inference timed out after %ss.", self.cfg.timeout_sec)
178         return None
179     if res.returncode != 0:
180         logger.error("WASB inference failed:\n%s", (res.stderr or
181             res.stdout)[-2000:])
182         return None
183     if not os.path.exists(expected_csv_win):
184         logger.error("WASB finished but expected CSV not found at %s",
185             expected_csv_win)
186         return None
187     logger.info("WASB trajectory CSV: %s", expected_csv_win)
188
189     # Success ? the trajectory CSV is the durable output; the staged video copy and
190     # the (huge) decoded frame cache are now pure intermediates. Delete them unless
191     # explicitly retained. On any earlier failure we return above WITHOUT cleaning,
192     # so the frame cache + detector cache remain for a resumable re-run.
193     # (Streaming never writes a frame cache, so there's only the staged video to
194     drop.)
195     if not self.cfg.keep_frames:
196         stale = [staged_video] if self.cfg.stream_video else [frames_dir,
197             staged_video]

```



```

191         logger.info("Cache hygiene: removing staged video%s for %s "
192                     "(set indexing.wasb.keep_frames=true to retain).",
193                     "" if self.cfg.stream_video else " + frame cache", key)
194         self._wsl_rm_rf(stale)
195         return expected_csv_win
196
197     # Trajectory parsing + windowing are inherited from tracknet_runner (model-
agnostic).
198     parse_trajectory_csv = staticmethod(TrackNetRunner.parse_trajectory_csv)
199     trajectory_to_action_windows =
staticmethod(TrackNetRunner.trajectory_to_action_windows)
200

```

60. assistant (2026-06-18T01:35:33.334Z)

Clean. Now let me revisit the `wasb\_infer.py` `run()` function more carefully, since it has the most complex file handling with `det\_f = open(...)` . There's a subtle issue I want to verify. At line 480:

```

```python
det_f = open(det_jsonl, "a")
```

```

This is in append mode, and `load\_and\_clean\_cache` (called at line 433 in resume) already rewrote the file with `\_atomic\_write\_text` to a clean prefix. So appending continues correctly. The `try/finally` closes it. Fine.

Let me reconsider the `extract\_frames` function (line 547-575) in wasb_infer.py:

```

```python
cap = cv2.VideoCapture(video_path)
...
while True:
 ok, frame = cap.read()
 if not ok:
 break
 cv2.imwrite(osp.join(frames_dir, f"{i:08d}.png"), frame)
 i += 1
cap.release()
```

```

If `cv2.imwrite` raises (e.g. disk full mid-write), `cap.release()` is skipped. This is a WSL-only utility script; minor.

Now let me focus on something potentially stronger. Let me re-examine `main.py` `upload\_part` for a concrete bug. Look at line 480-489 again:

```

```python
with open(st["tmp_path"], "ab") as fh:
 async for chunk in request.stream():
 received += len(chunk)
 if received > part_bytes: ... break
 if st["bytes"] + received > max_bytes: ... break
 fh.write(chunk)
```

```

There's a subtle resource concern: this is async with a blocking `open()` and synchronous `fh.write()` inside an async function. That's a style issue, not a resource leak. The `with` handles closing.

Let me look at the remaining DB-touching test-adjacent code and the calibrate_local/distill_local and rally_seg_eval more carefully for subprocess. Let me check gemini_refine.py which was mentioned.

61. user (2026-06-18T01:35:35.345Z)

No matches found

62. user (2026-06-18T01:35:36.170Z)

```
40         return total
41     return float(value)
42
43
44     def load_annotations(csv_path: str) -> List[Interval]:
45         """Load and validate ground-truth rally intervals from a CSV file.
46
47         Delegates to the canonical loader (gt_loader.py, ADR-008 prereq), which accepts BOTH
48         golden conventions: positional/headered ``start,end`` (the collector export) and the
49         rally-annotator ``rally_number,start_time,end_time,...`` schema. Strict: raises
50         ValueError on malformed rows (bad timestamp, start >= end) so labeling mistakes
51         surface immediately rather than silently skewing scores. Returns intervals sorted by
52         start time.
53         """
54         from backend.eval.gt_loader import load_gt_intervals
55         intervals = load_gt_intervals(csv_path, strict=True)
56         logger.info("Loaded %d ground-truth rallies from %s", len(intervals), csv_path)
57         return intervals
58
59
60     def write_scaffold(csv_path: str) -> None:
61         """Write an empty annotation file with just the header, for hand-filling."""
62         with open(csv_path, "w", newline="") as f:
63             writer = csv.writer(f)
64             writer.writerow(["start", "end"])
65
```

63. assistant (2026-06-18T01:35:45.772Z)

Let me reconsider the database.py more carefully, since that's the highest-value area for DB resource handling. Looking at the `\_connect` context manager:

```
```python
@contextmanager
def _connect(self):
 conn = self._get_connection()
 try:
 with conn:
 yield conn
 finally:
 conn.close()
```
```

But `\_get\_connection()` does multiple `conn.execute("PRAGMA ...")` calls (lines 35-41). If, say, `conn.execute("PRAGMA foreign\_keys = ON")` raises (unlikely but possible on a bad DB file), the connection created at line 32 leaks? `\_get\_connection` itself has no try/finally. But that's an edge case, low severity.

Let me look closer at a real concern in `\_get\_connection`. The `PRAGMA journal\_mode = WAL` is wrapped in try/except `OperationalError`, but the earlier `PRAGMA foreign\_keys` and `PRAGMA busy\_timeout` are not. On a locked DB, `PRAGMA busy\_timeout` won't fail. These are fine.

Now let me check the `claim\_due\_job` for a real concurrency/transaction bug. The `\_connect` wraps in `with conn:` (a transaction). Inside, it does a SELECT then UPDATE then `conn.commit()`. The explicit `conn.commit()` inside a `with conn:` block is redundant but harmless. The compare-and-set on `(id, status)` with `cur.rowcount` is correct.

Let me now examine the test files that are in scope per the grep (test_database, test_async_jobs)? actually tests aren't a concern for production defects. Let me look at the remaining script with sqlite/subprocess: `training/gen0/harness.py`.

64. user (2026-06-18T01:35:46.284Z)

```
1      """Gen-0 TAL harness (M0.4): ONE command -> train -> honest LOVO eval -> ship-gate ->
artifact.
2
3      python -m training.gen0.harness --manifest output/human_lovo_manifest.json \
4      --floor eval_baselines/heuristic_lovo_n6.json --out artifacts/rally_tal_gen0
--version 0.1.0
5
6      Productionizes the rally_seq prototype per ADR-008:
7      - TRAINS on the corpus manifest (per-video trajectory + golden labels), evaluates with
8      honest leave-one-video-out (split by match, threshold tuned on train only), then fits
9      the FINAL model on all videos.
10     - Emits a versioned ARTIFACT BUNDLE ? the only thing serving may ever consume:
11         <out>/<version>/weights.json      (gitignored; classifier + frozen normalization)
12         <out>/<version>/manifest.json     (the reviewable contract: feature-schema version,
13                                           calibration, training lineage + GT hashes,
14                                           eval results, attestations, gate verdict)
15     - Runs the SHIP-GATE as pre-flight: floor comparison + a PAIRED per-video sign test
16       (at n=6 a mean gap alone is noise-ambiguous), plus the hard attestation blocks
17       (WASB C3 unresolved => commercial ship refused; explicit F1@tIoU target still
18       undefined => no self-declared victory). The product side owns final admission
19       (backend/eval/regression.py); this harness never grades itself into serving.
20
21     Training may import backend.* (one-way wall); backend must never import training.*.
22     """
23     from __future__ import annotations
24
25     import argparse
26     import hashlib
27     import json
28     import os
29     import subprocess
30     import time
31     from math import comb
32     from typing import Dict, List, Optional
33
34     import numpy as np
35
36     from backend.eval.classifier import LogisticRegression
37     from backend.eval.gt_loader import load_gt_intervals
38     from backend.eval.regression import gt_hash
39     from backend.eval.rally_seq_proto import (
40         labels_for, run as lovo_run, _seg_f1,
41     )
42     from backend.features.rally_seq import FEAT_NAMES, FEATURE_SCHEMA, build_frame_arrays,
features
43     from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
44
45     # Post-processing constants ? part of the model's identity, recorded in the manifest.
46     CALIBRATION_DEFAULTS = {"smooth": 7, "merge_gap": 1.0, "min_dur": 1.0}
47     THRESHOLD_GRID = [round(t, 2) for t in np.arange(0.2, 0.81, 0.05)]
48
49
50     def _git_sha() -> str:
51         try:
52             return subprocess.run(["git", "rev-parse", "HEAD"], capture_output=True,
53                                   text=True, timeout=10).stdout.strip() or "unknown"
54         except Exception:
55             return "unknown"
56
57
58     def _sha256(path: str) -> str:
59         h = hashlib.sha256()
60         with open(path, "rb") as f:
```

```

61         for chunk in iter(lambda: f.read(65536), b''):
62             h.update(chunk)
63     return h.hexdigest()
64
65
66     def sign_test(learned: List[float], floor: List[float]):
67         """Paired one-sided sign test: is learned > floor more often than chance?
68
69         At n=6 a mean comparison alone is inside noise (SE ~0.11); the paired per-video
70         wins are the defensible statistic. Returns (wins, n, p_one_sided)."""
71         pairs = [(lv, fv) for lv, fv in zip(learned, floor) if lv != fv] # ties drop,
standard
72         wins = sum(1 for lv, fv in pairs if lv > fv)
73         n = len(pairs)
74         p = sum(comb(n, k) for k in range(wins, n + 1)) / (2 ** n) if n else 1.0
75         return wins, n, p
76
77
78     def train_final(specs: List[Dict]):
79         """Fit the FINAL model on ALL corpus videos; threshold tuned on the same (train-
side).
80
81         The honest generalization estimate is the LOVO result, not this fit ? recorded as
such."""
82         vids = []
83         for s in specs:
84             pts = TrackNetRunner.parse_trajectory_csv(s["trajectory"])
85             arr = build_frame_arrays(pts, float(s["frame_width"]), float(s["fps"]))
86             X, times = features(arr, float(s["fps"]))
87             gts = load_gt_intervals(s["labels"], strict=False)
88             vids.append({"name": s["name"], "X": X, "times": times,
89                         "y": labels_for(times, gts), "gts": gts})
90         Xall = np.vstack([v["X"] for v in vids])
91         yall = np.concatenate([v["y"] for v in vids])
92         clf = LogisticRegression(lr=0.2, epochs=2000, l2=1e-2)
93         clf.fit(Xall, yall, feature_names=FEAT_NAMES)
94         thr = max(THRESHOLD_GRID, key=lambda t: _seg_f1(clf, vids, float(t)))
95         return clf, float(thr)
96
97
98     def gate_verdict(eval_res: dict, floor: Optional[dict]) -> dict:
99         """Pre-flight ship-gate.
100
101         Provisional F1 target (NOT owner-ratified; calibration tracked in #172).
102         Floor (necessary): heuristic LOVO ~0.480 (from quality table).
103         Multi-court ref (makes owned \"worth serving\" per data): 0.66.
104         Proposed for \"very high P/R\" + significance: F1@tIoU0.5 >= 0.70 on
105         multi-court folds with paired-sign p<0.05 vs heuristic (refine as corpus grows).
106         Beating floor + sign test necessary; C3 main commercial blocker.
107         ship=True only when all non-C3 blocks clear (honesty first).
108         """
109         reasons: List[str] = []
110         floor_passed = None
111         sign = None
112         if floor:
113             floor_mean = float(floor["mean_f1"])
114             floor_passed = eval_res["mean_f1"] > floor_mean
115             if not floor_passed:
116                 reasons.append(f"LOVO mean {eval_res['mean_f1']:.3f} <= floor
{floor_mean:.3f}")
117             per = floor.get("per_video", {})
118             paired = [(r["f1"], per[r["video"]]) for r in eval_res["per_video"] if
r["video"] in per]
119             if paired:
120                 wins, n, p = sign_test([lv for lv, _ in paired], [fv for _, fv in paired])

```

```

121         sign = {"wins": wins, "n": n, "p_one_sided": round(p, 4)}
122         if p >= 0.05:
123             reasons.append(f"paired sign test not significant ({wins}/{n} wins,
124                             p={p:.3f}) "
125                             f"? grow the corpus before claiming superiority")
126         else:
127             reasons.append("no floor baseline provided ? nothing to beat")
128         # Provisional blocker (per owner review on #170; see #172 for calibration).
129         # (The number itself is not the blocker; calibration tracked in #172.)
130         reasons.append("ship-gate F1@tIoU target NOT YET CALIBRATED ? provisional; see issue
131 #172")
132         reasons.append("WASB checkpoint provenance C3 UNRESOLVED ? commercial ship blocked
133 regardless of F1")
134         # ship remains False until C3 resolved + other gates (by design).
135         return {"ship": False, "floor_passed": floor_passed, "sign_test": sign, "blockers":
136 reasons}
137
138 def run(manifest_path: str, out_dir: str, version: str,
139         floor_path: Optional[str] = None) -> dict:
140     with open(manifest_path, encoding="utf-8") as f:
141         specs = json.load(f)
142         floor = None
143         if floor_path:
144             with open(floor_path, encoding="utf-8") as f:
145                 floor = json.load(f)
146
147         # 1) Honest LOVO eval (the number that matters).
148         eval_res = lovo_run(specs)
149
150         # 2) Final model on all videos.
151         clf, thr = train_final(specs)
152
153         # 3) Gate (pre-flight).
154         verdict = gate_verdict(eval_res, floor)
155
156         # 4) Artifact bundle.
157         bundle_dir = os.path.join(out_dir, version)
158         os.makedirs(bundle_dir, exist_ok=True)
159         weights_path = os.path.join(bundle_dir, "weights.json")
160         payload = clf.to_dict()
161         payload["calibration"] = {"threshold": thr, **CALIBRATION_DEFAULTS}
162         payload["feature_schema_version"] = FEATURE_SCHEMA["version"]
163         with open(weights_path, "w", encoding="utf-8") as f:
164             json.dump(payload, f, indent=2)
165
166     manifest = {
167         "artifact": {
168             "name": "rally_tal_gen0", "version": version,
169             "created_at": time.strftime("%Y-%m-%dT%H:%M:%S%z"),
170             "git_sha": _git_sha(),
171             "weights_file": "weights.json", "weights_sha256": _sha256(weights_path),
172         },
173         "feature_schema": FEATURE_SCHEMA,
174         "calibration": {"threshold": thr, **CALIBRATION_DEFAULTS,
175             "note": "threshold tuned train-side on the full corpus; "
176                     "honest generalization = the LOVO eval below"},
177         "training_lineage": {
178             "corpus_manifest": manifest_path,
179             "split_protocol": "leave-one-video-out by match",
180             "label_source": "human (rally-annotator golden CSVs)",
181             "videos": [{"name": s["name"], "labels": s["labels"]},
182                         {"gt_hash": gt_hash(load_gt_intervals(s["labels"]],
183 strict=False)),

```

```

181                 "fps": s["fps"]} for s in specs],
182             },
183             "eval": {
184                 "protocol": "LOVO, F1@tIoU=0.5 segment-level + per-frame AUC",
185                 "per_video": eval_res["per_video"],
186                 "mean_f1": eval_res["mean_f1"],
187                 "floor": floor,
188             },
189             "attestation": {
190                 "deps_licenses": "numpy/scipy (BSD) + repo-internal code only (Gen-0)",
191                 "wasb_checkpoint_provenance": "C3-UNRESOLVED (ship-blocking; see
COMMERCIALIZATION C3)",
192                 "no_paid_llm_training_labels": True,
193                 "consent_ledger": "pending-M0.1 (owner-attested: owned footage, all-adult
sessions)",
194             },
195             "gate": verdict,
196         }
197     manifest_out = os.path.join(bundle_dir, "manifest.json")
198     with open(manifest_out, "w", encoding="utf-8") as f:
199         json.dump(manifest, f, indent=2)
200
201     print(f"\n=== Gen-0 artifact bundle: {bundle_dir} ===")
202     print(f"weights : {weights_path}")
203     print(f"manifest: {manifest_out}")
204     print(f"GATE      : ship={verdict['ship']} floor_passed={verdict['floor_passed']} "
205           f"sign_test={verdict['sign_test']}")
206     for b in verdict["blockers"]:
207         print(f" - {b}")
208     return manifest
209
210
211     def main() -> None:
212         ap = argparse.ArgumentParser(description=__doc__)
213         ap.add_argument("--manifest", required=True,
214                         help="corpus manifest JSON:
[{name,trajectory,fps,frame_width,labels}, ...]")
215         ap.add_argument("--floor", default=None,
216                         help="floor baseline JSON (e.g.
eval_baselines/heuristic_lovo_n6.json)")
217         ap.add_argument("--out", default=os.path.join("artifacts", "rally_tal_gen0"))
218         ap.add_argument("--version", required=True, help="semver for the artifact bundle")
219         args = ap.parse_args()
220         run(args.manifest, args.out, args.version, floor_path=args.floor)
221
222
223     if __name__ == "__main__":
224         main()
225

```

65. assistant (2026-06-18T01:35:58.346Z)

Clean. Now let me reconsider the `compiler.py` more carefully, as it has the most complex subprocess+temp file logic with a retry loop. Let me re-examine the watermark retry loop (lines 296-335) for a concrete bug.

Looking carefully at the loop logic:

```

```python
attempts = [vf_args, []] if vf_args else [[]]
produced = False
last_exc = None
cmd = base_cmd + [clip_path]
for attempt_vf in attempts:
 cmd = base_cmd + attempt_vf + [clip_path]
 try:

```

```

 subprocess.run(cmd, capture_output=True, check=True)
 except subprocess.CalledProcessError as e:
 last_exc = e
 if attempt_vf and vf_args:
 ... logger.warning ...
 continue
 break
 if os.path.exists(clip_path) and os.path.getsize(clip_path) > 0:
 produced = True
 ...
 break
 last_exc = None # ffmpeg succeeded but produced nothing
 break
...

```

This is correct. Now let me think about a subtle issue: when ffmpeg fails on the first (watermark) attempt and a partial `clip\_path` exists from that attempt, then on the second attempt (no watermark) ffmpeg runs with `-y` so it overwrites. Fine.

Now let me look at the most interesting candidate. Let me re-examine the `\_finalize` in tracknet\_runner's `trajectory\_to\_action\_windows`. Look at the R5 blob fallback. Actually, let me re-examine the line 494 noqa comment about B023 (closure over loop variable). Let me check `\_split\_overlong\_groups` line 449:

```

```python
split_i = min(range(1, len(g)), key=lambda i: abs(g[i][0] - mid))
```

```

This is fine because it executes immediately (min consumes it). Similarly line 494 has noqa B023. These are not bugs.

Let me now look at the `calibrate\_local.py` and `distill\_local.py` more closely since they load trajectory CSVs and run inference. Let me also double check the `rally\_seg\_eval.py` file handling at line 105-113.

## 66. user (2026-06-18T01:35:59.565Z)

```

95 return out
96
97
98 # ----- #
99 # Golden corpus I/O
100 # ----- #
101 def load_golden(manifest_path: str = DEFAULT_MANIFEST,
102 features_dir: str = DEFAULT_FEATURES_DIR) -> List[Dict[str, Any]]:
103 """Load each golden video's trajectory path + fps + frame_width (manifest) and GT
104 rally
105 intervals (`<name>_golden_features.json` ``gts``). Videos with no GTs are kept but
106 flagged."""
107 with open(manifest_path) as fh:
108 manifest = json.load(fh)
109 out: List[Dict[str, Any]] = []
110 for v in manifest:
111 feat = os.path.join(features_dir, f"{v['name']}_golden_features.json")
112 gts: List[Interval] = []
113 if os.path.exists(feat):
114 with open(feat) as fh:
115 gts = [(float(s), float(e)) for s, e in (json.load(fh).get("gts") or
116 [])]
117 out.append({"name": v["name"], "trajectory": v["trajectory"],
118 "fps": float(v["fps"]), "frame_width": float(v["frame_width"]),
119 "gts": gts})
120 return out
121
122
123

```

```

119 def windows_for(video: Dict[str, Any], preset: WindowingPreset,
120 min_crossings: int = 0) -> List[Interval]:
121 """Parse a video's trajectory, window it under ``preset`` ? predicted rally
intervals.
122
123 ``min_crossings`` > 0 applies the **net-crossings rally-state gate** (Gate-A,
124 ``rally_gate.gate_windows``): drop windows whose shuttle crosses the net fewer than
this many
125 times (a non-rally fragment). This is the cheapest rally-state cue (W2) ? CPU-only,
derived
126 from the trajectory + an auto-estimated net axis; it cleans up the over-split that
bounded
127 windowing (W1) leaves behind.
128
129 NEVER-EMPTY safeguard (mirrors ``FusionSegmenter._select_for_handoff``): if the gate
would
130 drop EVERY window of a video that had ?1, fall back to the ungated windows. The gate
is
131 strictly *pruning* ? it never zeroes out a non-empty video ? so W2-on-W1 is "do no
harm".
132 """
133 from backend.pipeline.detectors.tracknet_runner import TrackNetRunner
134 points = TrackNetRunner.parse_trajectory_csv(video["trajectory"])

```

#### 67. assistant (2026-06-18T01:36:06.886Z)

Clean. Now let me reconsider the strongest candidate I found. Let me re-examine the  
`\_get\_connection` in database.py once more for the WAL/foreign\_keys edge case, and re-read  
`claim\_due\_job` for the cross-`\_connect`-boundary issue at line 611.

Looking at `claim\_due\_job` (lines 591-614):

```

```python
with self._connect() as conn:
    cur = conn.cursor()
    row = cur.execute("SELECT id, status FROM jobs WHERE ...").fetchone()
    if not row:
        return None
    cur.execute("UPDATE jobs SET status='running' ... WHERE id=? AND status=?", ...)
    conn.commit()
    if cur.rowcount != 1:
        return None
return self.get_job(row["id"])
```

```

`self.get\_job(row["id"])` runs AFTER the `with self.\_connect()` block closes its connection,  
opening a NEW connection. This is correct and avoids holding the lock. Fine.

Let me now look at the per\_user\_regression.py and serve\_contrast.py / served\_gate\_a.py /  
calibrate\_local.py for any subprocess invocation that produces files. Let me grep for  
`os.makedirs`, file writes without context managers, and check the classifier.py save/load.

#### 68. user (2026-06-18T01:36:08.221Z)

```

100 m.w = np.asarray(d["w"], dtype=float) if d.get("w") is not None else None
101 m.b = float(d.get("b", 0.0))
102 m.mean = np.asarray(d["mean"], dtype=float) if d.get("mean") is not None else
None
103 m.std = np.asarray(d["std"], dtype=float) if d.get("std") is not None else None
104 m.feature_names = d.get("feature_names")
105 return m
106
107 def save(self, path: str) -> None:
108 with open(path, "w") as f:
109 json.dump(self.to_dict(), f, indent=2)
110

```



```

111 @classmethod
112 def load(cls, path: str) -> "LogisticRegression":
113 with open(path) as f:
114 return cls.from_dict(json.load(f))
115

```

## 69. assistant (2026-06-18T01:36:17.269Z)

Clean. I've now reviewed all the production subprocess/file/DB code paths thoroughly. Let me do a final targeted check on a couple of areas I want to confirm: the `gemini.py` chunk cleanup when `os.rmdir` is called on a non-empty dir (concurrency), and re-examine one thing in `main.py` `upload\_complete` re-keying.

Let me reconsider the `gemini.py` chunk dir. At line 263, after all chunks processed, `os.rmdir(chunks\_dir)`. If a chunk file failed to be removed (line 200-203 swallows exception), `os.rmdir` will fail but is also swallowed (line 264). No leak issue, just leaves a file. Fine.

Let me verify the one place I'm slightly suspicious about - the `wasb\_infer.py` `run()` function: the `det\_f` handle and what happens with `load\_and\_clean\_cache` rewriting the file. Actually, let me reconsider a subtle correctness issue: When NOT resuming (fresh), `reset\_cache` deletes the file, then `det\_f = open(det\_jsonl, "a")` recreates it. When resuming, `load\_and\_clean\_cache` rewrites it to the clean prefix and then we append. Both correct.

Let me do one final review of the `motion.py` segmenter for the cv2 capture handling, since it's a production segmenter.

## 70. user (2026-06-18T01:36:18.142Z)

```

20 def process_video(self, video_id: str, video_path: str, player_pool: List[str] =
None, max_frames: int = None) -> List[Dict[str, Any]]:
21 """
22 Processes a video to detect play segments using motion heuristics,
23 populates SQLite with detected segments, and indexes detailed events.
24 """
25 if not player_pool:
26 player_pool = ["Avi", "Suresh", "Ramesh", "Deepak"]
27
28 cap = cv2.VideoCapture(video_path)
29 if not cap.isOpened():
30 logger.error(f"Segmenter could not open video: {video_path}")
31 return Failure(message=f"Could not open video: {video_path}",
is_recoverable=False)
32
33 fps = cap.get(cv2.CAP_PROP_FPS)
34 total_frames = int(cap.get(cv2.CAP_PROP_FRAME_COUNT))
35 duration = total_frames / fps if fps > 0 else 0
36
37 # Sub-sample settings (5 FPS for analysis)
38 analysis_fps = 5.0
39 frame_interval = max(1, int(fps / analysis_fps))
40
41 idx_cfg = self.config.get("indexing", {})
42 motion_threshold = idx_cfg.get("motion_threshold", 0.08)
43 min_segment_duration = idx_cfg.get("min_segment_duration", 3.0)
44 dead_time_merge_gap = idx_cfg.get("dead_time_merge_gap", 2.0)
45
46 prev_gray = None
47 motion_signals = []
48 timestamps = []
49
50 frame_idx = 0
51 processed_count = 0
52 t_start = time.time()
53 # Emit a progress line roughly every 5% of the video so long runs are
54 # observable instead of silent (a full 1080p match is ~tens of minutes).

```

```

55 progress_step = max(1, total_frames // 20) if total_frames > 0 else 0
56 next_progress = progress_step
57 logger.info(f"[motion] start: {total_frames} frames @ {fps:.1f}fps "
58 f"({duration/60:.1f} min), analyzing every {frame_interval}th
frame")
59 while True:
60 # Skip decoding for intermediate frames using cap.grab() for high
performance
61 for _ in range(frame_interval - 1):
62 if not cap.grab():
63 break
64 frame_idx += 1
65
66 ret, frame = cap.read()
67 if not ret:
68 break
69
70 # Process frame
71 gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
72 # Apply Gaussian Blur to smooth noise
73 gray = cv2.GaussianBlur(gray, (21, 21), 0)
74
75 t = frame_idx / fps
76 timestamps.append(t)
77
78 if prev_gray is not None:
79 # Frame absolute difference (highly optimized motion tracking)
80 diff = cv2.absdiff(prev_gray, gray)
81 _, thresh = cv2.threshold(diff, 25, 255, cv2.THRESH_BINARY)
82
83 # Dilate to fill gaps
84 thresh = cv2.dilate(thresh, None, iterations=2)
85
86 # Motion ratio
87 motion_ratio = cv2.countNonZero(thresh) / (thresh.shape[0] *
thresh.shape[1])
88 motion_signals.append(motion_ratio)
89 else:
90 motion_signals.append(0.0)
91
92 prev_gray = gray
93 frame_idx += 1
94 processed_count += 1
95
96 if progress_step and frame_idx >= next_progress:
97 pct = 100.0 * frame_idx / total_frames
98 elapsed = time.time() - t_start
99 eta = elapsed / (frame_idx / total_frames) - elapsed if frame_idx else 0
100 logger.info(f"[motion] {pct:5.1f}% ? {frame_idx}/{total_frames} frames,
"
101 f"{processed_count} analyzed, {elapsed:.0f}s elapsed,
~{eta:.0f}s left")
102 next_progress += progress_step
103
104 if max_frames is not None and processed_count >= max_frames:
105 logger.debug(f"Reached max_frames limit of {max_frames}. Halting indexer
frame parsing.")
106 break
107
108 if frame_idx >= total_frames:
109 break
110
111 cap.release()
112
113 # Smooth signal using a moving average

```

```

114 window_size = 5
115 smoothed = []
116 for i in range(len(motion_signals)):
117 start = max(0, i - window_size // 2)
118 end = min(len(motion_signals), i + window_size // 2 + 1)
119 smoothed.append(sum(motion_signals[start:end]) / (end - start))

```

## 71. assistant (2026-06-18T01:36:35.013Z)

Same pattern: ``cap.release()`` after a loop that could raise in cv2 calls. Consistent with the codebase style and these are GPU/cv2-only paths.

I've now done a thorough cross-cut across all production subprocess/file/DB code. Let me reconsider whether any of these rise to a "real defect a maintainer would call a bug" with a concrete trigger.

The cv2 ``cap.release()``-skipped-on-exception pattern appears in many places but is consistent and the exceptions propagate up to fail the run (the process is short-lived per video). Not a strong finding.

Let me look at one thing I want to double-check: the ``_get_connection`` PRAGMA failure leak and the ``extract_frames`` cap leak. But these are edge cases. Let me instead verify a genuinely concrete concern ? the ``upload_part`` async function doing synchronous blocking file I/O, and re-examine whether ``st["bytes"]`` accounting has a real bug.

Looking at ``upload_part`` again carefully:

```

```python
received = 0
overflow = None
with open(st["tmp_path"], "ab") as fh:
    async for chunk in request.stream():
        received += len(chunk)
        if received > part_bytes: overflow=...; break
        if st["bytes"] + received > max_bytes: overflow=...; break
        fh.write(chunk)
...

```

There's a subtle but real bug here: ``received`` is incremented BEFORE the overflow checks, but the chunk is only written AFTER the checks pass. So when the part is exactly at the limit, the last chunk that pushes ``received`` over is NOT written (good ? it breaks). But on the success path, ``received`` correctly counts all written bytes since the break happens before ``fh.write``. Wait ? actually there IS a discrepancy. Let me re-read: ``received += len(chunk)`` counts the chunk, then if over limit, break WITHOUT writing. So ``received`` includes the un-written chunk's bytes. But on overflow we abort the whole upload, so it doesn't matter. On the non-overflow path, every counted chunk is written. The accounting (``st["bytes"] += received`` at line 496) is only reached on success where all received bytes were written. Correct.

I've thoroughly reviewed the codebase. Let me do one last check on ``tracknet_runner.run_predict`` cleanup path and the ``parse_trajectory_csv`` ? these are clean (context managers, swallowed errors).

After this comprehensive cross-cut by bug class (subprocess returncode handling, file handles, sqlite connections/cursors, temp files, decode errors, unquoted paths), I found the code to be remarkably well-hardened. The subprocess calls consistently use ``check=True`` with proper exception handling, ``capture_output`/`DEVNULL`` for stderr, paths passed as argv lists (no shell injection / unquoted-space issues), DB access through a closing context manager, and temp files via ``TemporaryDirectory`/`mkstemp`` with cleanup.

The only patterns worth noting (cv2 ``VideoCapture.release()`` skipped if a cv2 call raises mid-loop) are consistent across the codebase, run only on GPU/cv2 machines, in short-lived per-video processes, and propagate the exception to fail the run ? a maintainer would treat these as cosmetic at most, not a real defect with a concrete harmful trigger. They do not meet the precision bar.

Consistent with the prior precision pass, I report an empty list.

72. user (2026-06-18T01:36:35.587Z)

Structured output provided successfully

73. assistant (2026-06-18T01:36:50.145Z)

I completed a cross-cutting hunt by bug class across all production subprocess / file / DB code paths in the repo (excluding the four excluded eval files).

Files reviewed (concrete, line-by-line)

```
**Subprocess (ffmpeg/ffprobe/NVENC/WSL):**
- `backend/utils/video.py`, `backend/tools/rally_slicer.py`,
`backend/pipeline/stitcher/compiler.py`, `backend/pipeline/audio/hit_detector.py`
- `backend/pipeline/detectors/{base.py,tracknet_runner.py,wasb_runner.py,wasb_infer.py}`
- `backend/pipeline/validators/{format.py,pts_continuity.py,resolution_fps.py}`
- `backend/pipeline/segmenters/{gemini.py,yolo_hybrid.py,motion.py}`,
`backend/pipeline/detectors/person_detector.py`
- `backend/utils/run_telemetry.py`, `scripts/doctor.py`, `tools/cleanup_caches.py`,
`training/gen0/harness.py`, `backend/main.py`

**DB (sqlite connections/cursors/transactions):**
- `backend/infrastructure/database.py` (full read ? `_connect` closing context manager, UPSERT
vs REPLACE, compare-and-set `claim_due_job`, watermark prune)

**File / temp / CSV / JSON I/O:**
- `backend/utils/hashing.py`, `backend/eval/{export_wasb_segments.py,gt_loader.py,annotations.py
,classifiier.py,rally_seg_eval.py,splits.py}` and the JSON load/save sites across the eval
package.
```

Findings: none

The code is well-hardened against this bug class. Subprocess calls consistently use `check=True` with `CalledProcessError`/`TimeoutExpired` handling, capture or `DEVNULL` stderr, and pass paths as argv lists (no shell, so unquoted-space/injection is a non-issue); ffmpeg filtergraph operands are explicitly colon/quote-escaped (`\_ff\_quote`). DB access flows through a single `\_connect` context manager that both transaction-scopes and closes the connection. Temp files use `TemporaryDirectory`/`mkstemp` with cleanup; cache writes are atomic (tmp + `os.replace`, `fsync`).

The one recurring pattern I weighed and rejected: several `cv2.VideoCapture` users (`person\_detector.py`, `motion.py`, `wasb\_infer.extract\_frames`) skip `cap.release()` if a cv2 call raises mid-loop. It is consistent across the codebase, runs only on short-lived per-video GPU processes, and propagates the exception to fail the run ? cosmetic at most, below the precision bar for a reported defect.

Consistent with the prior precision pass, I return an empty list.